

Numerical Methods

Zusammenfassung der Vorlesungsnotizen von Prof. Dr.-Ing. Mark Schutera

Lernmaterial-Projekt*

16. Juni 2026

Inhaltsverzeichnis

*Basierend auf den „unfinished lecture notes“ (CC BY-NC-SA 4.0), github.com/Quillstacks/LectureMaterial.
Nachweislich fehlerhafte Stellen des Originals sind als [Korrektur ggü. Original] markiert, eigene didaktische
Zusätze als [Ergänzung].

Einleitung (S. 3–7)

Dem Skript vorangestellt ist das Motto (S. 3):

„An infinitely accurate approximation is no longer an approximation.“
— *probably someone smart*

Die Introduction (S. 7) steckt den Rahmen ab: Gegenstand des Kurses sind die numerischen Methoden – Verfahren, die mathematische Probleme näherungsweise lösen, wo exakte (analytische) Lösungen nicht verfügbar oder nicht praktikabel sind. Behandelt werden die fundamentalen Konzepte: Fehleranalyse (error analysis), Konditionierung von Problemen (problem conditioning), Stabilität (stability) sowie verschiedene numerische Techniken [Ergänzung: etwa zur Nullstellensuche, Optimierung und Integration – so die Kapitel des Skripts]. Ziel ist, die Fähigkeiten zu vermitteln, zuverlässige und effiziente Algorithmen im Scientific Computing und in Ingenieursanwendungen zu implementieren („to equip you with the skills needed to implement reliable and efficient algorithms in scientific computing and engineering applications“). Das Kursversprechen: Der Kurs vermittelt alles Nötige, um mit numerischen Methoden sicher umzugehen („everything you need to know to become proficient“) und sie in Machine Learning, Data Science und Ingenieursanwendungen (machine learning, data science, and engineering contexts) gewinnbringend einzusetzen. [Ergänzung: In diesen Feldern beruht praktisch jede Berechnung – vom Training neuronaler Netze bis zur Simulation – auf numerischen Approximationen.]

1 Enter Numerical Methods (S. 9–16)

1.1 Motivation und Einordnung

Numerische Methoden (numerical methods) sind ein Teilgebiet der Mathematik, in dem Lösungen nicht analytisch exakt, sondern approximativ berechnet werden – das Skript fasst dies unter dem Motto *Tapping into Computational Power* zusammen (S. 9). Es gibt gute Gründe für dieses Vorgehen: Viele Probleme sind analytisch gar nicht lösbar oder zu komplex, um praktikabel zu sein, und wir können Rechenleistung (computational power) anzapfen, um approximative Lösungen effizient zu erhalten (S. 9). In Machine Learning und künstlicher Intelligenz sind numerische Methoden entscheidend für das Training von Modellen, die Optimierung von Parametern und die Simulation komplexer Systeme, wo analytische Lösungen unmöglich (infeasible) sind; sie ermöglichen den effizienten Umgang mit großen Datensätzen und komplexen Algorithmen und stellen sicher, dass Modelle effektiv aus Daten lernen können, während die Rechenressourcen (computational resources) im Rahmen gehalten werden (S. 9). Besonders im Deep Learning, wo Modelle Millionen Parameter umfassen und umfangreiche Berechnungen erfordern, erleichtern numerische Methoden die Optimierungsprozesse, die dem Modelltraining zugrunde liegen – sie sind damit unverzichtbar für den Fortschritt dieser Modelle (S. 9).¹

Zwei Randnotizen des Skripts ordnen den Begriff historisch und technisch ein: Das Wort *Approximation* stammt vom lateinischen *approximare*, „to come near to“ (sich annähern) (S. 9). Und das *Moore'sche Gesetz* (Moore's Law) besagt, dass sich die Rechenleistung etwa alle zwei Jahre verdoppelt; heutige Consumer-GPUs haben Tausende Kerne und schaffen Billionen Gleitkommaoperationen pro Sekunde (S. 9).²

¹Further Reading (S. 9): T. Arens et al.: *Mathematik*, Springer; W. Dahmen, A. Reusken: *Numerik für Ingenieure und Naturwissenschaftler*, Springer; M. P. Deisenroth, A. A. Faisal, C. S. Ong: *Mathematics for Machine Learning*, Cambridge University Press; P. Deuffhard, A. Hohmann: *Numerische Mathematik 1 – Eine algorithmisch orientierte Einführung*, De Gruyter.

²G. E. Moore: *Cramming more components onto integrated circuits*, Electronics, 38(8):114–117, 1965. Als modernes Beispiel verweist das Skript auf den GPT-2-Release: <https://openai.com/index/gpt-2-1-5b-release/> (S. 9).

1.2 Lernziele (S. 11)

1. Erklären können, wann numerische Methoden eingesetzt werden und welche Probleme beim analytischen Lösen auftreten.
2. Diskretisierung (discretization) verstehen: kontinuierliche mathematische Probleme in diskrete, computerlösbare Approximationen transformieren.
3. Grundlegende numerische Techniken auf einfache Probleme von Hand anwenden und größere Probleme am Computer *crunchen*.

1.3 Hands-On: Die Grenzen analytischer Lösungen (S. 10–11)

Bevor die Theorie kommt, vermittelt das Skript ein Gefühl dafür, warum AI/ML so stark auf numerischen Methoden beruht: Die *Grenzen der Skalierung analytischer Lösungen* (limits of scaling analytical solutions) werden bei großskaligen Problemen offensichtlich (S. 10). Als Einstieg dient ein lineares 2×2 -Gleichungssystem, das analytisch per Substitution gelöst wird. Unter *Analytik* versteht das Skript dabei Methoden wie Substitution, Elimination, Matrixinversion etc. aus der linearen Algebra (Randnotiz, S. 10).

Definition 1.1: Numerische Methoden (informell)

„Numerical methods are a subfield of mathematics in which we calculate our solutions not analytically exactly, but approximately.“ – Numerische Methoden sind ein Teilgebiet der Mathematik, in dem Lösungen nicht analytisch exakt, sondern *approximativ* berechnet werden. Sie sind essenziell zur Lösung mathematischer Probleme, die analytisch nicht adressiert werden können (S. 9; vgl. Introduction, S. 7).

Beispiel 1.1: Analytische Lösung eines 2×2 -Systems per Substitution (S. 10)

Gegeben ist das lineare Gleichungssystem (Gl. 1 im Skript, S. 10):

$$\begin{bmatrix} 2 & 1 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 11 \\ 13 \end{bmatrix}$$

Vollständige Substitutionsrechnung:

- Aus der ersten Gleichung: $2w_1 + w_2 = 11 \Rightarrow w_2 = 11 - 2w_1$
- Einsetzen in die zweite Gleichung: $5w_1 + 1(11 - 2w_1) = 13$
- $5w_1 + 11 - 2w_1 = 13$
- $3w_1 + 11 = 13$
- $3w_1 = 2$
- $w_1 = \frac{2}{3}$
- Rückeinsetzen: $w_2 = 11 - 2\left(\frac{2}{3}\right) = 11 - \frac{4}{3} = \frac{33 - 4}{3} = \frac{29}{3}$

Verifikation:

$$1. \text{ Gleichung: } 2\left(\frac{2}{3}\right) + \frac{29}{3} = \frac{4}{3} + \frac{29}{3} = \frac{33}{3} = 11 \checkmark \quad 2. \text{ Gleichung: } 5\left(\frac{2}{3}\right) + 1\left(\frac{29}{3}\right) = \frac{10}{3} + \frac{29}{3} = \frac{39}{3} = 13 \checkmark$$

However (S. 11): Für zwei Gleichungen ist das gut lösbar, aber mit wachsender Systemgröße (z. B. Tausende Gleichungen mit Tausenden Unbekannten) werden analytische Lösungen wegen

der Rechenkomplexität (computational complexity) und Zeitbeschränkungen unpraktikabel – die Komplexität wächst mit der Systemgröße (Randnotiz, S. 10). Als Marginalien-Übung stellt das Skript dazu ein größeres 3×3 -System (Gl. 2 im Skript, S. 10; siehe Übungsaufgaben unten).

Analytische Lösungen stoßen auch bei *nichtlinearen* Gleichungen an Grenzen. Das Skript zeigt das an der Sigmoid-Funktion:

Satz/Formel 1.1: Sigmoid-Funktion (Gl. 3 im Skript, S. 11)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

In eigenen Worten: Die Sigmoid-Funktion bildet jede reelle Zahl x auf das offene Intervall $(0, 1)$ ab. Der Exponentialterm e^{-x} im Nenner macht es unmöglich, x mit elementaren Funktionen (Polynome, rationale, trigonometrische Funktionen) zu isolieren – man kann die Gleichung also nicht *nach x auflösen*. Laut Randnotiz (S. 11) ist $\sigma(x)$ die in neuronalen Netzen gebräuchliche Sigmoid-Aktivierungsfunktion und eine transzendente Funktion; eine geschlossene Lösung für $\sigma(x) = 0$ existiert nicht: $\sigma(x)$ nähert sich 0 asymptotisch für $x \rightarrow -\infty$, erreicht 0 aber für keinen endlichen Wert von x (Grenzfall/Warnung: die Nullstelle existiert gar nicht!).

Definition 1.2: Transzendente Funktionen (transcendental functions)

Transzendente Funktionen können nicht als endliche Kombinationen algebraischer Operationen (Addition, Subtraktion, Multiplikation, Division, Wurzeln) ausgedrückt werden und besitzen daher keine geschlossenen Lösungen (closed-form solutions) (S. 11).

1.4 Diskretisierung (S. 12)

Definition 1.3: Diskretisierung (Discretization)

Diskretisierung bedeutet, kontinuierliche Domänen (Zeit, Raum oder andere Funktionen) in endliche Schritte oder Gitter (grids) zu zerlegen und die Funktionen an diskreten Punkten mit endlicher Präzision auszuwerten (S. 12):

- Kontinuierliche Funktion: $f(x)$, $x \in [a, b]$
- Diskretisierte Funktion: $f(x_i)$, $x_i = a + ih$, $i = 0, 1, \dots, N$

Satz/Formel 1.2: Schrittweite (step size) (Gl. 4 im Skript, S. 12)

$$h = \frac{b - a}{N}$$

In eigenen Worten: Die Schrittweite h ergibt sich, indem man die Länge des Intervalls $[a, b]$ gleichmäßig in N Schritte unterteilt; N ist die Anzahl der Schritte auf dem Intervall. Kleines h (also großes N) bedeutet ein feineres Gitter – höhere Genauigkeit, aber mehr Auswertungen und damit mehr Rechenzeit.

Eine Randnotiz erinnert daran, dass die Klammern $[$ und $]$ ein *geschlossenes Intervall* (closed interval) bezeichnen: Die Randwerte a und b sind in der Definitionsmenge enthalten (Remember, S. 12). Das Skript illustriert die Diskretisierung mit Figure 1 (S. 12): die kontinuierliche Kurve $f(x) = \sin(x)$ und ihre diskretisierte Version (schwarze Punkte) auf $[-3, 1]$ mit Schrittweite $h = 1$.

1.5 Beispiel: Minimumsuche analytisch vs. numerisch (S. 12–13)

Beispiel 1.2: Analytische Minimumsuche von $\sin(x)$ auf $[-3, 1]$ (S. 12–13)

Gesucht ist $\min_{x \in [-3, 1]} \sin(x)$. Das Minimum tritt auf, wo die Ableitung verschwindet und die zweite Ableitung positiv ist:

$$f(x) = \sin(x), \quad f'(x) = \cos(x) = 0 \implies x^* = \frac{\pi}{2} + k\pi, \quad k \in \mathbb{Z}$$

(Die allgemeine Lösung von $\cos(x) = 0$ liefern die trigonometrischen Regeln; falls vergessen, lässt sie sich am Einheitskreis herleiten – Randnotiz, S. 12.) Kritische Punkte in $[-3, 1]$:

$$x_1 = -\frac{\pi}{2} \approx -1.5708, \quad x_2 = \frac{\pi}{2} \approx 1.5708 \quad (> 1, \text{ nicht im Intervall – Grenzfall!})$$

Prüfung der Endpunkte und von x_1 :

$$\sin(-3) \approx -0.1411, \quad \sin\left(-\frac{\pi}{2}\right) = -1, \quad \sin(1) \approx 0.8415$$

Ergebnis: Das Minimum ist -1 bei $x = -\frac{\pi}{2} \approx -1.5708$. (Vgl. Figure 2, S. 12: Sinuskurve, Ableitung $\cos(x)$ gestrichelt, analytisches Minimum als schwarzer Punkt; die gestrichelte graue Linie markiert den kritischen Punkt bei $x = -\pi/2$.)

Definition 1.4: Extrema auf geschlossenem Intervall (Stolperfalle)

Das Minimum (oder Maximum) einer Funktion auf einem geschlossenen Intervall $[a, b]$ kann an einem kritischen Punkt (Ableitung null oder nicht definiert) im Inneren *oder* an den Endpunkten a, b auftreten – auch wenn dort die Ableitung nicht null ist. Deshalb müssen bei der Extremasuche auf geschlossenen Intervallen stets kritische Punkte *und* Endpunkte geprüft werden (S. 13).

On to the numerical solution (S. 13): Statt analytisch abzuleiten, wertet man die Funktion an diskreten Punkten über dem Intervall aus und wählt den Punkt mit minimalem Wert – eine Brute-Force-Gittersuche.

Definition 1.5: Brute Force / Gittersuche (grid search)

Brute Force bedeutet, alle möglichen Optionen auszuprobieren und die beste zu wählen – hier umgesetzt per Gittersuche (grid search): Die Funktion wird an allen Gitterpunkten ausgewertet, und der beste (z. B. minimale) Wert wird ausgewählt (Randnotiz, S. 13).

Beispiel 1.3: Numerische Minimumsuche von $\sin(x)$ per Grid Search (S. 13)

Diskretisierung von $[-3, 1]$ mit Schrittweite $h = 1$ und Auswertung an allen Gitterpunkten (vgl. Figure 3, S. 13, die wie Figure 1 die Sinuskurve mit den diskreten Stützstellen zeigt):

x_i	Funktionswert
$x_0 = -3$	$\sin(-3) \approx -0.1411$
$x_1 = -2$	$\sin(-2) \approx -0.9093$
$x_2 = -1$	$\sin(-1) \approx -0.8415$
$x_3 = 0$	$\sin(0) = 0$
$x_4 = 1$	$\sin(1) \approx 0.8415$

Ergebnis: „the minimum among these is $\sin(-2) \approx -0.9093$ at $x = -2$.“ Der Approximationsfehler gegenüber der analytischen Lösung beträgt

$$|-1 - (-0.9093)| = 0.0907.$$

Die Wahl der Schrittweite h beeinflusst die Genauigkeit der Approximation: Eine kleinere Schrittweite führt näher ans wahre Minimum heran (Randnotiz, S. 13). Auch die Intervallwahl spielt eine Rolle.^a

^aDie zugehörige Randnotiz auf S. 13 bricht im Original mitten im Satz ab („Further the interval selection matters, in a sense that it “); der Rest des Gedankens ist nicht rekonstruierbar und wird hier weggelassen.

Definition 1.6: Approximationsfehler (Approximation Error)

Der Approximationsfehler ist die Differenz zwischen analytischer und numerischer Lösung (Randnotiz, S. 13). Im Sinus-Beispiel konkret: $|-1 - (-0.9093)| = 0.0907$.

1.6 Grid Search für das lineare System (S. 14–15)

Auch das lineare Gleichungssystem von oben lässt sich per Brute-Force-Diskretisierung und Approximation lösen. Das Skript schreibt das System dazu mit den Variablen w und b (Gl. 5 im Skript, S. 14):

$$\begin{bmatrix} 2 & 1 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} w \\ b \end{bmatrix} = \begin{bmatrix} 11 \\ 13 \end{bmatrix}$$

Die Bezeichner sind kein Zufall: Eine Randnotiz (S. 14) weist darauf hin, dass $xw + b$ ein lineares Modell (linear regression) bildet, das man auch als die einfachste Form eines neuronalen Netzes mit einer einzigen Einheit θ auffassen kann – der direkte ML-Anwendungsbezug.

Satz/Formel 1.3: Fehlerdefinition der Grid Search (Gl. 6 im Skript, S. 14)

$$\text{error}_1 = |2w + b - 11|, \quad \text{error}_2 = |5w + b - 13|$$

In eigenen Worten: Für jede Kandidatenkombination (w, b) misst man, wie stark die linke Seite jeder Gleichung von der rechten Seite abweicht – als Betrag der Differenz, damit sich positive und negative Abweichungen nicht aufheben. Der Gesamtfehler (akkumulierter Fehler) ist die Summe der absoluten Differenzen beider Gleichungen. Man wertet alle Kombinationen aus und wählt das (w, b) -Paar mit dem kleinsten akkumulierten Fehler.

Beispiel 1.4: Grid Search für das lineare System, Schrittweite 2.5 (S. 14–15, Tab. 1)

Die Variablen w und b werden über das Gitter $w, b \in \{0, 2.5, 5, 7.5, 10\}$ diskretisiert (Schrittweite 2.5); für jede der $5 \times 5 = 25$ Kombinationen werden $2w + b$ und $5w + b$ samt Fehlern nach Gl. 6 berechnet. Tabelle 1 des Skripts (S. 15) vollständig – Werte von $2w + b$ und $5w + b$ mit Fehlern zu 11 bzw. 13 in Klammern; die letzte Spalte zeigt den akkumulierten Fehler (Summe der absoluten Fehler):

w	b	$2w + b$ (error ₁)	$5w + b$ (error ₂)	Error
0	0	0 (11)	0 (13)	24
0	2.5	2.5 (8.5)	2.5 (10.5)	19
0	5	5 (6)	5 (8)	14
0	7.5	7.5 (3.5)	7.5 (5.5)	9
0	10	10 (1)	10 (3)	4*
2.5	0	5 (6)	12.5 (0.5)	6.5
2.5	2.5	7.5 (3.5)	15 (2)	5.5
2.5	5	10 (1)	17.5 (4.5)	5.5
2.5	7.5	12.5 (1.5)	20 (7)	8.5
2.5	10	15 (4)	22.5 (9.5)	13.5
5	0	10 (1)	25 (12)	13
5	2.5	12.5 (1.5)	27.5 (14.5)	16
5	5	15 (4)	30 (17)	21
5	7.5	17.5 (6.5)	32.5 (19.5)	26
5	10	20 (9)	35 (22)	31
7.5	0	15 (4)	37.5 (24.5)	28.5
7.5	2.5	17.5 (6.5)	40 (27)	33.5
7.5	5	20 (9)	42.5 (29.5)	38.5
7.5	7.5	22.5 (11.5)	45 (32)	43.5
7.5	10	25 (14)	47.5 (34.5)	48.5
10	0	20 (9)	50 (37)	46
10	2.5	22.5 (11.5)	52.5 (39.5)	51
10	5	25 (14)	55 (42)	56
10	7.5	27.5 (16.5)	57.5 (44.5)	61
10	10	30 (19)	60 (47)	66

Ergebnis: Der minimale Fehler tritt bei $w = 0$, $b = 10$ mit einem Fehler von 4 auf (mit * markiert).

[Ergänzung] Vergleich mit der wahren Lösung: Das Grid-Search-Minimum $w = 0$, $b = 10$ weicht stark von der exakten Lösung $w = 2/3$, $b = 29/3 \approx 9.67$ ab – das illustriert die Grenzen grober Gitter. $b = 10$ liegt zwar nahe am wahren b , w ist jedoch deutlich falsch. Ein feineres Gitter (kleinere Schrittweite) würde die Approximation verbessern, aber den Rechenaufwand quadratisch in der Zahl der Gitterpunkte pro Variable erhöhen.

Der Wert des Handrechnens (S. 14). „There is value in doing this by hand“ – die Handrechnung hilft, die Mechanik numerischer Methoden zu verstehen, und gibt eine Intuition über die Rechenkosten von Brute-Force-Methoden („This was only 100 operations“; später im Kurs folgen effizientere Ansätze). Die Botschaft: Genau so etwas will man an den Computer übergeben und automatisieren – der mit dieser Problemgröße nicht einmal ansatzweise Probleme hat.

Enter the machine (S. 14). Viele numerische Methoden sind von vornherein für die Computer-Implementierung konzipiert. Im Kurs wird zwischen Handrechnungen (kleine Beispiele) und Computer-Implementierungen (größere Probleme) gewechselt. Zu diesem Kapitel gibt es ein vorgehostetes Jupyter Notebook mit einem Python-Template für die Übung; die Selbstreflexionsfragen dienen als Leitfaden beim Explorieren und Experimentieren.³ Eine Randnotiz zu den Übungen (S. 14, im Original „Excercises“ [sic]) gibt einen Lernhinweis: Übungen dienen der Übung und Festigung der Konzepte – erst selbst versuchen, Dinge ausprobieren, damit spielen, diskutieren; das ist kein Zeitrennen. Es ist keine Schande, nicht bei der richtigen Antwort zu landen: Gute Fragen aufzudecken und hin- und herzuwerfen ist langfristig meist sehr fruchtbar.

1.7 Recap of Key Concepts (S. 16)

- Numerische Methoden sind essenziell für komplexe mathematische Probleme ohne analytische Lösung.
- Diskretisierung transformiert kontinuierliche Probleme in diskrete Approximationen, die für rechnerische Methoden geeignet sind.
- Numerische Berechnungen von Hand lohnen sich für kleine Probleme (Verständnis der Mechanik); für größere Probleme sind Computer unverzichtbar.
- *Errors everywhere*: Mathematische Modelle sind Vereinfachungen der Realität, und numerische Methoden führen zusätzliche Fehler durch Approximation ein.

Satz/Formel 1.4: Approximation durch Diskretisierung (Gl. 7 im Skript, S. 16)

$$f(x) \approx f(x_i), \quad x_i = a + ih$$

In eigenen Worten: Die kontinuierliche Funktion $f(x)$ wird nur noch an den Gitterpunkten x_i ausgewertet – dazwischen kennen wir sie nicht. Genau dieses „ $\approx$ “ ist die Quelle des Approximationsfehlers. Die numerische Berechnung führt damit einige Fehlertypen ein, die man verstehen muss, „in order to be able to fully harness this new methodology“ – die Überleitung zu den Kapiteln 2 und 3.

Teaser (Randnotiz, S. 16). „Can you think of a simple way to improve the accuracy to compute ratio of our grid search example from above?“ – Gibt es einen einfachen Weg, das Verhältnis von Genauigkeit zu Rechenaufwand der Grid Search zu verbessern? [Ergänzung] Ein Denkanstoß in Richtung adaptiver bzw. verfeinerter Suche: erst grob suchen, dann nur um den besten Kandidaten herum mit feinerem Gitter weitersuchen.

1.8 Übungsaufgaben und Selbstreflexionsfragen

Übungen (nur Aufgabenstellung; Lösungen in den Übungsblättern):

1. (Randnotiz, S. 10) „Now take a shot and solve this larger system of three equations with three unknowns“ – löse das 3×3 -System (Gl. 2 im Skript, S. 10):

$$\begin{bmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \\ 3 & 2 & 5 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 14 \\ 20 \\ 32 \end{bmatrix}$$

2. (S. 14) Exploriere und experimentiere mit der Grid Search für Gl. 5/6 im vorgehosteten Jupyter Notebook (Python-Template).

³Code-Hosting laut Randnotiz (S. 14): <https://enlitenment.schutera.com/landing> (URL-Schreibweise im Original unsicher, evtl. *enlightenment*).

Self-Reflection (S. 15):

1. Warum ist die Wahl der Schrittweite h beim Diskretisieren einer kontinuierlichen Funktion wichtig, und wie beeinflusst sie die Genauigkeit und die Rechenzeit der numerischen Lösung?
2. Wie beeinflusst die Wahl des Intervalls $[a, b]$ die Ergebnisse der Diskretisierung und die Lage der numerisch gefundenen Extrema?
3. Was sind die Hauptunterschiede zwischen einer kontinuierlichen Funktion und ihrer diskretisierten Version, und welche Implikationen hat das für das numerische Lösen mathematischer Probleme?

Typische Fehlerquellen & Merksätze

Typische Fehlerquellen:

- Bei der Extremasuche auf geschlossenen Intervallen die *Endpunkte* vergessen – Extrema können auch am Rand liegen, wo die Ableitung nicht null ist (S. 13).
- Kritische Punkte außerhalb des Intervalls mitzählen: $x = \pi/2 \approx 1.5708 > 1$ liegt *nicht* in $[-3, 1]$ (Grenzfall!).
- Nach Nullstellen von Funktionen suchen, die gar keine haben: $\sigma(x) = 0$ hat keine Lösung – die Sigmoid-Funktion nähert sich 0 nur asymptotisch (S. 11).
- Grid-Search-Ergebnisse mit der exakten Lösung verwechseln: Bei grobem Gitter kann das gefundene Optimum ($w = 0, b = 10$) weit von der wahren Lösung ($w = 2/3, b = 29/3$) entfernt sein.
- Die Wirkung der Schrittweite unterschätzen: h steuert den Kompromiss zwischen Genauigkeit und Rechenaufwand.

Merksätze:

- Numerisch heißt: nicht exakt, sondern *approximativ* – dafür skalierbar mit Rechenleistung.
- Diskretisierung = kontinuierliches Problem $\rightarrow$ endliches Gitter: $x_i = a + ih$ mit $h = (b - a)/N$.
- Sobald man approximiert, gilt *errors everywhere*: $f(x) \approx f(x_i)$ – Fehler verstehen ist Pflicht, nicht Kür.
- Brute Force ist die einfachste numerische Methode: alles ausprobieren, das Beste behalten – teuer, aber ehrlich.

2 Floating-Point Arithmetic (S. 17–24)

2.1 Motivation und Einordnung

Unter dem Motto *Getting used to Errors Everywhere* (S. 17) knüpft das Kapitel an Thema 1 an: Numerische Methoden führen Fehler durch Approximation ein. Ist $f(x)$ eine kontinuierliche Funktion auf $[a, b]$, so approximiert die diskretisierte Version $f(x_i)$ die Funktion an diskreten Punkten x_i mit einem Fehler, der von der Schrittweite h und der Glattheit (smoothness) von f abhängt. Hinzu kommt: Numerische Werte können im Computerspeicher selbst nur approximativ gespeichert werden – in Gleitkommadarstellung (floating-point representation). Das führt zu Rundungsfehlern (rounding errors) bei arithmetischen Operationen, die sich zu den Abschneide- bzw. Trunkierungsfehlern (truncation errors) addieren, welche beim Approximieren unendlicher

Prozesse durch endliche entstehen (auch Approximationsfehler genannt) (S. 17). Es gibt gute Gründe, diese Fehler und ihr Zusammenspiel mit den Maschinen zu verstehen: Numerische Methoden führen Rundungs- und Abschneidefehler ein, und je nach Maschine wirken sich diese Fehler unterschiedlich aus – sie können sich verstärken (amplify) und akkumulieren (accumulate) (S. 17).

Bezug zu ML/AI (S. 17): Numerische Methoden sind fürs Training zentral, Gleitkomma-Arithmetik ist aber genauso relevant in der Modell-Inferenz (model inference) – besonders in rechenarmen Umgebungen *on the edge* oder beim Einsatz quantisierter Modelle (quantized models). Eine Randnotiz ergänzt: On-the-Edge-Modelle nutzen niedrigpräzise Arithmetik, z. B. 8-Bit-Integer oder sogar binäre Gewichte und Aktivierungen; bei binären neuronalen Netzen (binary neural networks, BNNs) sind Gewichte und Aktivierungen auf $\{-1, +1\}$ beschränkt – eine drastische Reduktion von Speicher- und Rechenbedarf, die aber Gleitkommadarstellung und Rundungseffekte umso kritischer macht (S. 17).⁴

Definition 2.1: Modellierungsfehler (Modeling Error)

Der Modellierungsfehler tritt auf, da alle Modelle Vereinfachungen der Realität sind; die Differenz zwischen Modell und realem System führt einen Fehler ein. Er wird in diesem Kurs nicht vertieft – „Yet, mind the fine difference between what we term $f(x)$ and what actually is a $f(x)$.“ (den feinen Unterschied beachten zwischen dem, was wir $f(x)$ *nennen*, und dem, was tatsächlich ein $f(x)$ *ist*) (Randnotiz, S. 17).

2.2 Lernziele (S. 18)

1. Die verschiedenen Typen numerischer Fehler verstehen: Modellierungs-, Abschneide- und Rundungsfehler (modeling, truncation, rounding errors).
2. Gleitkommadarstellung (floating-point representation), Maschinengenauigkeit (machine epsilon) und Auslöschung (loss of significance) verstehen.
3. Numerische Fehler in praktischen Berechnungen handhaben können.

2.3 Hands-On: Rundungsfehler (S. 18)

Beispiel 2.1: Division $10 \div 3$ – warum Rundungsfehler unvermeidlich sind (S. 18)

Schriftliche Division mit vollständiger Dezimalentwicklung:

- $10 \div 3 = 3$, Rest 1
- „Bring down a 0“: $10 \div 3 = 3$, Rest 1
- „Bring down a 0“: $10 \div 3 = 3$, Rest 1
- „Bring down a 0“: $10 \div 3 = 3$, Rest 1
- ... und so weiter $\Rightarrow \frac{10}{3} = 3.333\dots$ – die Dreien wiederholen sich unendlich.

Beim Aufschreiben oder Eingeben in Rechner/Computer muss man irgendwo stoppen (Gl. 8 im Skript, S. 18):

$$\frac{10}{3} \approx 3.333 \quad (\text{gerundet bzw. abgeschnitten/trunkiert nach 3 Ziffern})$$

„No matter how many digits you write, as soon as you stop, you introduce an error“ (Gl. 9

⁴M. Courbariaux, Y. Bengio: *Binarynet: Training deep neural networks with weights and activations constrained to +1 or -1*, CoRR, abs/1602.02830, 2016, <http://arxiv.org/abs/1602.02830> (Fußnote 5 im Skript, S. 17).

im Skript, S. 18):

$$\text{Rounding error} = |3.333333 \dots - 3.333| = 0.000333 \dots$$

Je mehr Ziffern man schreibt, desto kleiner der Fehler; er verschwindet aber nie vollständig – außer man schreibe unendlich viele Ziffern, „which well, you know is impossible.“^a

^aRandnotiz *Impossible?* (S. 18): Die mathematische Notation hilft hier mit der Periodenschreibweise: $3.\overline{3}$.

Definition 2.2: Rundungsfehler (rounding error)

Der Rundungsfehler ist die Differenz zwischen dem wahren Wert und dem Wert, den man nach dem Abschneiden (truncate) der Entwicklung erhält (S. 18). Computer speichern Zahlen stets mit endlich vielen Ziffern, daher treten Rundungsfehler unvermeidlich auf und müssen gemanagt werden. Diese kleinen Fehler können akkumulieren, sich verstärken und zu signifikanten Ungenauigkeiten führen – besonders in iterativen Algorithmen, wie sie in numerischen Methoden und Machine Learning üblich sind.

Eine Randnotiz (S. 18) listet die **Typen numerischer Darstellungen in Computern** am Beispiel von 10/3:

Darstellung	Beispielwert	Präzision [Ergänzung]
Integer (int)	3	ganzzahlig
Floating-point (float)	3.3333333	≈ 7 signifikante Dezimalstellen
Double precision (double)	3.3333333333333333	≈ 16 signifikante Dezimalstellen
Fixed-point (z. B. 4 Stellen)	3.3333	4 Nachkommastellen

[Ergänzung: Die Spalte „Präzision“ ist nicht Teil der Original-Randnotiz (dort stehen nur die Beispielwerte); bei float/double fallen signifikante Stellen und Nachkommastellen im Beispiel 10/3 nur zufällig zusammen, maßgeblich sind die signifikanten Dezimalstellen.]

2.4 Gleitkommadarstellung (S. 19–20)

Definition 2.3: Gleitkommazahlen (floating-point numbers) und IEEE 754

Gleitkommazahlen sind eine Methode für Computer, reelle Zahlen mit einer endlichen Anzahl von Bits darzustellen.^a Der IEEE-754-Standard ist das am weitesten verbreitete Format.^b Die Darstellung erlaubt einen weiten Wertebereich, aber nur eine *endliche Menge* reeller Zahlen ist exakt darstellbar. IEEE 754 single precision (float) umfasst 32 Bit: 1 Vorzeichenbit + 8 Exponentenbits + 23 Mantissenbits (S. 19).

^aEin *Bit* (kurz für *binary digit*) ist die grundlegendste Informationseinheit in Computertechnik und digitaler Kommunikation; es kann den Wert 0 oder 1 haben (Randnotiz, S. 19).

^bIEEE Computer Society: *IEEE standard for floating-point arithmetic*, IEEE Std 754-2008 (Revision von IEEE Std 754-1985), August 2008, <https://ieeexplore.ieee.org/document/4610935> (Fußnote 6 im Skript, S. 19).

Satz/Formel 2.1: Gleitkommadarstellung (Gl. 10 im Skript, S. 19)

$$x = (-1)^s \cdot m \cdot 2^e$$

In eigenen Worten: Eine Gleitkommazahl besteht aus drei Teilen: s ist das Vorzeichenbit

(sign bit) und bestimmt über $(-1)^s$, ob die Zahl positiv oder negativ ist; m ist die Mantisse (mantissa, auch significand) und bestimmt die *Präzision*, also die gültigen Ziffern; e ist der Exponent und bestimmt die *Skala* (Größenordnung/magnitude) der Zahl, indem er die Mantisse mit einer Zweierpotenz multipliziert – das Komma „gleitet“ je nach Exponent.

Die Bit-Aufteilungen der gängigen IEEE-754-Formate zeigt das Skript in den Figures 4–6 (S. 19):

Format	Bits gesamt	Sign	Exponent	Mantisse
IEEE 754 HALF (16)	16	1	5	10
IEEE 754 SINGLE (32)	32	1	8	23
IEEE 754 DOUBLE (64)	64	1	11	52

Satz/Formel 2.2: Exponent mit Bias (S. 19)

$$e = E - 127_{10} \quad (\text{IEEE 754 single precision: 8 Exponentenbits, Bias 127})$$

In eigenen Worten: Der Exponent wird nicht direkt, sondern mit einem Bias gespeichert, um positive *und* negative Exponenten zu ermöglichen – so lassen sich sehr kleine und sehr große Größenordnungen darstellen. Mit 8 Bits speichert das Exponentenfeld Werte von 0 ($2^0 - 1$) bis 255 ($2^8 - 1$); durch Subtraktion des Bias (127) kann der tatsächliche Exponent e positive und negative Werte annehmen, zentriert um null. „This makes encoding and comparison of floating-point numbers easier in hardware.“ (vereinfacht Kodierung und Vergleich in Hardware).

Beispiel 2.2: Constructing scale – Skala über den Exponenten (S. 20)

Wie Zehnerpotenzen in der Gleitkommadarstellung kodiert werden (gespeicherter Exponent E = wahrer Exponent + 127):

$$\begin{aligned} 10^1 &= 10_{10} = 1010_2 = 1.010 \times 2^3, & E &= 10000010_2 \\ 10^2 &= 100_{10} = 1100100_2 = 1.100100 \times 2^6, & E &= 10000101_2 \\ 10^3 &= 1000_{10} = 1111101000_2 = 1.111101000 \times 2^9, & E &= 10001000_2 \end{aligned}$$

[Ergänzung] Zur Kontrolle: $10000010_2 = 130 = 3 + 127$, $10000101_2 = 133 = 6 + 127$, $10001000_2 = 136 = 9 + 127$ – der Bias von 127 wird jeweils auf den wahren Exponenten addiert.

Eine Randnotiz (S. 20) ordnet die Größenordnungen ein: Die größte Zahl in single precision ist ca. 10^{38} , gesetzt durch den größten Exponenten $e = +127$; der Dezimalexponent 38 kommt von $\log_{10}(2^{128}) \approx 38.5$. Als Erinnerung folgen die SI-Präfixe: mega (10^6), giga (10^9), tera (10^{12}), peta (10^{15}), exa (10^{18}), zetta (10^{21}), yotta (10^{24}), ronna (10^{27}), quetta (10^{30}) – für 10^{33} gibt es kein offizielles SI-Präfix (S. 20).

Die Mantisse (S. 20). Die Mantisse bestimmt, wie fein Zahlen *zwischen* Zweierpotenzen dargestellt werden können.

Beispiel 2.3: 2-Bit-Mantisse (unnormalisiert) (S. 20)

Mit den Mantissen-Bitkombinationen 00, 01, 10, 11 und unnormalisiertem Significand $0.xx_2$ ergeben sich:

$$00 : 0.00_2 = 0 + 0 \times 2^{-1} + 0 \times 2^{-2} = 0.0$$

$$01 : 0.01_2 = 0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 0.25$$

$$10 : 0.10_2 = 0 + 1 \times 2^{-1} + 0 \times 2^{-2} = 0.5$$

$$11 : 0.11_2 = 0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 0.75$$

Eine 2-Bit-Mantisse erlaubt also vier verschiedene Werte zwischen zwei beliebigen Zweierpotenzen; eine 3-Bit-Mantisse acht Werte usw.

Satz/Formel 2.3: Mantissenbits und darstellbare Werte (S. 20)

t Mantissenbits $\Rightarrow 2^t$ verschiedene Werte zwischen zwei Zweierpotenzen

In eigenen Worten: Jedes zusätzliche Mantissenbit verdoppelt die Anzahl der Stützstellen zwischen zwei benachbarten Zweierpotenzen – die Auflösung wird also exponentiell feiner mit der Bitzahl. Skaliert wird dieses Raster durch den Exponenten: Zwischen 2^e und 2^{e+1} liegen immer gleich viele darstellbare Zahlen, aber ihr absoluter Abstand wächst mit der Größenordnung.

2.5 Machine Epsilon und Präzision (S. 20–21)

Definition 2.4: Machine Epsilon ($\varepsilon_{\text{mach}}$, Maschinengenauigkeit)

Machine Epsilon ist die kleinste positive Zahl, sodass $1 + \varepsilon_{\text{mach}} \neq 1$ in der Computeraarithmetik gilt. Es quantifiziert die obere Schranke des relativen Fehlers durch Rundung in der Gleitkomma-Arithmetik (S. 20). Konkrete Werte (Randnotiz, S. 20): IEEE 754 single precision $\varepsilon_{\text{mach}} \approx 1.19 \times 10^{-7}$; double precision $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.

Satz/Formel 2.4: Machine Epsilon (Gl. 11 im Skript, S. 20)

$$\varepsilon_{\text{mach}} = 2^{-t}$$

In eigenen Worten: t ist die Anzahl der Mantissenbits. Da t Bits genau 2^t Abstufungen zwischen zwei Zweierpotenzen erlauben, ist der feinste relative Schritt – und damit der maximale relative Rundungsfehler – gerade 2^{-t} . Im 2-Bit-Beispiel von oben: $\varepsilon_{\text{mach}} = 2^{-2} = 0.25$.

Satz/Formel 2.5: Relative vs. absolute Präzision (Gl. 12 im Skript, S. 20)

$$\text{relative Präzision} \approx 2^{-t}, \quad \text{absolute Präzision} = \varepsilon_{\text{mach}} \cdot |x|$$

In eigenen Worten: Die *relative* Präzision von Gleitkommazahlen ist konstant (ca. 2^{-t} , unabhängig von der Größe der Zahl), während die *absolute* Präzision von der Größenordnung der dargestellten Zahl x abhängt: Der absolute Abstand zwischen benachbarten darstellbaren Zahlen wächst proportional zu $|x|$. Die Randnotiz (S. 20) veranschaulicht: $1 : 2 = 0.5$, aber $10 : 2 = 5$ – gleiche Anzahl von Schritten (relative Präzision), aber Lücken von 0.5 vs. 5. In anderen Worten (S. 21): Große Zahlen haben größere absolute

Lücken (gaps) zwischen darstellbaren Werten als kleine Zahlen. (Vgl. Figure 7, S. 21: doppelt-logarithmischer Plot des absoluten Fehlers für single und double precision als Funktion des dargestellten Werts x – der Fehler wächst linear mit x und ist proportional zum jeweiligen Machine Epsilon.)

2.6 Festkommadarstellung (Fixed-Point) (S. 21)

Definition 2.5: Festkommadarstellung (fixed-point representation)

Fixed-Point ist eine andere Art, reelle Zahlen in Computern zu speichern, besonders wenn vorhersagbare Präzision und Performance gewünscht sind. Man entscheidet vorab, wie viele Bits für den Ganzzahl- und wie viele für den Bruchteil verwendet werden. Dadurch ist die Lücke zwischen darstellbaren Zahlen (die Präzision) immer gleich, egal wie groß oder klein der Wert ist – „Which gives more control.“ (S. 21).

Beispiel 2.4: Fixed-Point mit Skalierungsfaktor (S. 21)

Aufgabe: Zahlen zwischen -1000 und 1000 mit einem 32-Bit signed Integer speichern. Normal stellt ein 32-Bit-Integer Werte von -2147483648 bis 2147483647 dar – viel mehr als nötig. Für mehr Präzision verwendet man einen Skalierungsfaktor: z. B. jede reelle Zahl mit 10^6 multiplizieren und als Integer speichern. So wird 1.234567 zu 1234567 im Speicher. Mit dem Skalierungsfaktor 10^{-6} (beim Auslesen) ist die kleinste darstellbare Differenz 0.000001 ; der maximale Rundungsfehler ist ein halber Schritt: $0.5 \times 10^{-6} = 0.0000005$ (Randnotiz *Precision*, S. 21).

2.7 Auslöschung (Loss of Significance) (S. 22–23)

Definition 2.6: Auslöschung (loss of significance / catastrophic cancellation)

Auslöschung tritt auf, wenn zwei nahezu gleiche Zahlen subtrahiert werden: Die führenden Ziffern heben sich auf, und nur die weniger signifikanten, rundungsfehleranfälligen Ziffern bleiben übrig. „This can greatly amplify rounding errors.“ (kann Rundungsfehler stark verstärken) (S. 22).

Definition 2.7: Pseudo-Accuracy (Scheingenauigkeit)

Pseudo-Accuracy bezeichnet allgemein einen ungerechtfertigt hohen Detailgrad, der ein irreführendes, künstliches Genauigkeitsgefühl erzeugt (Randnotiz, S. 22) – eine Warnung davor, vielen angezeigten Nachkommastellen blind zu vertrauen.

Beispiel 2.5: Loss of Significance, Beispiel 1 (S. 22)

Zwei Zahlen a , b werden im Computer mit begrenzter Präzision gespeichert, je auf 8 signifikante Stellen gerundet (Fixed-Point-Darstellung):

$$a = 12345678.5, \quad b = 12345678.0$$

Mit 8 signifikanten Stellen gespeichert als:

$$\tilde{a} = 12345679, \quad \tilde{b} = 12345678$$

Subtraktion der gespeicherten Werte:

$$\tilde{a} - \tilde{b} = 12345679 - 12345678 = 1$$

Wahre Differenz:

$$a - b = 12345678.5 - 12345678.0 = 0.5$$

Der Fehler im Resultat beträgt 0.5 – genauso groß wie der Rundungsfehler in $\tilde{a}$ bzw. $\tilde{b}$ einzeln auf Machine-Epsilon-Niveau 0.5.

Beispiel 2.6: Loss of Significance, Beispiel 2 – minimale Abweichung als Gegenstück (S. 22)

Nun mit minimal veränderter Eingabe:

$$a = 12345678.4, \quad b = 12345678.0$$

Mit 8 signifikanten Stellen gespeichert (12345678.4 rundet $ab!$):

$$\tilde{a} = 12345678, \quad \tilde{b} = 12345678$$

Subtraktion der gespeicherten Werte:

$$\tilde{a} - \tilde{b} = 12345678 - 12345678 = 0$$

Wahre Differenz:

$$a - b = 12345678.4 - 12345678.0 = 0.4$$

Diskussion der Beispiele (S. 23). Die wahren Resultate beider Beispiele unterscheiden sich nur um 0.1 (0.5 vs. 0.4); die Maschinenresultate sind aber 1 im einen und 0 im anderen Fall – „which is a huge relative error. This shows how loss of significance can lead to large errors in computations, especially when the numbers being subtracted are very close to each other.“ Das ist die zentrale Warnung des Kapitels. Eine Randnotiz (S. 22) ergänzt philosophisch: „in mathematics there is an infinity between 0 and 1. The difference between something and nothing.“ – zwischen Ergebnis 0 und 1 liegt eben nicht nur *ein bisschen*, sondern der Unterschied zwischen *etwas* und *nichts*. Eine weitere Randnotiz (S. 23) gibt einen Denkanstoß zum Grenzfall: „What happens for $a > b$? Think along the lines of significant digits.“

Die katastrophale Auslöschung zeigt das Skript zusätzlich grafisch (Figure 8, S. 23): Der Ausdruck $1/(1 + \epsilon - 1)$ wird gegen ϵ geplottet (log-x, linear-y). Für große ϵ folgt das Ergebnis dem erwarteten Verlauf $1/\epsilon$; für sehr kleine ϵ (in der Figur ab ca. $10^{-16.5} \approx 3 \cdot 10^{-17}$, also knapp unterhalb des double-precision-Epsilons $\epsilon_{\text{mach}} \approx 2.22 \cdot 10^{-16}$) dominiert der Rundungsfehler und das Ergebnis wird unendlich [Ergänzung: weil $1 + \epsilon$ im Speicher zu exakt 1 wird und der Nenner zu 0 auslöscht].⁵

Definition 2.8: Overflow und Underflow

Overflow tritt auf, wenn Zahlen großer Magnitude als $+\infty$ oder $-\infty$ approximiert werden. **Underflow** tritt auf, wenn Zahlen nahe null auf null gerundet werden (Randnotizen, S. 23).

⁵Der zugehörige Code ist verfügbar unter https://github.com/Quillstacks/lecturecode_numericalmethods. **git** (Randnotiz, S. 23).

2.8 Recap of Key Concepts (S. 24)

- Die Gleitkommadarstellung erlaubt Computern, einen weiten Bereich reeller Zahlen mit endlich vielen Bits zu speichern, führt aber Rundungsfehler ein.
- Machine Epsilon quantifiziert die kleinste darstellbare Differenz in der Gleitkomma-Arithmetik und beeinflusst die Präzision numerischer Berechnungen.
- Auslöschung (loss of significance) tritt beim Subtrahieren nahezu gleicher Zahlen auf, verstärkt Rundungsfehler und führt zu ungenauen Resultaten.
- *Knowing what can go wrong* (Überleitung zu Kapitel 3): Wir sind nah dran zu verstehen, wie wir definieren, was *gut* ist, und wie wir die Qualität unserer Methoden messen. „We now know that there is a true function f and an approximated function $\hat{f}$, further we have a true input x and a rounded input $\tilde{x}$. These effect and characterize our numerical methods.“ – Notation: wahre Funktion f , approximierte Funktion $\hat{f}$; wahre Eingabe x , gerundete Eingabe $\tilde{x}$.

Teaser (Randnotiz, S. 24). „Can you think of metrics for numerical methods, based on the approximations and errors we discussed?“ – Welche Metriken für numerische Methoden lassen sich aus den diskutierten Approximationen und Fehlern ableiten? Die Brücke zu Kapitel 3 (Error Analysis).

2.9 Übungsaufgaben und Selbstreflexionsfragen

Übungen (nur Aufgabenstellung; Lösungen in den Übungsblättern):

1. *Hands on machine epsilon* (S. 23; zugehöriger Marginalientitel: „Is double enough?“): Überlege – bevor du an die Maschine gehst –, wie man das Machine Epsilon eines beliebigen Systems für ein spezifisches Gleitkommaformat per Programm bestimmen würde. Wiederhole dazu die Definition von Machine Epsilon und notiere deine Gedanken als Pseudocode. „What would such a program show when run on a fixed-point system vs a floating-point system?“ Probiere anschließend verschiedene Typen am eigenen Rechner in Code aus, notiere die Beobachtungen und reflektiere. Frage: „When you would build a calculator, which type would you choose and why?“ (Wenn du einen Taschenrechner bauen würdest – welchen Typ würdest du wählen und warum?)⁶
2. *Loss of significance example* (S. 23): Überlege, wie man Auslöschung auf der eigenen Maschine demonstrieren würde; notiere den Pseudocode, bevor du weiterliest.
3. *Reason about* (S. 23): Begründe, wie das Berechnen von $f(\epsilon) = 1/(a + \epsilon - b)$ für kleines ϵ und $a = b$ diesen Zweck erfüllen würde. „What do you expect to see when ϵ is very small?“

Self-Reflection (S. 24):

1. Wie ist die Gleitkommadarstellung strukturiert, und was sind ihre Komponenten?
2. Was ist Machine Epsilon, und wie hängt es mit der numerischen Präzision zusammen?
3. Wie wirken sich diese Konzepte in der Praxis auf numerische Berechnungen aus?

Typische Fehlerquellen & Merksätze

Typische Fehlerquellen:

⁶Hierzu Fußnote 7 im Skript (S. 23): D. Blochinger: *Numerische Methoden – Foliensatz*, Zentrum für Angewandte Ökonomik (ZAO), DHBW Ravensburg, 2025, CC BY-NC-SA 4.0, Stand 28. Mai 2025, <https://www.econicon.de/repository/index.html>.

- Rundungsfehler (Speicherung) und Abschneide-/Trunkierungsfehler (Approximation unendlicher Prozesse) verwechseln – beide addieren sich, haben aber verschiedene Ursachen (S. 17).
- Relative und absolute Präzision verwechseln: relativ ist sie konstant ($\approx 2^{-t}$), absolut wächst sie mit $|x|$ – große Zahlen haben große Lücken (S. 20–21).
- Beim Exponenten den Bias vergessen: gespeichert wird E , der wahre Exponent ist $e = E - 127$ (single precision) (S. 19).
- Nahezu gleiche Zahlen bedenkenlos subtrahieren: Auslöschung kann aus einem Rundungsfehler von 0.5 einen relativen Fehler von 100 % machen (1 statt 0.5, 0 statt 0.4) (S. 22–23).
- Vielen Nachkommastellen blind vertrauen: Pseudo-Accuracy erzeugt ein künstliches Genauigkeitsgefühl (S. 22).
- Overflow/Underflow ignorieren: sehr große Werte werden zu $\pm\infty$, sehr kleine zu 0 (S. 23).

Merksätze:

- Drei Fehlertypen: Modellierungsfehler (Modell $\neq$ Realität), Trunkierungsfehler (endlich statt unendlich), Rundungsfehler (endliche Ziffern im Speicher).
- Gleitkommazahl = Vorzeichen $\times$ Mantisse $\times$ Zweierpotenz: $x = (-1)^s \cdot m \cdot 2^e$ – Mantisse macht die Präzision, Exponent die Skala.
- $\varepsilon_{\text{mach}} = 2^{-t}$: die kleinste Zahl, die zu 1 addiert noch einen Unterschied macht.
- Subtraktion fast gleicher Zahlen ist der Klassiker der Auslöschung – wenn möglich, Formeln umformen statt subtrahieren.
- Fixed-Point: konstante absolute Lücken (mehr Kontrolle); Floating-Point: konstante relative Lücken (mehr Dynamikumfang).

3 Error Analysis (Fehleranalyse) (S. 25–34)

3.1 Motivation und Einordnung (The Why, S. 25)

„Some call it Error. I call it Character.“ — Konditionierung, Stabilität, Konsistenz und Konvergenz (Conditioning, Stability, Consistency, Convergence) sind die vier fundamentalen Konzepte der numerischen Analysis, mit denen sich das Verhalten numerischer Algorithmen und ihre Zuverlässigkeit beim Lösen mathematischer Probleme verstehen und beschreiben lassen (S. 25). Konkret wollen wir: die Sensitivität des *Problems* gegenüber kleinen Eingabeänderungen einschätzen, die Sensitivität der *numerischen Lösung* gegenüber kleinen Eingabeänderungen bewerten, sicherstellen, dass numerische Methoden genaue und reproduzierbare Resultate liefern, und garantieren, dass unsere Approximationen zur wahren Lösung konvergieren. Im Machine-Learning-Kontext ist das zentral, denn das Training neuronaler Netze ist ein großskaliges, iterativ-numerisches Optimierungsproblem: „In Deep Learning convergence has the center stage, then almost as an afterthought comes Complexity – the number of operations and amount of time it takes to get there.“ (Im Deep Learning steht die Konvergenz im Zentrum; fast als Nachgedanke folgt die Komplexität — Anzahl der Operationen und benötigte Zeit, S. 25). AI-Modelle θ lassen sich später genau mit diesen Konzepten bewerten — „So the maths aside, this will come in handy.“

Randnotiz: Modellfehler (S. 25). Mit $f(\cdot)$ ist das *exakte Modell* eines Systems gemeint. In der Praxis sind Modelle Vereinfachungen der Realität: Für die Distanz zwischen zwei Punkten A und B verwendet man z.B. ein vereinfachtes Manhattan- oder euklidisches Distanzmodell, das Terrain, Fortbewegungsart oder Hindernisse nicht berücksichtigt. Also im Hinterkopf behalten: „our $f(\cdot)$ here, is another $f(\cdot)$.“ — Lesetipp des Skripts: I. Goodfellow, Y. Bengio, A. Courville: *Deep Learning*, MIT Press, 2016, Kap. 4 (Numerical Computation).

3.2 Lernziele (S. 28)

- Intuition über die Konzepte Konditionierung, Stabilität, Konsistenz und Konvergenz in numerischen Methoden entwickeln.
- Einfache numerische Probleme *quantitativ* bezüglich dieser Konzepte analysieren.

3.3 Intuitiver Zugang (Hands On Experience, S. 26–28)

Das Skript baut die Intuition am vertrauten Beispiel aus Kapitel 1 auf (Minimum von $\sin(x)$ auf $[-3, 1]$, diskretisiert) und gibt zu jedem Konzept *zwei* schnelle Gegenüberstellungen, „as we want to highlight two sides of the same coin for each concept“ — also bewusst Extreme des Spektrums:

- **Conditioning (Konditionierung):** „is about how sensitive the solution is to small changes in the input data.“ Annahme: Die approximierte Funktion ist die wahre Funktion (Sinus), aber die Eingabe $\tilde{x}$ ist eine durch Messfehler oder Rundung gestörte Version von x (± 0.25). In Figure 9 (S. 26: Sinuskurve mit zwei grauen vertikalen Bändern der Breite 0.5 bei $x = -1.5$ und $x = 0$) sieht man: Bei $x = -1.5$ (nahe dem Minimum, Funktion flach) ist die Region *gut konditioniert* (well-conditioned) — kleine Änderungen in x bewirken kleine Änderungen in $f(x)$. Das Band um $x = 0$ (steiler Anstieg) ist *schlecht konditioniert* (ill-conditioned) — kleine Änderungen in x können große relative Änderungen in $f(x)$ bewirken. *Wichtige Abgrenzung (Randnotiz S. 26):* „This is about floating-point representation and not about step size or optimal approximation. Make sure to wrap your head around that, before moving on.“ — Es geht hier um die gestörte Eingabe, nicht um die Diskretisierung.
- **Stability (Stabilität):** „refers to the sensitivity of the numerical solution to the small changes in the input data.“ Beobachtung am grauen Band um $x = 0$: Bei feiner Schrittweite $h = 0.2$ (Figure 10, S. 26) folgen die diskretisierten Punkte der Sinuskurve eng — kleine Verschiebungen $\tilde{x} \pm 0.25$ führen zu Fluktuationen in den berechneten Werten $\hat{f}(\tilde{x})$: *Instabilität*. Bei grober Schrittweite $h = 1$ (Figure 11, S. 27) liegt nur ein einziger Gitterpunkt im Band — die Approximation ist in dieser Region *stabil*, unabhängig von den Abweichungen in $\tilde{x}$. [Ergänzung: Pointe — Gröbere Diskretisierung ist stabiler, aber, wie gleich zu sehen, weniger konsistent!]
- **Consistency (Konsistenz, S. 27):** „quantifies how well our numerical method matches the exact solution of the original problem.“ Veranschaulicht über Balkendiagramme der stückweise konstanten Approximation mit $h = 1$ und $h = 0.2$ (Figures 12–13, S. 27): Für infinitesimal kleine Schrittweiten $h \rightarrow 0$ kommen die diskretisierten Punkte der wahren Sinuskurve immer näher — im Grenzfall perfekte Übereinstimmung zwischen numerischer Methode und exakter Lösung: *optimale Konsistenz*.
- **Convergence (Konvergenz, S. 27–28):** entsteht aus der Kombination von Stabilität und Konsistenz: „A numerical method is convergent if, as we refine our approximation $\hat{f}(x)$ (for example, by decreasing the step size h), the computed solution approaches the exact solution of the problem regardless of the deviations in $\tilde{x}$, or by implicitly accounting for them.“ — Beim Verfeinern der Approximation nähert sich die berechnete Lösung der

exakten Lösung, unabhängig von (oder unter impliziter Berücksichtigung von) Störungen in $\tilde{x}$. (Meta-Randnotiz S. 27: „Intuitive convergence example with sine, wanted. If you have a better idea ... let me know.“ — ein Autoren-Aufruf ohne Fachinhalt, der Vollständigkeit halber erwähnt.)

3.4 Quantitative Charakterisierung: Notation und Definitionen (S. 29)

Definition 3.1: Notation — Hut vs. Tilde (Hat versus Tilde, S. 29)

$f(\cdot)$ bezeichnet die *exakte (analytische) Lösung* eines Problems; $\hat{f}(\cdot)$ bezeichnet die *numerische Methode* (den Algorithmus), die eine Approximation liefert. x steht für die exakten Eingabedaten, $\tilde{x}$ für die tatsächlich verwendeten, z.B. durch Messfehler oder Rundung gestörten Eingabedaten.

Merkhilfe (Randnotiz S. 29): „The hat $\hat{}$ marks the numerical method (algorithm), while the tilde $\tilde{}$ marks a perturbed input. So $\hat{f}(\tilde{x})$ reads as: numerical method evaluated on perturbed input data.“ — Der Hut markiert die numerische Methode, die Tilde die gestörte Eingabe; $\hat{f}(\tilde{x})$ liest sich als *numerische Methode, ausgewertet auf gestörten Eingabedaten*.

Definition 3.2: Conditioning (Konditionierung) und Konditionszahl κ (S. 29)

Die Konditionierung beschreibt, wie sensitiv die Lösung eines *Problems* auf kleine Änderungen der Eingabedaten reagiert. Formal wird die Konditionierung eines Problems bei x durch die Konditionszahl (condition number) κ quantifiziert (Gl. 13 im Skript, S. 29):

$$\kappa = |f(x) - f(\tilde{x})| \quad (\text{Gl. 13})$$

In Worten: κ misst, wie stark sich der *exakte* Funktionswert ändert, wenn man statt der wahren Eingabe x die gestörte Eingabe $\tilde{x}$ einsetzt. Beachte: Hier kommt nur f (das Problem), nicht $\hat{f}$ (die Methode) vor — Konditionierung ist eine Eigenschaft des Problems, unabhängig von der gewählten numerischen Methode. „ κ is often normalized to express it as a relative measure.“ (κ wird oft normalisiert, um es als relatives Maß auszudrücken.)

Definition 3.3: Stability (Stabilität) (S. 29)

Stabilität ist gegeben, wenn kleine Fehler in der Eingabe oder in Zwischenschritten nicht zu unverhältnismäßig großen Fehlern in der Ausgabe führen. Mathematisch stellt eine stabile Methode sicher, dass der Fehler in der berechneten Lösung $\hat{f}(\tilde{x})$ durch ein konstantes Vielfaches des Eingabefehlers beschränkt bleibt (Gl. 14 im Skript, S. 29):

$$|\hat{f}(\tilde{x}) - \hat{f}(x)| \leq s \cdot |\tilde{x} - x| \quad (\text{Gl. 14})$$

wobei s eine Konstante ist. In Worten: Links steht, wie stark die *Methode* auf die Eingabestörung reagiert; rechts steht die Größe der Störung selbst, skaliert mit s . „For $s \approx 1$, the error in the computed solution develops linearly with the error in the input data.“ (Für $s \approx 1$ wächst der Ausgabefehler linear mit dem Eingabefehler.) Beachte: Hier kommt zweimal $\hat{f}$ vor — Stabilität ist eine Eigenschaft der *Methode*.

Definition 3.4: Consistency (Konsistenz) (S. 29)

Konsistenz misst, wie gut die numerische Lösung die exakte Lösung des Originalproblems

approximiert (Gl. 15 im Skript, S. 29):

$$|\hat{f}(x) - f(x)| \leq c \quad (\text{Gl. 15})$$

wobei c eine Konstante ist, die den Konsistenzfehler quantifiziert. In Worten: Methode $\hat{f}$ und exakte Lösung f werden auf *denselben, ungestörten* Eingabedaten x verglichen — Konsistenz isoliert also den reinen Methodenfehler (z. B. Diskretisierungsfehler), ohne Eingabestörungen.

Definition 3.5: Convergence (Konvergenz) (S. 29)

Konvergenz bezieht sich allgemein darauf, dass unsere Approximation einen spezifischen, stabilen Grenzwert erreicht. Eine Methode ist konvergent, wenn (Gl. 16 im Skript, S. 29):

$$|\hat{f}(\tilde{x}) - f(x)| \rightarrow \lim \quad (\text{Gl. 16})$$

„That is, as both the real solution is approximated and deviations in data are controlled by an error margin.“ — Sowohl die Annäherung an die echte Lösung als auch die Abweichungen in den Daten werden durch eine Fehlerschranke kontrolliert. In Worten: Verglichen wird die Methode auf *gestörten* Daten $\hat{f}(\tilde{x})$ mit der exakten Lösung auf *exakten* Daten $f(x)$ — Konvergenz kombiniert also Stabilität (Umgang mit $\tilde{x}$) und Konsistenz (Nähe von $\hat{f}$ zu f).

Satz/Formel 3.1: Nicht-Null-Konvergenz und systematischer Fehler (Bias) (Randnotiz S. 29)

Konvergenz „usually aims for zero error margin, which is often the exact solution $f(x)$. However, often we will experience non-zero convergence.“ — Konvergenz zielt üblicherweise auf eine Fehlerschranke von null (also die exakte Lösung $f(x)$); oft erlebt man aber Konvergenz gegen einen *von null verschiedenen* Wert. **Konvergiert die Methode gegen einen von $f(x)$ verschiedenen Wert, zeigt das einen systematischen Fehler (Bias) der Methode an.** [Ergänzung: Das ist die Brücke zur ML-Terminologie — ein konstanter Restfehler trotz Verfeinerung entspricht einem Bias, Fluktuationen durch Eingabestörungen einer Varianz; vgl. die Selbstreflexionsfrage zu Bias und Varianz auf S. 33.]

Satz/Formel 3.2: Zusammenhang Stabilität und Konditionierung für $h \rightarrow 0$ (Randnotiz S. 31)

Das Skript stellt als Übungsaufgabe fest: „Stability equals the conditioning of the system, for $h \rightarrow 0$. Show it.“ — Die Stabilität (skonstante) entspricht im Grenzfall $h \rightarrow 0$ der Konditionierung des Systems. [Ergänzung zur Plausibilisierung: Für $h \rightarrow 0$ wird die diskretisierte Methode $\hat{f}$ zur exakten Funktion f ; dann misst die Stabilitätsungleichung $|\hat{f}(\tilde{x}) - \hat{f}(x)| \leq s |\tilde{x} - x|$ genau dieselbe Größe $|f(\tilde{x}) - f(x)| = \kappa$ wie die Konditionierung — die Methodenempfindlichkeit geht in die Problemempfindlichkeit über.]

3.5 Durchgerechnete Analyse am Sinus-Beispiel (S. 30–33)

Aufgabenstellung (S. 30): Erneut das Minimum der Sinusfunktion auf $[-3, 1]$; Diskretisierung mit zwei Schrittweiten $h = 1$ und $h = 0.2$. Annahme: Die Eingabe x ist durch $\tilde{x} = x \pm 0.25$ gestört (Messfehler); gerechnet wird mit einer Präzision von zwei Dezimalstellen. Analysiere

Konditionierung, Stabilität, Konsistenz und Konvergenz der numerischen Methode in beiden Fällen und quantifiziere mit den obigen Formeln. *Erinnerung (Randnotiz S. 31)*: Die analytische Lösung von $\min \sin(x)$ auf $[-3, 1]$ ist -1 bei $x = -\pi/2$ (Kapitel 1).

Beispiel 3.1: Konditionierung des Sinus-Problems (S. 30–31)

Die Konditionierung ist *unabhängig von der verwendeten numerischen Methode* — sie beschreibt die Sensitivität des Problems selbst. Analysiert wird also rein die Wirkung kleiner Änderungen in x auf $\sin(x)$. Um $x = -1$ (nahe dem Minimum) ist der Sinus relativ flach $\Rightarrow$ gute Konditionierung; zum Vergleich $x = 0$, wo der Sinus sich schnell ändert $\Rightarrow$ schlechte Konditionierung. Rechnungen mit Gl. 13 ($\tilde{x} = x \pm 0.25$):

$$\kappa(-1, +0.25) = |\sin(-1) - \sin(-0.75)| \approx |-0.8415 - (-0.6816)| = 0.1599$$

$$\kappa(-1, -0.25) = |\sin(-1) - \sin(-1.25)| \approx |-0.8415 - (-0.9489)| = 0.1074$$

$\Rightarrow$ **Gute Konditionierung** (well-conditioning) bei $x = -1$ mit $0.1074 \leq \kappa \leq 0.1599$.

$$\kappa(0, +0.25) = |\sin(0) - \sin(0.25)| \approx |0 - 0.2474| = 0.2474$$

$$\kappa(0, -0.25) = |\sin(0) - \sin(-0.25)| \approx |0 - (-0.2474)| = 0.2474$$

$\Rightarrow$ **Schlechte Konditionierung** (ill-conditioning) bei $x = 0$ mit $\kappa \approx 0.2474$ — der Wert schöpft das Maximum der betrachteten Störungen ($|\tilde{x} - x| = 0.25$) nahezu voll aus und liegt deutlich über dem gut konditionierten Fall. [Korrektur ggü. Original, S. 31: Das PDF schreibt „ $\kappa \leq 0.247$ “ — das ist weder numerisch korrekt ($0.2474 > 0.247$) noch als obere Schranke geeignet, eine Schlecht-Konditionierung zu belegen.] [Ergänzung: Eine Störung gleicher Größe (± 0.25) verursacht bei $x = 0$ einen mehr als doppelt so großen Ausgabefehler wie der günstigste Fall bei $x = -1$ — und bei $x = 0$ ist zusätzlich der *relative* Fehler dramatisch, da $\sin(0) = 0$.]

Beispiel 3.2: Stabilität der diskretisierten Methode für $h = 1$ und $h = 0.2$ (S. 31)

Die Stabilität hängt von der numerischen *Methode* ab (hier: Diskretisierung + Minimumsuche). Qualitativ: Für $h = 1$ ist die Methode stabil — kleine Änderungen in $\tilde{x}$ führen zu kleinen Änderungen in $\hat{f}(\tilde{x})$. Für $h = 0.2$ ist sie weniger stabil — kleine Änderungen in $\tilde{x}$ können größere Fluktuationen in $\hat{f}(\tilde{x})$ verursachen. Quantifizierung über die Konstante s in der Stabilitätsungleichung (Gl. 17 im Skript, S. 31; identisch mit Gl. 14):

$$|\hat{f}(\tilde{x}) - \hat{f}(x)| \leq s \cdot |\tilde{x} - x| \quad (\text{Gl. 17})$$

„Due to symmetry we only consider the case $\tilde{x} = x + 0.25$.“ (Aus Symmetriegründen nur der Fall $\tilde{x} = x + 0.25$.)

Für $h = 1$ (bei $x = -1$):

$$s_{h=1, x=-1} \cdot |\tilde{x} - x| = |\hat{f}(\tilde{x}) - \hat{f}(x)| = |\hat{f}_1(-0.75) - \hat{f}_1(-1)| = |\hat{f}_1(-1) - \hat{f}_1(-1)| = 0$$

$$s_{h=1, x=-1} \approx 0 \div 0.25 \approx 0$$

Erklärung des entscheidenden Schritts (Randnotiz A, S. 31): „See how the step size $h = 1$ leads to the same function value at both points. For reference, Fig. 12.“ — Bei der stückweise konstanten Approximation mit $h = 1$ fallen -0.75 und -1 auf denselben Gitterwert, daher $\hat{f}_1(-0.75) = \hat{f}_1(-1)$ und der Zähler wird null: maximal stabil.

[Korrektur ggü. Original, S. 31: Das PDF druckt innerhalb derselben Rechnung einen Indexwechsel $s_{h=1, x=-1} \rightarrow s_{h=1, x=0}$; gemeint ist durchgängig derselbe Fall ($\tilde{x} = x + 0.25$, $x = -1$). Das Ergebnis $s \approx 0$ ist korrekt.]

Für $h = 0.2$ (bei $x = -1$):

$$\begin{aligned} s_{h=0.2, x=-1} \cdot |\tilde{x} - x| &= |\hat{f}(\tilde{x}) - \hat{f}(x)| = |\hat{f}_{0.2}(-0.75) - \hat{f}_{0.2}(-1)| \\ &= |\hat{f}_{0.2}(-0.8) - \hat{f}_{0.2}(-1)| = |-0.71736 - (-0.84147)| = 0.12411 \end{aligned}$$

$$s_{h=0.2, x=-1} = 0.12411 \div 0.25 \approx 0.4964$$

[Ergänzung: Der gestörte Punkt -0.75 fällt beim 0.2er-Gitter auf den Gitterpunkt -0.8 , der bereits einen anderen Funktionswert trägt als der Gitterpunkt -1 — das feine Gitter *sieht* die Störung und reagiert darauf; daher $s \approx 0.5 > 0$, also weniger stabil als bei $h = 1$.]

Warnung (Stolperfalle, S. 31): „Be aware that stability has anomalies in border regions of the discretization. This is especially a problem for piece-wise constant approximations.“ — Stabilität hat Anomalien in den Randregionen der Diskretisierung; besonders problematisch für stückweise konstante Approximationen.

Beispiel 3.3: Konsistenz für $h = 1$ und $h = 0.2$ (S. 31–32)

Die Konsistenz wird durch Vergleich der numerischen Lösung $\hat{f}(x)$ mit der exakten Lösung $f(x)$ für das Minimum des Sinus in $[-3, 1]$ bestimmt. Quantifizierung der Konstante c (Gl. 18 im Skript, S. 31; identisch mit Gl. 15):

$$|\hat{f}(x) - f(x)| \leq c \quad (\text{Gl. 18})$$

Für $h = 1$ (S. 32): Die exakte Minimalstelle ist $x = -\pi/2$ mit $f(-\pi/2) = -1$; das Skript ordnet ihr beim $h = 1$ -Gitter den Gitterpunkt -1 zu:

$$c_1 \geq |\hat{f}_1(-\frac{\pi}{2}) - f(-\frac{\pi}{2})| \geq |\hat{f}_1(-1) - f(-\frac{\pi}{2})| \geq |-0.84147 - (-1)| \geq 0.15853$$

Für $h = 0.2$ (S. 32): Beim 0.2er-Gitter ordnet das Skript $-\pi/2$ dem Gitterpunkt -1.2 zu; dort gilt $\hat{f}_{0.2}(-1.2) = \sin(-1.2) = -0.93204$:

$$c_{0.2} \geq |\hat{f}_{0.2}(-\frac{\pi}{2}) - f(-\frac{\pi}{2})| \geq |\hat{f}_{0.2}(-1.2) - f(-\frac{\pi}{2})| \geq |-0.93204 - (-1)| \geq 0.06796$$

[Korrektur ggü. Original, S. 32: Das PDF druckt -0.94898 und als Endwert 0.05102 ; -0.94898 ist aber $\sin(-1.25)$, und -1.25 ist kein Punkt des 0.2er-Gitters auf $[-3, 1]$ ($\dots, -1.4, -1.2, -1.0, \dots$). Mit dem im Skript selbst genannten Gitterpunkt -1.2 ist $\sin(-1.2) = -0.93204$ und damit $c_{0.2} \approx 0.06796$.]

Anmerkung zur Gitterpunkt-Zuordnung: Warum das Skript $-\pi/2 \approx -1.5708$ gerade den Gitterpunkten -1 (bei $h = 1$) bzw. -1.2 (bei $h = 0.2$) zuordnet, wird im Original nicht erklärt; die Zuordnung weicht vom sonst im Skript verwendeten Nächste-Nachbarn-Snapping ab (der jeweils *nächste* Gitterpunkt zu $-\pi/2$ wäre -2 bzw. -1.6).

Befund: „Consistency improves with smaller step sizes, as expected.“ — Die Konsistenz verbessert sich erwartungsgemäß mit kleineren Schrittweiten: $c_1 = 0.15853$ vs. $c_{0.2} \approx 0.06796$ [Korrektur ggü. Original, S. 32: dort 0.05102].

Beispiel 3.4: Konvergenz für $h = 1$ und $h = 0.2$ (S. 32–33)

Konvergenz kombiniert die Ideen von Stabilität und Konsistenz. Quantifiziert wird der Gesamtfehler zwischen numerischer Lösung auf gestörten Daten $\hat{f}(\tilde{x})$ und exakter Lösung

$f(x)$ (Gl. 19 im Skript, S. 32):

$$|\hat{f}(\tilde{x}) - f(x)| \quad (\text{Gl. 19})$$

Aus Symmetriegründen wieder nur $\tilde{x} = x + 0.25$.

Für $h = 1$:

$$\begin{aligned} |\hat{f}_1(\tilde{x}) - f(x)| &= |\hat{f}_1(-1) - f(-\frac{\pi}{2})| = |\hat{f}_1(-0.75) - f(-\frac{\pi}{2})| \\ &= |\hat{f}_1(-1) - f(-\frac{\pi}{2})| = |-0.84147 - (-1)| = 0.15853 \end{aligned}$$

Für $h = 0.2$: Ausgangspunkt ist der Gitterpunkt -1.2 , dem das Skript $-\pi/2$ zuordnet (vgl. Anmerkung zur Gitterpunkt-Zuordnung in der Konsistenzbox oben); die gestörte Eingabe ist $\tilde{x} = -1.2 + 0.25 = -0.95$, und diese fällt beim Auswerten auf den Gitterpunkt -1 :

$$|\hat{f}_{0.2}(\tilde{x}) - f(x)| = |\hat{f}_{0.2}(-0.95) - f(-\frac{\pi}{2})| = |\hat{f}_{0.2}(-1) - f(-\frac{\pi}{2})| = |-0.84147 - (-1)| = 0.15853$$

[Korrektur ggü. Original, S. 32: Das PDF druckt in beiden Fällen den Endwert „= 0.1599“; korrekt ist $|-0.84147 - (-1)| = 0.15853$. Zudem fehlen im PDF in den Umformungsketten die verbindenden Relationszeichen (die Zeilen stehen ohne Gleichheitszeichen gestapelt untereinander), und im $h = 0.2$ -Block steht einmal fälschlich $\hat{f}_1(-1.2)$ statt $\hat{f}_{0.2}(-1.2)$. Redaktionelle Klarstellung: Die im PDF zusätzlich gestapelte Zeile $|\hat{f}_{0.2}(-1.2) - f(-\pi/2)|$ (Auswertung am ungestörten Bezugspunkt -1.2 , Wert 0.06796) ist *nicht* gleich den übrigen Gliedern und wurde hier deshalb nicht als Gleichung in die Kette aufgenommen, sondern textlich als Ausgangspunkt der Herleitung beschrieben.]

Konvergenz-Diskussion (S. 33): Wegen der Stabilitätsprobleme im Fall $h = 0.2$ verbessert sich die Konvergenz in diesem Operationspunkt *nicht* mit kleineren Schrittweiten — die Störung $\tilde{x} = x + 0.25$ schiebt die Auswertung beim feinen Gitter wieder auf den Wert -0.84147 , sodass beide Schrittweiten denselben Gesamtfehler 0.15853 liefern. Zwei Erkenntnisse: (1) Beide numerischen Lösungen konvergieren zum gleichen Grenzwert; (2) daraus lässt sich auch sagen, dass die Methode des iterativen Verkleinerns der Schrittweite h ebenfalls zum gleichen Grenzwert konvergiert. „Notice that we use the characteristics for both a numerical solution and the numerical method.“ — Begriffsdifferenzierung: Die Charakteristika werden sowohl für eine einzelne *numerische Lösung* als auch für die *numerische Methode* als Ganzes verwendet.

3.6 Übungsaufgaben und Selbstreflexionsfragen (S. 31, 33)

Übungen:

- (Randnotiz S. 31) Zeige: Die Stabilität entspricht der Konditionierung des Systems für $h \rightarrow 0$.
- (Hands on compute-driven error analysis, S. 33) Brute-Force-Fehleranalyse des Zwei-Gleichungs-Systems aus Kapitel 1: Konditionierung, Stabilität, Konsistenz und Konvergenz der numerischen Lösung bestimmen und durch Optimierung der numerischen Methode verbessern. (Code: github.com/Quillstacks/lecturecode_numericalmethods.git)

Self-Reflection (S. 33):

- Wie gut konditioniert ist das Zwei-Gleichungs-System, verglichen mit der numerischen Lösung des Sinus-Minimums?
- Konvergiert die Stabilität im Zwei-Gleichungs-System mit kleineren Schrittweiten? Gegen was?
- Wie wirken sich Störungen der Eingabedaten auf die numerische Lösung des Zwei-Gleichungs-Systems aus? Gibt es ein Zusammenspiel mit der Schrittweite?

- Vergleiche Konvergenz mit Stabilität und Konsistenz. Was beobachtest du? Kannst du deine Beobachtungen in Begriffen von Bias und Varianz ausdrücken?
- Beobachtest du bei immer kleineren Schrittweiten eine Grenze der Genauigkeit der numerischen Lösung? Wenn ja, warum?
- Welches weitere Problem beobachtest du beim Verfeinern der Schrittweite?

3.7 Recap und Ausblick (S. 33–34)

Konditionierung, Stabilität, Konsistenz und Konvergenz (im Original mit Tippfehler „Conditioning“ [sic]) sind die fundamentalen Konzepte zum Verständnis numerischer Algorithmen; sie lassen sich qualitativ *und* quantitativ ausdrücken, um numerische Lösungen und Methoden zu analysieren. **Teaser (Randnotiz S. 33):** „So far we did brute-force numerical solutions. Can you think of a way to refine brute-force methods to get better results with less effort?“ — Brute-Force-Methoden verfeinern, um mit weniger Aufwand bessere Resultate zu erzielen; mit dem Vorlesungscode designen und testen. **Überleitung (S. 34, wörtlich):** „Now that we understand and can characterize the behavior of individual numerical solutions, we can move on to understanding how to analyze entire numerical methods and algorithms. So far we have taken the assumption of where to look for a solution, inside a specific interval, for granted. This is usually not a given, and we will need to move away from local optimization methods in the next lecture.“

Typische Fehlerquellen & Merksätze

Typische Fehlerquellen:

- **Hut und Tilde verwechseln:** $\hat{f}$ = numerische Methode, $\tilde{x}$ = gestörte Eingabe. $\hat{f}(\tilde{x})$ = Methode auf gestörten Daten. Wer das vertauscht, verwechselt Methoden mit Datenfehlern.
- **Konditionierung vs. Stabilität:** Konditionierung ist eine Eigenschaft des *Problems* (f , Gl. 13) — Stabilität eine Eigenschaft der *Methode* ($\hat{f}$, Gl. 14). Die Konditionierung ändert sich nicht, wenn man die Methode wechselt!
- **„Feiner ist immer besser“ ist falsch:** Kleineres h verbessert die Konsistenz ($c_{0,2} \approx 0.06796 < c_1 = 0.15853$ [Korrektur ggü. Original, S. 32]), kann aber die Stabilität verschlechtern ($s_{0,2} \approx 0.4964 > s_1 \approx 0$) — und die Konvergenz im gestörten Fall gar nicht verbessern (beide 0.15853).
- **Randregionen der Diskretisierung:** Dort hat die Stabilität Anomalien — besonders bei stückweise konstanten Approximationen.
- **Rechenfehler-Falle aus dem Original:** $|-0.84147 - (-1)| = 0.15853$ (nicht 0.1599 — das ist der Wert von $\kappa(-1, +0.25)$, der im PDF an der Konvergenzstelle fälschlich wieder auftaucht).

Merksätze:

- *Das Problem ist konditioniert, die Methode ist stabil, konsistent und konvergent.*
- *Konvergenz = Stabilität + Konsistenz:* Nur wer Störungen kontrolliert *und* die exakte Lösung approximiert, konvergiert.
- *Konvergenz gegen den falschen Wert = Bias:* Ein stabiler Grenzwert $\neq f(x)$ verrät einen systematischen Methodenfehler.
- *Für $h \rightarrow 0$ wird Stabilität zur Konditionierung* — die Methode erbt die Empfindlichkeit des Problems.

4 Thema 4: Newton Methods (Newton-Verfahren) — S. 35–46

4.1 Motivation und Einordnung (The Why, S. 35)

„A Step in the Right direction.“ — Im vorigen Kapitel haben wir numerische Methoden mit Konditionierung, Stabilität, Konsistenz und Konvergenz charakterisiert. Die Brute-Force-Gittersuche (grid search) ist jedoch fundamental begrenzt: „it explores the solution space blindly, requiring $O(N^d)$ evaluations for N grid points in d dimensions.“ — Sie erkundet den Lösungsraum blind und benötigt $O(N^d)$ Auswertungen für N Gitterpunkte in d Dimensionen (Komplexitätsangabe; [Ergänzung] das ist der *Fluch der Dimensionalität*: der Aufwand explodiert exponentiell mit der Dimension). Die Schlüsselidee (the key insight) „is deceptively simple: instead of asking ‘what is the value here?’ at every grid point, we also ask ‘which way should I go next?’“ — Statt an jedem Gitterpunkt nur *Was ist hier der Wert?* zu fragen, fragen wir auch *In welche Richtung soll ich als Nächstes gehen?* Die Steigung der Funktion, ihre Ableitung, zeigt die Richtung zur Lösung — das transformiert die Suche fundamental. Nutzen: lokale Information für anspruchsvolle Schritte statt Brute-Force-Suche; schnellere und optimalere Konvergenz.

ML/Deep-Learning-Bezug (S. 35). „In Machine Learning and Deep Learning, Newton’s method simplified to first order is also known as gradient descent. One could argue that the entire field of deep learning is built on the idea of using local gradient information to navigate toward minima in a high-dimensional loss landscape, to train models that are optimized towards specific objectives.“ — Newtons Methode, auf erste Ordnung vereinfacht, ist der *Gradientenabstieg* (gradient descent); das gesamte Feld Deep Learning baut darauf auf, mit lokaler Gradienteninformation in hochdimensionalen Loss-Landschaften zu Minima zu navigieren. *Randnotiz Backpropagation*: „is of course as vital to distribute gradient information through the layers of a neural network, but the core optimization step is still a form of gradient-based navigation.“ — Backpropagation verteilt die Gradienteninformation durch die Schichten; der Kern-Optimierungsschritt bleibt gradientenbasierte Navigation.

Randnotiz Trade-off (S. 36). „We pay for this sophistication with more hyperparameters that we need to determine and more complex computations (derivative evaluation and division), but we gain in convergence speed.“ — Wir bezahlen die Raffinesse mit mehr Hyperparametern und komplexeren Berechnungen (Ableitungsauswertung, Division), gewinnen aber Konvergenzgeschwindigkeit.

4.2 Lernziele (S. 37)

- Newton-Raphson-Iteration für die Nullstellensuche (root finding) in 1D herleiten und anwenden.
- Taylor-Entwicklung (Taylor expansion) und ihre Rolle in Newtons Methode herleiten und verstehen (im Original „understad“ [sic]).
- Konvergenzraten und Fehlermodi (failure modes) von Newtons Methode analysieren.
- Sekantenverfahren (Secant method) verstehen und anwenden, wenn Ableitungen nicht verfügbar sind.

4.3 Hands On: Grid Search vs. Newton an der Sinusfunktion (S. 36–37)

Intuitives Gefühl für die Kraft lokaler Information: Vergleich von Grid Search und Newtons Methode an derselben Sinusfunktion wie in Kapitel 3 — nun aber als *Nullstellensuche*: $\sin(x) = 0$ auf $[2.5, 4]$ („the solution is near $\pi \approx 3.14159$ “). Figure 14 (S. 36) zeigt die fallende Sinuskurve auf $[2.5, 4]$ mit den Gitterpunkten ($h = 0.3$), Figure 15 (S. 36) die Newton-Iterationen ab $x_0 = 3.5$ mit schneller Konvergenz gegen π .

Beispiel 4.1: Grid Search zur Nullstellensuche von $\sin(x)$ auf $[2.5, 4]$, $h = 0.3$ (S. 36)

Grid Search „evaluates blindly“ — alle Gitterpunkte werden stumpf ausgewertet (vollständige Werte):

$$f(2.5) \approx 0.599 \quad [\text{Anm.: korrekt gerundet } 0.598, \text{ da } \sin(2.5) = 0.598472; \text{ PDF: } 0.599]$$

$$f(2.8) \approx 0.335$$

$$f(3.1) \approx 0.042 \quad \leftarrow \text{am nächsten an null (closest to zero)}$$

$$f(3.4) \approx -0.256$$

$$f(3.7) \approx -0.530$$

$$f(4.0) \approx -0.757$$

Ergebnis: „We found $x \approx 3.1$ with 6 evaluations, with an error $|3.1 - \pi| \approx 0.042$.“ — 6 Auswertungen, Fehler ≈ 0.042 .

Beispiel 4.2: Newton von Hand an $\sin(x)$, Start $x_0 = 3.5$ (S. 37)

Newtons Methode hat einen *adaptiven* Suchansatz: An jedem Punkt werden Funktion *und* Ableitung ausgewertet; die lokale Information liefert den Schritt Richtung Lösung. Die Ableitung $f'(x) = \cos(x)$ gibt die Steigung der Tangente. Es gibt mehrere Wege, diese Information zu nutzen (S. 37): „We for sure want to move in the direction where the function decreases, then we could take a step with fixed size, or we could make the step size proportional to the derivative. Let’s just take the next guess where the tangent crosses zero for now – which is the solution to the linear approximation of f at x_n .“ — Sicher in Richtung fallender Funktionswerte; dann entweder fester Schritt oder Schritt proportional zur Ableitung; hier: nächste Schätzung dort, wo die *Tangente die Null kreuzt* (Lösung der linearen Approximation von f bei x_n).

$$x_1 = x_0 - \frac{\sin(x_0)}{\cos(x_0)} = 3.5 - \frac{-0.351}{-0.936} = 3.5 - 0.375 = 3.125$$

$$x_2 = 3.125 - \frac{\sin(3.125)}{\cos(3.125)} = 3.125 - \frac{0.01659}{-0.99986} \approx 3.125 + 0.0166 \approx 3.1416$$

[Korrektur ggü. Original, S. 37: Das PDF druckt im x_2 -Schritt „ $\frac{0.0008}{-0.9999}$ “; tatsächlich ist $\sin(3.125) \approx 0.01659$ und $\cos(3.125) \approx -0.99986$. Das Endergebnis $x_2 \approx 3.1416$ stimmt, der gedruckte Zwischenwert nicht.]

$$x_3 \approx 3.14159, \quad \sin(3.14159) \approx 0$$

Fazit (S. 37): „After just 2 iterations, to be fair this means 4 function/derivative evaluations, we have $x \approx 3.14159$ with error $< 10^{-5}$. The derivative told us where to look way more efficiently than we have been used to.“ — Vergleich: Grid Search 6 Auswertungen, Fehler 0.042; Newton 4 Funktions-/Ableitungsauswertungen, Fehler $< 10^{-5}$.

Randnotiz zur Historie (S. 37). Die Methode heißt im Skript „Newton-Rhapson“ [sic] — korrekt *Newton-Raphson*: „because Isaac Newton came up with the general idea, while Joseph Raphson simplified the approach into a practical iterative method.“ — Isaac Newton hatte die allgemeine Idee, Joseph Raphson vereinfachte den Ansatz zur praktischen iterativen Methode.

4.4 Herleitung der Newton-Raphson-Methode (S. 38)

Satz/Formel 4.1: Newton-Raphson-Iteration: Herleitung über die lineare Approximation (Gl. 20–21, S. 38)

Ziel formalisieren: Finde x^* mit $f(x^*) = 0$ (Nullstellensuche).

Schritt 1 — Lineare Approximation (Gl. 20 im Skript, S. 38):

$$f(x) \approx f(x_n) + f'(x_n) \cdot (x - x_n) \quad \text{für } x \text{ nahe } x_n \quad (\text{Gl. 20})$$

In Worten: Nahe am aktuellen Punkt x_n verhält sich f ungefähr wie seine Tangente — Funktionswert plus Steigung mal Abstand. *Randnotiz (S. 38):* Diese lineare Approximation ist genau die Taylor-Entwicklung erster Ordnung von f um x_n ; auf Taylor-Entwicklungen kommen wir zurück, wenn Genauigkeit und Approximation formal untersucht werden.

[Korrektur ggü. Original, S. 38: Das PDF druckt durch einen Layoutfehler „ $f(x) \approx f'(x_n) \cdot (x - x_n)$ for x close to $x_n + f(x_n)$ “ — der Term $+f(x_n)$ ist hinter die Nebenbedingung gerutscht. Die korrekte Form oben wird durch die Folgezeile $0 = f(x_n) + f'(x_n)(x_{n+1} - x_n)$ im Skript selbst bestätigt.]

Schritt 2 — Nächste Schätzung finden: Wir wollen wissen, wo diese lineare Approximation null kreuzt, „because this is our best guess for where the actual function crosses zero“ (denn das ist unsere beste Schätzung dafür, wo die tatsächliche Funktion die Null kreuzt). Approximation gleich null setzen und nach x_{n+1} auflösen:

$$\begin{aligned} 0 &= f(x_n) + f'(x_n) \cdot (x_{n+1} - x_n) \\ f'(x_n) \cdot (x_{n+1} - x_n) &= -f(x_n) \\ x_{n+1} - x_n &= -\frac{f(x_n)}{f'(x_n)} \end{aligned}$$

Schritt 3 — Newton-Raphson-Iterationsformel (Gl. 21 im Skript, S. 38):

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (\text{Gl. 21})$$

In Worten: Der neue Punkt ist der alte Punkt, korrigiert um den Funktionswert geteilt durch die Steigung — also genau der Schnittpunkt der Tangente mit der x -Achse. Geometrische Intuition (Figure 16, S. 38): Für $f(x) = x^2 - 2$ schneidet die Tangente bei $(x_0, f(x_0))$ mit $x_0 = 2$ die x -Achse bei $x_1 \approx 1.5$ — der nächsten Approximation.

Algorithm 1 Newton-Raphson Method (Algorithmus 1 im Skript, S. 38)

Require: Startwert x_0 , Funktion f , Ableitung f' , Toleranz ϵ

```
1:  $x \leftarrow x_0$ 
2: while  $|f(x)| > \epsilon$  do
3:   Compute derivative:  $d \leftarrow f'(x)$ 
4:   if  $|d| < \epsilon_{\text{machine}}$  then
5:     error „Derivative too small“
6:   end if
7:   Update:  $x \leftarrow x - f(x)/d$ 
8: end while
9: return  $x$ 
```

(Im PDF steht die letzte Zeile als „end whilereturn x“ — ein Layoutartefakt für „end while; return x“.) *Randnotiz zum Fehlerfall (S. 39):* „This error occurs when the derivative $f'(x_k)$ approaches zero, which would cause division by zero or numerical instability in Newton’s method.“ — Der Abbruch „Derivative too small“ schützt vor Division durch (nahezu) null und numerischer Instabilität.

4.5 Taylor-Entwicklung: Aufbau Ordnung für Ordnung (S. 39–41)

Bisher haben wir der linearen Approximation bei x_n einfach vertraut (Gl. 22 im Skript, S. 39):

$$f(x) \approx f'(x_n)(x - x_n) + f(x_n) \quad (\text{Gl. 22})$$

um schnell die nächste Schätzung x_{n+1} zu finden — „but is that trust well placed?“ (Ist dieses Vertrauen gerechtfertigt?) Die Antwort liegt in der Taylor-Entwicklung, „a fundamental tool that tells us how well polynomials approximate smooth functions at a given point“ — einem fundamentalen Werkzeug, das angibt, wie gut Polynome glatte Funktionen an einem Punkt approximieren. Das Skript leitet sie von Grundprinzipien (first principles) her und illustriert sie an der schrittweisen Approximation der Sinusfunktion (Figures 17–20, S. 39–40: Sinuskurve; konstante Approximation $P(x) = f(-1)$ als horizontale gestrichelte Linie bei ≈ -0.84 ; lineare Approximation als Tangente; quadratische Approximation als angeschmiegte Parabel, jeweils verankert bei $x = -1$).

Definition 4.1: Glattheit (Smoothness) (Randnotiz S. 39)

„A function is smooth if it has derivatives of all orders, and is thus differentiable.“ — Eine Funktion ist *glatt* (smooth), wenn sie Ableitungen aller Ordnungen besitzt und somit differenzierbar ist. [Ergänzung: Glattheit ist die Voraussetzung dafür, dass die Taylor-Entwicklung mit beliebig vielen Termen gebildet werden kann.]

Satz/Formel 4.2: Konstruktion der Taylor-Entwicklung (Gl. 23–26, S. 39–40)

Gesucht ist ein Polynom $P(x)$, das $f(x)$ nahe x_n approximiert. Idee: P soll bei x_n in immer mehr Ableitungen mit f übereinstimmen.

0. Ordnung — Funktionswert matchen (Gl. 23 im Skript, S. 39):

$$P(x_n) = f(x_n) \quad (\text{Gl. 23})$$

Das Polynom stimmt nur am Punkt selbst mit f überein. Didaktischer Querbezug des Skripts: „This comes very close to what grid search does: it only looks at the function value at discrete points, without any information about how the function behaves between those points.“ — Die 0. Ordnung entspricht der Grid Search: nur Funktionswerte an diskreten Punkten, keine Information über das Verhalten dazwischen.

1. Ordnung — erste Ableitung matchen (Gl. 24 im Skript, S. 39): Nahe x_n zählt die Steigung; wir fordern zusätzlich $P'(x_n) = f'(x_n)$:

$$P(x) = f(x_n) + f'(x_n)(x - x_n) \quad (\text{Gl. 24})$$

Das ist exakt die Tangente — und die Approximation, die Newton verwendet.

2. Ordnung — zweite Ableitung matchen (Gl. 25 im Skript, S. 40): „The curvature captures how the slope changes.“ (Die Krümmung erfasst, wie sich die Steigung ändert.) Wir fordern zusätzlich $P''(x_n) = f''(x_n)$:

$$P(x) = f(x_n) + f'(x_n)(x - x_n) + \frac{f''(x_n)}{2}(x - x_n)^2 \quad (\text{Gl. 25})$$

Warum durch 2 teilen? (Randnotiz S. 40): „When we differentiate $(x - x_n)^2$ twice, we get 2. So if we want $P''(x_n) = f''(x_n)$, we need to divide by 2 to cancel this factor.“ — Zweimaliges Ableiten von $(x - x_n)^2$ erzeugt den Faktor 2; die Division durch 2 hebt ihn auf. [Ergänzung: Allgemein erzeugt k -faches Ableiten von $(x - x_n)^k$ den Faktor $k!$ — daher die Fakultäten in der allgemeinen Formel.]

n -te Ordnung — das **allgemeine Muster** führt auf die Taylor-Entwicklung (s. Definition unten).

Randnotiz „Around a point“ (S. 40): „Visually we can see that the approximations are anchored on a specific point x_n . The Taylor expansion is a local approximation.“ — Alle Approximationen sind an einem spezifischen Punkt x_n verankert; die Taylor-Entwicklung ist eine *lokale* Approximation.

Definition 4.2: Taylor-Entwicklung (Taylor expansion) (Gl. 26, S. 40)

Die Taylor-Entwicklung einer (glatten) Funktion f um einen Punkt x_n :

$$f(x) = f(x_n) + f'(x_n)(x - x_n) + \frac{f''(x_n)}{2!}(x - x_n)^2 + \frac{f'''(x_n)}{3!}(x - x_n)^3 + \dots$$

kompakter (Gl. 26 im Skript, S. 40):

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_n)}{k!} (x - x_n)^k \quad (\text{Gl. 26})$$

In Worten: Die Funktion wird als unendliche Summe von Polynomtermen geschrieben, wobei der k -te Term die k -te Ableitung am Entwicklungspunkt verwendet, normiert durch $k!$ und gewichtet mit der k -ten Potenz des Abstands $(x - x_n)$.

Satz/Formel 4.3: Warum genügt die lineare Approximation für Newton? (S. 41)

„We want to solve $f(x^*) = 0$, not compute $f(x)$ as best as we can. Improving an approximation of course comes with a cost: We need to compute higher derivatives and evaluate more complex polynomials. In numerical computations we always need to decide where to cut off the Taylor series. We do so at some finite order, and the linear term is the first non-constant term that gives us directional information.“

In Worten: Ziel ist die *Nullstelle*, nicht die bestmögliche Funktionsapproximation. Höhere Ordnungen kosten (höhere Ableitungen berechnen, komplexere Polynome auswerten). In numerischen Berechnungen muss die Taylor-Reihe immer bei endlicher Ordnung abgeschnitten werden — und der lineare Term ist der erste nicht-konstante Term, der *Richtungsinformation* liefert. Genau diese genügt für den nächsten Schritt.

4.6 Konvergenzrate: Herleitung der quadratischen Konvergenz (S. 41–42)

Satz/Formel 4.4: Quadratische Konvergenz von Newton-Raphson (Gl. 27–33, S. 41–42)

Ausgangspunkt: Erinnerung an Kapitel 3 — eine Methode ist konvergent, wenn die Approximation einen spezifischen stabilen Grenzwert erreicht. Für Newtons Methode mit

Wurzel x^* , $f(x^*) = 0$, fordern wir (Gl. 27 im Skript, S. 41):

$$|x_n - x^*| \rightarrow 0 \quad \text{für } n \rightarrow \infty \quad (\text{Gl. 27})$$

Dank der Taylor-Entwicklung können wir quantifizieren, *wie schnell* der Fehler abnimmt.

Schritt 1 — Fehler definieren und $f(x_n)$ um die Wurzel entwickeln: Sei $e_n = x_n - x^*$ der Fehler bei Iteration n . Taylor-Entwicklung 2. Ordnung von f um die *wahre Wurzel* x^* , ausgewertet bei x_n (Gl. 28 im Skript, S. 41):

$$f(x_n) = f(x^*) + f'(x^*)(x_n - x^*) + \frac{f''(\xi)}{2}(x_n - x^*)^2 \quad (\text{Gl. 28})$$

wobei ξ ein Punkt zwischen x_n und x^* ist (Lagrange-Restglied-Form; [Ergänzung] das Restglied macht aus der abgeschnittenen Reihe eine *exakte* Gleichung — der gesamte Rest steckt im Term mit $f''(\xi)$).

Schritt 2 — Nullstelleneigenschaft ausnutzen: Da $f(x^*) = 0$, vereinfacht sich das mit $e_n = x_n - x^*$ zu (Gl. 29 im Skript, S. 41):

$$f(x_n) = f'(x^*) \cdot e_n + \frac{f''(\xi)}{2}e_n^2 \quad (\text{Gl. 29})$$

Schritt 3 — Newton-Formel auf den Fehler anwenden: Der Fehler der nächsten Iteration ist

$$e_{n+1} = x_{n+1} - x^* = x_n - \frac{f(x_n)}{f'(x_n)} - x^* = (x_n - x^*) - \frac{f(x_n)}{f'(x_n)} = e_n - \frac{f(x_n)}{f'(x_n)}$$

In Worten: Der neue Fehler ist der alte Fehler minus der Newton-Schritt.

Schritt 4 — Taylor-Ausdruck einsetzen (Gl. 30 im Skript, S. 41):

$$e_{n+1} = e_n - \frac{f'(x^*) \cdot e_n + \frac{f''(\xi)}{2}e_n^2}{f'(x_n)} \quad (\text{Gl. 30})$$

Schritt 5 — Nähe zur Wurzel ausnutzen: Für Punkte nahe x^* gilt die Annahme $f'(x_n) \approx f'(x^*)$ (Gl. 31 im Skript, S. 41):

$$e_{n+1} \approx e_n - \frac{f'(x^*) \cdot e_n + \frac{f''(\xi)}{2}e_n^2}{f'(x^*)} = e_n - e_n - \frac{f''(\xi)}{2f'(x^*)}e_n^2 \quad (\text{Gl. 31})$$

In Worten: Der Bruch zerfällt in zwei Teile; der erste Teil $f'(x^*)e_n/f'(x^*) = e_n$ *hebt den Fehlerterm erster Ordnung exakt auf* (S. 42) — das ist der didaktische Kern: Newton eliminiert den linearen Fehleranteil vollständig.

Schritt 6 — Ergebnis (Gl. 32 im Skript, S. 42): Übrig bleibt nur der quadratische Term:

$$e_{n+1} \approx -\frac{f''(\xi)}{2f'(x^*)}e_n^2 \quad (\text{Gl. 32})$$

Abstrakter (Gl. 33 im Skript, S. 42):

$$|e_{n+1}| \leq C \cdot |e_n|^2 \quad (\text{Gl. 33})$$

„This is quadratic convergence.“ — Das ist quadratische Konvergenz: Der nächste Fehler ist proportional zum *Quadrat* des aktuellen Fehlers.

Definition 4.3: Quadratische Konvergenz (quadratic convergence) (S. 42)

Der Fehler der nächsten Iteration ist ungefähr proportional zum Quadrat des aktuellen Fehlers, $|e_{n+1}| \leq C \cdot |e_n|^2$ (Gl. 33 im Skript, S. 42); C ist eine Konstante, die von Krümmung und Steigung der Funktion an der Wurzel abhängt (konkret $C = |f''(\xi)/(2f'(x^*))|$, vgl. Gl. 32). Die Konvergenzordnung ist 2. [Ergänzung: Praktisch bedeutet das eine *Verdopplung der korrekten Stellen pro Iteration*, sobald man nah genug an der Wurzel ist.]

Beispiel 4.3: Berechnung von $\sqrt{2}$ mit Newton (Tabelle 2, S. 42)

Berechnung von $\sqrt{2}$ mit Newtons Methode, Start $x_0 = 2$ („our introductory example“; Querbezug auf Figure 16 mit $f(x) = x^2 - 2$ — ein expliziter früherer $\sqrt{2}$ -Bezug taucht im Text vorher nicht auf). Tabellenunterschrift im Original: „Newton’s method for computing $\sqrt{2}$. See how the error approximately squares each iteration.“ — Man sieht, wie sich der Fehler pro Iteration näherungsweise quadriert. Vollständige Tabelle 2:

n	x_n	$ e_n $ (Fehler)
0	2.000000000	5.86×10^{-1}
1	1.500000000	8.58×10^{-2}
2	1.416666667	2.45×10^{-3}
3	1.414215686	2.12×10^{-6}
4	1.414213562	1.6×10^{-12}

[Korrektur ggü. Original, S. 42: Das PDF druckt in Zeile $n = 4$ den Wert 4.5×10^{-12} — das ist jedoch das *Residuum* $|f(x_4)| = |x_4^2 - 2| \approx 4.51 \times 10^{-12}$, nicht der Fehler $|e_4| = |x_4 - \sqrt{2}| \approx 1.6 \times 10^{-12}$, den die Spaltendefinition (und alle übrigen Zeilen) verwendet. Zudem ist $|e_3| = 2.124 \times 10^{-6}$, also korrekt gerundet 2.12×10^{-6} (PDF: 2.13×10^{-6}). Konsistenzcheck der quadratischen Konvergenz mit $C = \frac{1}{2\sqrt{2}} \approx 0.354$ (vgl. Gl. 32): $0.354 \cdot (2.12 \times 10^{-6})^2 \approx 1.6 \times 10^{-12}$ (passt exakt) — die korrigierten Werte demonstrieren Gl. 33 sogar sauberer.]

Randnotiz (S. 41), „Notice the assumption play out“: $10^{-1} \rightarrow 10^{-2} \rightarrow 10^{-3} \rightarrow 10^{-6} \rightarrow 10^{-12}$. „Once we’re close enough and our assumptions hold (after iteration 2), the error squares perfectly, demonstrating quadratic convergence.“ — Anfangs ist die Konvergenz *noch nicht* quadratisch; erst wenn man nah genug an der Wurzel ist und die Annahme $f'(x_n) \approx f'(x^*)$ greift (ab Iteration 2), quadriert sich der Fehler perfekt: $(10^{-3})^2 \sim 10^{-6}$, $(10^{-6})^2 \sim 10^{-12}$.

4.7 Failure Modes: Wie Newton scheitern kann (S. 42)

„Newton’s method can fail in several ways, which we will endure for now, and for which we will find solutions later on. As a brain teaser, it is a nice exercise to visualize the failure modes in this plot.“ (Denksportaufgabe: die Fehlermodi in Figure 21 visualisieren — der Plot zeigt $f(x) = x^3 - x$ mit mehreren Nullstellen auf ca. $[-0.4, 1.4]$.) Die drei Fehlermodi:

1. **Nullableitung (zero derivative):** Wenn $f'(x_n) = 0$, ist die Tangente horizontal und kreuzt die x -Achse nie — die Iteration ist undefiniert (Division durch null).
2. **Oszillation:** Besonders bei symmetrischen Funktionen kann Newton zwischen Punkten zykeln, statt zu konvergieren.
3. **Mehrdeutige Startpunktwahl:** Der Startwert x_0 beeinflusst stark, *welche* Wurzel gefunden wird.

Überleitung zum Sekantenverfahren: „Now something we can not endure, is if we can’t compute

$f'(x)$.“ — Was wir nicht hinnehmen können: wenn $f'(x)$ gar nicht berechnet werden kann. Randnotiz „*This can happen because*“ (S. 42) — Gründe, warum die Ableitung nicht verfügbar sein kann:

- „The derivative is expensive to compute.“ (Die Ableitung ist teuer zu berechnen.)
- „The function is given as a black box (e.g., a simulation).“ (Die Funktion ist eine Black Box, z. B. eine Simulation.)
- „The function isn’t differentiable everywhere.“ (Die Funktion ist nicht überall differenzierbar.)

4.8 Das Sekantenverfahren (Secant Method, S. 43)

Definition 4.4: Sekantenverfahren — „A poor man’s derivative“ (S. 43)

Idee: Die Ableitung wird *nur mit Funktionsauswertungen* approximiert — das eliminiert den Bedarf einer expliziten Ableitung. Differenzenquotient aus den zwei jüngsten Punkten:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

Visuell ist das die Steigung der *Sekante* durch die Punkte $(x_{n-1}, f(x_{n-1}))$ und $(x_n, f(x_n))$ auf dem Graphen von f (Figure 22, S. 43: Sekante durch $(x_0, f(x_0))$ und $(x_1, f(x_1))$ liefert als Achsenschnitt die nächste Approximation x_2). **Hinweis:** Es werden *zwei Startwerte* x_0 und x_1 benötigt (statt nur einem bei Newton).

Satz/Formel 4.5: Sekanten-Iterationsformel (Gl. 34, S. 43)

Einsetzen des Differenzenquotienten in Newtons Formel (Gl. 21) liefert die iterative Sekantenmethode (Gl. 34 im Skript, S. 43):

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \quad (\text{Gl. 34})$$

In Worten: Wie bei Newton wird der Funktionswert durch eine Steigung geteilt — nur ist die Steigung jetzt der Differenzenquotient (sein Kehrwert steht als Faktor in der Formel). Der neue Punkt ist der Schnittpunkt der Sekante mit der x -Achse.

Algorithm 2 Secant Method (Algorithmus 2 im Skript, S. 43)

Require: Startwerte x_0, x_1 , Funktion f , Toleranz ϵ

```

1: while  $|f(x_1)| > \epsilon$  do
2:   Compute secant slope:  $s \leftarrow (f(x_1) - f(x_0)) / (x_1 - x_0)$ 
3:   Store previous:  $x_{\text{old}} \leftarrow x_1$ 
4:   Update:  $x_1 \leftarrow x_1 - f(x_1)/s$ 
5:    $x_0 \leftarrow x_{\text{old}}$ 
6: end while
7: return  $x_1$ 
```

(Im PDF steht die letzte Zeile als „end whilereturn x1“ — Layoutartefakt.)

4.9 Newton und Sekante von Hand an $f(x) = x^3 - x$ (S. 44–45)

Beispiel 4.4: Newton von Hand: Wurzel von $f(x) = x^3 - x = 0$ ab $x_0 = 1.4$ (S. 44) — korrigiert

Erst geometrisch, dann von Hand. „The roots are at $x = -1, 0, 1$. Let's find the root at $x = 1$ starting from $x_0 = 1.4$.“ (Figure 23, S. 44: Kubik $f(x) = x^3 - x$ schwarz, Ableitung $f'(x) = 3x^2 - 1$ grau gestrichelt auf $[-0.4, 1.4]$; Zielwurzel bei $x = 1$ markiert, Startwert als offener Kreis bei $x_0 = 1.4$.) Taylor 1. Ordnung um x_n :

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n), \quad f(x_n) \approx f(x) - f'(x_n)(x - x_n)$$

Mit $f(x) = x^3 - x = 0$ und $f'(x) = 3x^2 - 1$; umstellen für die nächste Schätzung ab $x_0 = 1.4$ (vollständige korrekte Rechnung):

Schritt 1 ($f(1.4) = 2.744 - 1.4 = 1.344$, $f'(1.4) = 5.88 - 1 = 4.88$):

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = x_0 - \frac{x_0^3 - x_0}{3x_0^2 - 1} = 1.4 - \frac{2.744 - 1.4}{5.88 - 1} = 1.4 - \frac{1.344}{4.88} \approx 1.4 - 0.2754 \approx 1.1246$$

Schritt 2 ($f(1.1246) \approx 1.4223 - 1.1246 = 0.2977$, $f'(1.1246) \approx 3.7942 - 1 = 2.7942$):

$$x_2 = 1.1246 - \frac{1.1246^3 - 1.1246}{3(1.1246)^2 - 1} \approx 1.1246 - \frac{0.2977}{2.7942} \approx 1.1246 - 0.1065 \approx 1.0181$$

Schritt 3 ($f(1.0181) \approx 1.0553 - 1.0181 = 0.0372$, $f'(1.0181) \approx 3.1096 - 1 = 2.1096$):

$$x_3 = 1.0181 - \frac{1.0181^3 - 1.0181}{3(1.0181)^2 - 1} \approx 1.0181 - \frac{0.0372}{2.1096} \approx 1.0181 - 0.0176 \approx 1.0005$$

„The root at $x = 1$ is found quickly.“ — Die Wurzel bei $x = 1$ wird schnell gefunden: Die korrekte Newton-Folge $1.4 \rightarrow 1.1246 \rightarrow 1.0181 \rightarrow 1.0005$ konvergiert *monoton von oben* gegen die Wurzel $x = 1$ (typisch für eine konvexe Funktion rechts der Wurzel).

[Korrektur ggü. Original, S. 44: PDF druckt die Kette $1.127 \rightarrow 0.976 \rightarrow 1.014$ mit u. a. falschem Zähler 0.432 statt ≈ 0.297 (so in der selbstkonsistenten Kette oben: $f(1.1246) \approx 0.2977$; selbst mit dem PDF-gerundeten $x_1 = 1.127$ wäre $f(1.127) \approx 0.304$, nicht 0.432). Durch den zu großen Zähler springt die gedruckte Folge fälschlich unter die Wurzel und scheint um 1 zu oszillieren; tatsächlich konvergiert Newton hier *monoton von oben*. Die Methodik des Skripts (Gl. 21) ist korrekt.]

Beispiel 4.5: Sekante von Hand: $f(x) = x^3 - x$ mit $x_0 = 1.2$, $x_1 = 1.4$ (S. 44–45) — korrigiert

Das Sekantenverfahren auf dasselbe Problem anwenden — „Again first geometrically, then by hand.“

Schritt 0 — korrekte Funktionswerte:

$$f(1.2) = 1.2^3 - 1.2 = 1.728 - 1.2 = 0.528, \quad f(1.4) = 1.4^3 - 1.4 = 2.744 - 1.4 = 1.344$$

[Korrektur ggü. Original, S. 45: Das PDF druckt $f(1.2) = 0.128$ (korrekt: 0.528) und $f(1.4) = 0.744$ (korrekt: 1.344) und baut darauf die fehlerhafte Iterationskette $x_2 = 1.4 - 0.744 \cdot \frac{0.2}{0.616} \approx 1.4 - 0.241 \approx 1.159$ sowie $x_3 = 1.159 - 0.283 \cdot \frac{-0.241}{0.283 - 0.744} \approx 1.159 - 0.283 \cdot 0.522 \approx 1.159 - 0.148 \approx 1.011$ auf. Die gedruckte Kette (1.159, 1.011) beruht vollständig auf den falschen Funktionswerten; die Methodik (Gl. 34) ist korrekt. Im Folgenden die komplette Iteration mit den korrekten Werten.]

Schritt 1 (Gl. 34):

$$x_2 = x_1 - f(x_1) \cdot \frac{x_1 - x_0}{f(x_1) - f(x_0)} = 1.4 - 1.344 \cdot \frac{1.4 - 1.2}{1.344 - 0.528} = 1.4 - 1.344 \cdot \frac{0.2}{0.816} \approx 1.4 - 0.3294 \approx 1.0706$$

Funktionswert am neuen Punkt: $f(1.0706) = 1.0706^3 - 1.0706 \approx 1.2271 - 1.0706 = 0.1565$.

Schritt 2 (jüngste Punkte $x_1 = 1.4$, $x_2 = 1.0706$):

$$x_3 = x_2 - f(x_2) \cdot \frac{x_2 - x_1}{f(x_2) - f(x_1)} = 1.0706 - 0.1565 \cdot \frac{1.0706 - 1.4}{0.1565 - 1.344} = 1.0706 - 0.1565 \cdot \frac{-0.3294}{-1.1875} \approx 1.0706 - 0.0434 \approx 1.0272$$

Funktionswert: $f(1.0272) \approx 1.0838 - 1.0272 = 0.0566$.

Schritt 3 (jüngste Punkte $x_2 = 1.0706$, $x_3 = 1.0272$):

$$x_4 = 1.0272 - 0.0566 \cdot \frac{1.0272 - 1.0706}{0.0566 - 0.1565} = 1.0272 - 0.0566 \cdot \frac{-0.0434}{-0.0999} \approx 1.0272 - 0.0246 \approx 1.0026$$

Funktionswert: $f(1.0026) \approx 1.0078 - 1.0026 = 0.0052$.

Schritt 4 (jüngste Punkte $x_3 = 1.0272$, $x_4 = 1.0026$):

$$x_5 = 1.0026 - 0.0052 \cdot \frac{1.0026 - 1.0272}{0.0052 - 0.0566} = 1.0026 - 0.0052 \cdot \frac{-0.0246}{-0.0514} \approx 1.0026 - 0.0025 \approx 1.0001$$

Funktionswert: $f(1.0001) \approx 0.0002$.

Zusammenfassung der korrigierten Iteration:

n	x_n	$f(x_n)$
0	1.2	0.528
1	1.4	1.344
2	1.0706	0.1565
3	1.0272	0.0566
4	1.0026	0.0052
5	1.0001	0.0002

Die Folge konvergiert klar gegen die Zielwurzel $x = 1$. [Ergänzung: Man erkennt eine Konvergenz, die schneller als linear, aber langsamer als Newtons quadratische Konvergenz ist — der Preis dafür, dass keine echte Ableitung verwendet wird; dafür kostet jede Iteration nur *eine* neue Funktionsauswertung.]

4.10 Übungsaufgaben und Selbstreflexionsfragen (S. 45–46)

Übungen:

- **Geometrical Failure Search (S. 45):** Finde Startpunkte x_0 für die verschiedenen Fehlermodi: einen, bei dem Newton wegen Nullableitung scheitert; einen, bei dem es zu einer Nicht-Ziel-Wurzel konvergiert; und einen, bei dem es zwischen Punkten oszilliert.
- **Enter the machine (S. 45):** Konvergenz und Konvergenzraten von Grid Search, Newton und Sekantenverfahren in Python betrachten; mit verschiedenen Startpunkten experimentieren (im Original „different starting points“ [sic]) und Failure Modes in der Praxis beobachten. „Try to first find the starting points geometrically, then try finding them analytically, before finally moving over to verifying in code.“ (Code: github.com/Quillstacks/lecturecode_number, S. 44)

Self-Reflection (S. 45–46):

- Was ist der Hauptvorteil von Newtons Methode gegenüber Grid Search?

- Warum ist eine lineare Approximation für die Nullstellensuche ausreichend? Was sagt die Taylor-Entwicklung über diese Wahl?
- Warum ist quadratische Konvergenz so mächtig? Wie viele Iterationen bräuchte Newton, um von Fehler 10^{-1} auf Fehler 10^{-16} zu kommen?
- In welchen Situationen würdest du das Sekantenverfahren Newton vorziehen? Warum nicht in anderen?

4.11 Recap und Ausblick (S. 46)

- „Newton-Raphson gives (limited) convergence guarantees near simple roots.“ — Newton-Raphson gibt (begrenzte) Konvergenzgarantien nahe einfacher Wurzeln (simple roots).
- „Taylor expansion provides a systematic way to approximate functions locally.“ — Die Taylor-Entwicklung ist der systematische Weg, Funktionen lokal zu approximieren.
- Die Taylor-Entwicklung rechtfertigt die lineare Approximation in Newtons Methode, erklärt die quadratische Konvergenz und erlaubt eine Fehlerschätzung in jeder Iteration.
- Wenn die Ableitung schwer zu berechnen ist, approximiert das Sekantenverfahren die Ableitung aus zwei Punkten und konvergiert.

Überleitung — Local methods find local solutions (S. 46): „Newton converges to whatever optimum is nearby, it is also prone to oscillate between points, or diverge if the derivative is zero or near zero. In the next chapter, we will reformulate the root finding into an optimization problem. And from there we’ll explore how to go on and handle global optimization when the landscape has many valleys or is otherwise pathologically difficult.“ — Lokale Methoden finden lokale Lösungen; im nächsten Kapitel wird die Nullstellensuche als Optimierungsproblem reformuliert und anschließend die globale Optimierung behandelt. *Randnotiz-Teaser:* „How can we make sure that we find the global solution, and not just a local one?“

[Ergänzung mit Vorgriff auf Kapitel 5: Dort wird Newton auf die *Ableitung* f' angewendet, um Extremstellen zu finden — also Punkte mit $f'(x) = 0$; der anschließende Second-Derivative-Test klassifiziert sie über f'' . Dazu gehört auch eine Korrektur: [Korrektur ggü. Original, S. 53: Das PDF druckt dort „Newton’s method finds points where $f''(x) = 0$ “ — ein Druckfehler; Newton auf f' findet Punkte mit $f'(x) = 0$.]]

Typische Fehlerquellen & Merksätze

Typische Fehlerquellen:

- **Vorzeichen-/Formelfehler:** Newton ist $x_{n+1} = x_n - f(x_n)/f'(x_n)$ (Gl. 21) — *minus*, und der Funktionswert steht im Zähler, die Ableitung im Nenner. Bei der Sekante (Gl. 34) wird durch die *Differenz der Funktionswerte* geteilt, nicht durch die Schrittweite.
- **Nullableitung übersehen:** $f'(x_n) \approx 0$ bedeutet horizontale Tangente $\Rightarrow$ Division durch (fast) null. Algorithmus 1 fängt das explizit mit „Derivative too small“ ab.
- **Quadratische Konvergenz gilt erst nahe der Wurzel:** Die Herleitung benutzt $f'(x_n) \approx f'(x^*)$. Weit weg von x^* kann Newton oszillieren, zur falschen Wurzel laufen oder divergieren (vgl. $\sqrt{2}$ -Tabelle: erst ab Iteration 2 quadriert sich der Fehler).
- **Sekante braucht zwei Startwerte** (x_0 und x_1) — Newton nur einen, dafür aber die Ableitung.
- **Rechenfehler-Fallen aus dem Original:** Newton an $x^3 - x$: korrekte Folge $1.1246 \rightarrow 1.0181 \rightarrow 1.0005$, monoton von oben (nicht $1.127 \rightarrow 0.976 \rightarrow 1.014$, S. 44); $f(1.2) = 0.528$ und $f(1.4) = 1.344$ (nicht $0.128/0.744$, S. 45); $\sin(3.125) \approx 0.0166$ (nicht 0.0008 , S. 37); Gl. 20 lautet $f(x) \approx f(x_n) + f'(x_n)(x - x_n)$ (S. 38); Tabelle 2: $|e_3| = 2.12 \times 10^{-6}$ und $|e_4| \approx 1.6 \times 10^{-12}$ (das gedruckte 4.5×10^{-12} ist das Residuum $|f(x_4)|$, S. 42).

Merksätze:

- *Newton = Tangente schneidet Achse; Sekante = Sehne schneidet Achse.*
- *Die Ableitung sagt, wohin — der Funktionswert, wie weit:* Genau das unterscheidet Newton von der blinden $O(N^d)$ -Gittersuche.
- *Quadratische Konvergenz = Stellenverdopplung pro Iteration:* $10^{-1} \rightarrow 10^{-2} \rightarrow 10^{-4} \rightarrow 10^{-8} \rightarrow 10^{-16}$ — vier Iterationen von Fehler 10^{-1} zu 10^{-16} .
- *Newton 1. Ordnung = Gradient Descent:* Deep Learning ist im Kern gradientenbasierte Navigation in der Loss-Landschaft.
- *Taylor ist lokal:* Alle Approximationen sind an einem Punkt x_n verankert — Vertrauen in die Tangente ist nur *nahe* x_n gerechtfertigt.

5 Thema 5: Globale Optimierung (Global Optimization) [S. 47–56]

5.1 Motivation und Einordnung [S. 47]

Kapitel motto: „And then there were many.“ — Im vorigen Kapitel wurden Newtons Methode und der Gradientenabstieg (gradient descent) für die Nullstellensuche entwickelt. Diese Verfahren sind jedoch *lokale* Methoden: Sie konvergieren zu derjenigen Lösung, die in der Nähe des Startpunkts liegt. Was aber, wenn diese nahe Lösung schlecht ist — oder eine viel bessere Lösung weiter entfernt liegt? Das Skript formuliert es so: „Time to do away with oversimplifications.“ In diesem Kapitel lernen wir Shekel’s Foxholes als klassische Testfunktion für globale Optimierung kennen, sehen, wie sich ein Optimierungsproblem als Nullstellenproblem (root-finding problem) formulieren lässt, und führen Strategien zum Finden *globaler* Minima ein.

Der Bezug zum maschinellen Lernen ist zentral: Im Deep Learning ist das Verständnis multidimensionaler und globaler Optimierung entscheidend, damit große Modelle effektiv trainieren, konvergieren und tatsächlich etwas Nützliches lernen. Die Loss-Landschaften neuronaler Netze (neural network loss landscapes, Li et al. 2018, Fußnote 8 [S. 47]) sind hochgradig nicht-konvex, besitzen flache Regionen und enthalten viele lokale Minima — verschiedene Initialisierungen führen zu verschiedenen Endlösungen.

5.2 Lernziele (Learning Objectives) [S. 50]

1. Erklären können, warum lokale Methoden das Finden globaler Minima nicht garantieren können.
2. Strategien zum Finden globaler Extrema anwenden können.
3. Verstehen, wie der Gradientenabstieg (gradient descent) hilft, die Optimierung in Richtung von Minima zu lenken.

5.3 Hands On: Shekel’s Foxholes [S. 48–50]

Definition 5.1: Shekel’s Foxholes (1D-Testfunktion), Gl. (35) [S. 48]

Shekel’s Foxholes ist eine klassische Testfunktion für globale Optimierung mit *kontrollierter Multimodalität* (controlled multimodality), d. h. man kann gezielt einstellen, wie viele Minima die Funktion hat und wie sie aussehen. In 1D lautet die einfache Form (Original-Gl. (35); Fußnote 9: J. Shekel, „Test functions for multimodal search techniques“, 1971):

$$f(x) = - \sum_{i=1}^m \frac{c_i}{(x - a_i)^2 + r_i} \quad (35)$$

Dabei sind a_i die Positionen der „Fuchslöcher“ (foxholes), c_i steuert die *Tiefe* jedes Lochs und r_i die *Breite*. Durch Anpassen dieser Parameter lässt sich eine Landschaft mit meh-

renen lokalen Minima und einem globalen Minimum erzeugen — „an ideal playground for testing optimization algorithms“.

[**Ergänzung**] Intuition: Jeder Summand ist ein nach unten gezogener „Trichter“ um die Stelle a_i ; je kleiner r_i , desto schmaler und (bei festem c_i) tiefer ist der Trichter, denn am Lochzentrum $x = a_i$ gilt $f \approx -c_i/r_i$ für diesen Term.

Beispiel 5.1: Konkrete Shekel-Instanz mit drei Foxholes [S. 48]

Im Skript wird folgende konkrete Instanz mit $m = 3$ Löchern verwendet:

$$f(x) = -\frac{1}{(x-0)^2 + 0.7} - \frac{1.7}{(x-4)^2 + 0.2} - \frac{1.1}{(x-7)^2 + 0.4}$$

Das **globale Minimum** liegt bei $x = 4$ (tiefstes Loch: großes $c_2 = 1.7$, kleines $r_2 = 0.2$); die beiden anderen Minima bei $x = 0$ und $x = 7$ sind nur *lokale* Minima. Wichtig: Diese Information steht uns beim Lauf eines Optimierungsalgorithmus natürlich *nicht* zur Verfügung — wir sehen nur die Funktionswerte und Gradienten an den ausgewerteten Punkten. Für das Studium des Verhaltens von Optimierungsmethoden ist das Wissen aber nützlich.

Sensitivität gegenüber dem Startpunkt x_0 : Startet Newtons Methode nahe $x = 0$ oder $x = 7$, konvergiert sie zu einem *lokalen* Minimum — nicht zum globalen bei $x = 4$ (vgl. Figure 24 [S. 48]: Funktion mit Tälern bei $x = 0, 4, 7$; die Ableitung kreuzt an jedem Minimum die Null).

Satz/Formel 5.1: Optimierung als Nullstellensuche: Newton-Update, Gl. (36) [S. 49]

Um Newtons Methode zur *Optimierung* (statt Nullstellensuche) einzusetzen, lösen wir das Nullstellenproblem auf der *Ableitung*: $f'(x) = 0$. Das ist die zentrale Reformulierung des Optimierungsproblems. Das Newton-Update wird dann (Original-Gl. (36)):

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)} \quad (36)$$

In eigenen Worten: Beim gewöhnlichen Newton-Verfahren für Nullstellen von g lautet das Update $x_{n+1} = x_n - g(x_n)/g'(x_n)$. Setzt man $g = f'$, wird daraus genau Gl. (36): Zähler f' , Nenner f'' . Randnotiz [S. 48]: Die als gestrichelte Linie dargestellte Ableitung *ist* die Antwort darauf, wie man Optimierung als Nullstellensuche formuliert — „The minima of $f(x)$ correspond to the roots of $f'(x)$.“ An Extrema ist $f'(x)$ null; gefunden werden dabei allerdings Minima *oder* Maxima (Klassifikation: siehe Second-Derivative-Test weiter unten).

Beispiel 5.2: Newton auf der Shekel-Funktion ab $x_0 = 7.2$ (vollständige Rechnung) [S. 49]

Wir wenden Gl. (36) auf die obige Drei-Löcher-Instanz an. Die Ableitungen vereinfachen sich zu (Quotientenregel auf jeden Summanden; das Minuszeichen der Funktion dreht das

Vorzeichen):

$$f'(x) = \frac{2x}{(x^2 + 0.7)^2} + \frac{3.4(x-4)}{((x-4)^2 + 0.2)^2} + \frac{2.2(x-7)}{((x-7)^2 + 0.4)^2}$$

Auswertung bei $x_0 = 7.2$:

$$f'(7.2) \approx 0.005 + 0.100 + 2.27 = 2.38$$

Anmerkung zur Rundung [S. 49]: Die drei Summanden ergeben arithmetisch 2.375; das PDF rundet zulässig auf 2.38 — keine Korrektur erforderlich.

$$f''(x) = \frac{2(0.7 - 3x^2)}{(x^2 + 0.7)^3} + \frac{3.4(0.2 - 3(x-4)^2)}{((x-4)^2 + 0.2)^3} + \frac{2.2(0.4 - 3(x-7)^2)}{((x-7)^2 + 0.4)^3}$$

$$f''(7.2) \approx -0.002 - 0.091 + 7.23 = 7.14$$

(Vgl. Figure 25 [S. 49]: f' schwarz, f'' grau gestrichelt auf der Shekel-Instanz; „the second derivative is positive at minima, confirming local curvature“ — $f'' > 0$ an den Minima bestätigt die lokale Krümmung.) Damit lauten die **Newton-Schritte**:

$$x_1 = 7.2 - \frac{2.38}{7.14} \approx 6.87, \quad x_2 \approx 7.02, \quad x_3 \approx 7.00$$

[Anm.: Exakt nachgerechnet ist $x_3 \approx 6.9907$ (≈ 6.99 , PDF: 7.00); das lokale Minimum liegt durch die Überlappung der drei Foxhole-Terme knapp unterhalb von 7 bei $x \approx 6.9907$. Die Kernaussage (Konvergenz zum lokalen Minimum $x^\circ \approx 7$) bleibt unverändert richtig.]

Kernbeobachtung [S. 49]: Nullstellenverfahren lassen sich zur Optimierung nutzen, indem man sie auf die *Ableitung* anwendet: $f'(x)$ ist an Extrema null, dort finden wir also Minima oder Maxima.

Das fundamentale Problem [S. 50]: Hier hatten wir Pech — die Methode wird vom nahegelegenen Foxhole bei $x^\circ \approx 7$ angezogen (zur Notation x° siehe unten). Lokale Optimierung garantiert nur das Finden *nahe* Extrema. Selbst mit Konvergenzgarantien: Von $x_0 = 3$ aus hätten wir Glück gehabt, von $x_0 = -2$ oder $x_0 = 8$ aus verfehlen wir das globale Minimum. — Genug Motivation für die globalen Optimierungsstrategien.

Randnotiz Notation [S. 49]: Der Stern (*) ist konventionell für *globale* Optima reserviert. Zur Unterscheidung lokaler von globalen Minima bieten sich alternative Notationen wie x° (x-circle) oder x_i für das i -te lokale Minimum an.

5.4 Definitionen: Globale Optimierungsstrategien [S. 51–53]

Definition 5.2: Lokale vs. globale Optimierung, Gl. (37) und (38) [S. 51]

Lokale Optimierung (local optimization) findet ein *nahe*s Minimum:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathcal{N}(\mathbf{x}_0)} f(\mathbf{x}) \quad (37)$$

wobei $\mathcal{N}(\mathbf{x}_0)$ eine *Nachbarschaft* (neighborhood) des Startpunkts ist. Randnotiz [S. 51]: $\mathcal{N}$ ist eine *Menge* von Punkten um $\mathbf{x}_0$, z. B. $\mathcal{N}(\mathbf{x}_0) = \{\mathbf{x} : \|\mathbf{x} - \mathbf{x}_0\| < \epsilon\}$ für einen Radius ϵ .

Globale Optimierung (global optimization) findet das *beste* Minimum:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \Omega} f(\mathbf{x}) \quad (38)$$

wobei Ω die gesamte Suchdomäne ist. Das Skript merkt an: „It is easy to see how the search domain can be infinitely [sic] large.“ — die Suchdomäne kann unendlich groß sein, was die Aufgabe fundamental schwerer macht.

Definition 5.3: Konvexität als Trennlinie (convexity is the dividing line) [S. 51]

Eine Funktion f heißt **konvex** (convex), wenn

$$f(\lambda x + (1 - \lambda) y) \leq \lambda f(x) + (1 - \lambda) f(y) \quad \text{für alle } \lambda \in [0, 1].$$

[**Ergänzung**] Anschaulich: Die Verbindungsstrecke (Sehne) zwischen zwei beliebigen Kurvenpunkten liegt nie unterhalb des Graphen — die Funktion hat eine einzige „Schüssel“-Form ohne Nebentäler.

Für konvexe Funktionen ist *jedes lokale Minimum das globale Minimum* — lokale Methoden genügen; jede lokale Methode findet das globale Optimum. Die Schwierigkeit entsteht erst bei **nicht-konvexer** Optimierung, deren Landschaft mehrere lokale Minima, Sattelpunkte (saddle points) und Plateaus enthält — wie Shekel’s Foxholes oder die Loss-Flächen neuronaler Netze.

Definition 5.4: Anziehungsgebiet (basin of attraction) [S. 51, 55]

Das **Anziehungsgebiet** eines (lokalen) Minimums ist die Menge aller Startpunkte, von denen aus ein lokaler Optimierer zu genau diesem Minimum konvergiert. [**Ergänzung**] Die Suchdomäne zerfällt dadurch in „Einzugsgebiete“: Wo immer man startet, rutscht der lokale Optimierer in das Minimum des jeweiligen Basins. Der Anteil p , den das Basin des *globalen* Minimums an der Suchdomäne hat, bestimmt direkt die Erfolgswahrscheinlichkeit von Random Restarts (Gl. (39)). Randnotiz-Übung [S. 55]: Shekel ist insgesamt nicht-konvex, die einzelnen Basins sind es jedoch; man soll auch eine Funktion notieren, bei der die Basins *nicht* so leicht zu kartieren sind.

Definition 5.5: Random Restarts [S. 51]

Random Restarts (zufällige Neustarts) ist die einfachste globale Strategie: Man startet den lokalen Optimierer wiederholt von zufällig gezogenen Startpunkten der Suchdomäne und behält die beste gefundene Lösung. Das Skript kommentiert: „in its notion it feels like brute-forcing again“ — vom Ansatz her fühlt es sich wieder wie Brute-Forcing an. Ablauf siehe Algorithmus ?? (Original: Algorithmus 3). Randnotiz [S. 51]: Das Training mehrerer neuronaler Netze mit verschiedenen Seeds ist *implizites* Random Restarts — für große Modelle jedoch nicht praktikabel.

Algorithm 3 Random Restarts (Original: Algorithmus 3 [S. 51])

Require: Suchdomäne Ω , lokaler Optimierer LOCALOPTIMIZE, Anzahl Restarts K

```
1:  $x_{\text{best}} \leftarrow \text{NONE}; f_{\text{best}} \leftarrow +\infty$ 
2: for  $k = 1$  to  $K$  do
3:    $x_0 \leftarrow \text{RANDOMPOINT}(\Omega)$ 
4:    $x^* \leftarrow \text{LOCALOPTIMIZE}(x_0)$ 
5:   if  $f(x^*) < f_{\text{best}}$  then
6:      $x_{\text{best}} \leftarrow x^*; f_{\text{best}} \leftarrow f(x^*)$ 
7:   end if
8: end for
9: return  $x_{\text{best}}$ 
```

Satz/Formel 5.2: Erfolgswahrscheinlichkeit von Random Restarts, Gl. (39)
[S. 51]

Hat das Anziehungsgebiet des globalen Minimums die Wahrscheinlichkeit p (Anteil an der Suchdomäne, aus der ein zufälliger Startpunkt ins globale Basin fällt), dann gilt mit K Restarts:

$$P(\text{find global}) = 1 - (1 - p)^K \quad (39)$$

In eigenen Worten: Ein einzelner Restart *verfehlt* das globale Basin mit Wahrscheinlichkeit $1 - p$. Da die Restarts unabhängig sind, verfehlen alle K Versuche gemeinsam mit Wahrscheinlichkeit $(1 - p)^K$; das Komplement davon ist die Erfolgswahrscheinlichkeit. Zahlenbeispiel des Skripts: Für $p = 0.1$ und $K = 20$ ist $P = 1 - 0.9^{20} \approx 0.88$.

Auflösen nach K (Randnotiz-Hint [S. 54]): Aus $P = 1 - (1 - p)^K$ folgt

$$K = \frac{\ln(1 - P)}{\ln(1 - p)}.$$

Praxisproblem: Üblicherweise ist p vorab *nicht* bekannt. Heuristik für K ohne Kenntnis von p : K erhöhen, bis sich die beste Lösung stabilisiert (keine Verbesserung nach mehreren Restarts) — siehe Patience.

Definition 5.6: Patience (Geduld-Hyperparameter) [S. 51]

Patience ist der übliche Hyperparameter der K -Heuristik: Er kontrolliert, wie viele Restarts *ohne Verbesserung* abgewartet werden, bevor gestoppt wird. **[Ergänzung]** Dieselbe Idee kennt man aus dem Deep Learning als Early-Stopping-Patience: Statt eine feste Versuchszahl vorab festzulegen, beendet man adaptiv, wenn weitere Versuche längere Zeit nichts mehr verbessern.

Definition 5.7: Basin Hopping [S. 52]

Basin Hopping erkundet die Landschaft durch *Abwechseln* von lokaler Optimierung und zufälligen Sprüngen (random jumps). Das ist verschieden von Random Restarts, das die Suche jeweils *komplett zurücksetzt*: Basin Hopping erlaubt das Entkommen aus lokalen Minima, während weiterhin lokale Optimierung genutzt wird, um bessere Lösungen zu finden — man springt vom gerade gefundenen lokalen Minimum aus weiter, statt blind neu zu starten. Die **Sprungverteilung** (jump distribution) wird als $\delta \sim \mathcal{D}$ spezifiziert — z. B. eine bei null zentrierte Gauß-Verteilung oder eine Gleichverteilung über einen bestimmten Bereich. Ablauf: Algorithmus ?? (Original: Algorithmus 4).

Algorithm 4 Basin Hopping (Original: Algorithmus 4 [S. 52])

Require: Startpunkt x_0 , lokaler Optimierer LOCALOPTIMIZE, Sprungverteilung für δ , Iterationen K

```
1:  $x_{\text{best}} \leftarrow x_0$ ;  $f_{\text{best}} \leftarrow f(x_0)$ 
2:  $x_{\text{current}} \leftarrow x_0$ 
3: for  $k = 1$  to  $K$  do
4:    $x^* \leftarrow \text{LOCALOPTIMIZE}(x_{\text{current}})$ 
5:   if  $f(x^*) < f_{\text{best}}$  then
6:      $x_{\text{best}} \leftarrow x^*$ ;  $f_{\text{best}} \leftarrow f(x^*)$ 
7:   end if
8:   Sprung ziehen:  $\delta \sim \mathcal{D}$ 
9:    $x_{\text{current}} \leftarrow x^* + \delta$  ▷ Random jump
10: end for
11: return  $x_{\text{best}}$ 
```

Definition 5.8: Simulated Annealing (simuliertes Abkühlen) [S. 52]

Simulated Annealing kombiniert Basin Hopping mit einem *probabilistischen Akzeptanzkriterium* für Kandidatenpunkte: Zu Beginn wird ein Sprung in eine *schlechtere* Lösung mit Wahrscheinlichkeit

$$\exp\left(-\frac{\Delta f}{T}\right)$$

akzeptiert, wobei T (*Temperatur*) über die Zeit abnimmt. Randnotiz [S. 52]: Δf ist der *Anstieg* des Zielfunktionswerts beim Übergang von der aktuellen Lösung zu einer schlechteren Kandidatenlösung. Später wird der Algorithmus konservativer und akzeptiert nur noch Kandidaten, die das Ziel verbessern — so kann er zu einem Optimum konvergieren. **[Ergänzung]** Intuition: Bei hohem T ist $\Delta f/T$ klein, die Akzeptanzwahrscheinlichkeit also nahe 1 — viel Exploration; bei $T \rightarrow 0$ geht sie für jede Verschlechterung gegen 0 — reine Ausbeutung (Exploitation). Der Name stammt vom kontrollierten Abkühlen (Annealing) in der Metallurgie.

Definition 5.9: Stochastische Methoden (stochastic methods) und Noisy Newton [S. 52–53]

Stochastische Methoden fügen Rauschen hinzu, indem die Loss-Landschaft *approximiert* wird, um lokalen Optima zu entkommen. Beispielaufgabe des Skripts: die 1D-Shekel-Funktion als Funktion zweiter Ordnung approximieren, indem man drei Punkte auf der Foxhole-Kurve wählt (vgl. Figure 26 [S. 53]: die Parabel durch drei Stützpunkte erfasst den allgemeinen Trend, glättet aber die einzelnen Foxholes weg).

Noisy Newton (Randnotiz [S. 52]): folgt derselben Hopping-Idee, indem Gauß-Rauschen direkt zum Newton-Schritt addiert wird: $\mathbf{x}_{n+1} = \dots + \boldsymbol{\xi}_n$.

The take away [S. 52–53]: Die Loss-Landschaft ist *nicht statisch*, sondern kann konstruiert und geformt werden (loss landscape engineering). Durch Approximation der Loss-Landschaft auf Mini-Batches (jeweils andere Punktauswahl) erhält man eine *verrauschte Schätzung* der wahren Loss-Landschaft, die hilft, lokalen Minima zu entkommen. Die Loss-Landschaft wird natürlich auch stark durch die Wahl der Loss-Funktion selbst beeinflusst. Aber: Der Newton-Schritt ist sehr *rauschempfindlich* — später werden Ansätze folgen, die damit besser umgehen.

5.5 Newtons Methode zum Finden von Minima [S. 53]

Satz/Formel 5.3: Newton-Update für Minima und Second-Derivative-Test, Gl. (40) [S. 53]

Die im Hands-on gesehene Reformulierung von Optimierung als Nullstellensuche erlaubt Newtons Methode zum Finden von Optima (Original-Gl. (40), identisch zu Gl. (36)):

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)} \quad (40)$$

Minima, Maxima und Sattelpunkte: Newtons Methode (auf f' angewandt) findet Punkte mit $f'(x) = 0$ [Korrektur ggü. Original, S. 53: das PDF druckt „Newton’s method finds points where $f''(x) = 0$ “; sachlich findet Newton auf f' angewandt Punkte mit $f'(x) = 0$ (Extremstellen); erst der anschließende Second-Derivative-Test klassifiziert sie über f''] — diese Punkte können aber Minima, Maxima oder Sattelpunkte sein. Der **Test der zweiten Ableitung** (second derivative test) unterscheidet:

- (a) $f''(x^*) > 0$: lokales **Minimum** („curve bends upward“ — die Kurve biegt nach oben);
- (b) $f''(x^*) < 0$: lokales **Maximum** („curve bends downward“ — die Kurve biegt nach unten);
- (c) $f''(x^*) = 0$: **nicht schlüssig** (inconclusive) — möglicher Sattelpunkt oder Wendepunkt (saddle point / inflection point).

Satz/Formel 5.4: Newton Richtung Minima lenken: der $|f''|$ -Trick, Gl. (41) [S. 53–54]

Modifiziertes Update mit dem *Betrag* der zweiten Ableitung im Nenner (Original-Gl. (41)):

$$x_{n+1} = x_n - \frac{f'(x_n)}{|f''(x_n)|} \quad (41)$$

So wird sichergestellt, dass wir uns *immer* in Richtung abnehmenden $f(x)$ bewegen: Der Schritt geht stets in Richtung des negativen Gradienten („downhill so to speak“). **In eigenen Worten / [Ergänzung]:** Beim reinen Newton-Update (Gl. (40)) bestimmt das Vorzeichen von f'' die Schrittrichtung mit. In einer Region mit $f'' < 0$ (Kurve biegt nach unten, also nahe einem Maximum) zeigt der Newton-Schritt *bergauf* — Newton wird vom Maximum angezogen. Nimmt man $|f''|$, bleibt die adaptive Schrittweite $1/|f''|$ erhalten, aber die Richtung ist immer $-\text{sign}(f'(x_n))$, also bergab. Geometrisch (Figures 27/28 [S. 54], Beispiel am Punkt $x = 6.1$): Die Tangente an f' hat dort Steigung $f''(6.1) \approx -2.98$ [Korrektur ggü. Original, S. 54: Die Captions von Fig. 27/28 drucken $\approx \mp 2.66$ — das ist f'' bei $x \approx 6.0$ ($f''(6.0) \approx -2.63$); bei $x = 6.1$ ist $f'' \approx -2.98$]; der Ersatz $f'' \rightarrow |f''|$ klappt die Steigung auf $\approx +2.98$ um — das Vorzeichen der Krümmung wird entfernt, Abwärtsschritte werden erzwungen und die Gefahr sinkt, auf ein lokales Maximum zuzulaufen. Randnotiz-Denkfrage [S. 53]: Wie müsste das Update modifiziert werden, um stattdessen *Maxima* zu finden?

Randnotiz „In deep learning“ [S. 53] — die vier DL-Hoffnungen: Im Deep Learning verwendet man typischerweise lokale Methoden (SGD, Adam), aber *hofft*, dass: (1) die meisten lokalen Minima ungefähr gleich gut sind, (2) Überparametrisierung (overparameterization) schlechte Minima selten macht, (3) Mini-Batch-Rauschen hilft, scharfen

Minima zu entkommen, (4) adaptive Lernraten helfen, Plateaus zu durchqueren.

5.6 Durchgerechnete Beispiele: Strategien auf konkreten Zahlen [S. 54–55]

Beispiel 5.3: Random Restarts: Erfolgswahrscheinlichkeit und nötige Restart-Zahl [S. 54–55]

Aufgabe: Eine Funktion hat 5 lokale Minima; das Anziehungsgebiet des globalen Minimums deckt 15 % der Suchdomäne ab ($p = 0.15$). Wie groß ist die Wahrscheinlichkeit, das globale Minimum mit $K = 10$ Random Restarts zu finden?

Rechnung mit Gl. (39):

$$P = 1 - (1 - 0.15)^{10} = 1 - 0.85^{10} = 1 - 0.1969 = 0.803$$

Etwa 80 % Chance — „not too bad, but far from certain“ (nicht schlecht, aber weit von Gewissheit entfernt).

Folgefrage: Wie viele Restarts brauchen wir für 99 %? Mit der nach K aufgelösten Formel (Randnotiz-Hint [S. 54]: $K = \ln(1 - P) / \ln(1 - p)$):

$$K = \frac{\ln(1 - 0.99)}{\ln(1 - 0.15)} = \frac{\ln(0.01)}{\ln(0.85)} = \frac{-4.605}{-0.1625} \approx 28.3$$

[S. 55] Da K ganzzahlig sein muss und das Ziel erreicht werden soll, wird aufgerundet: $K = 29$ Restarts.

[**Ergänzung**] Beachte die ungünstige Skalierung: Je kleiner das globale Basin ($p \downarrow$), desto stärker wächst K , denn $\ln(1 - p) \approx -p$ für kleine p , also $K \approx -\ln(1 - P)/p$ — der Aufwand wächst etwa wie $1/p$. (Anschlussübung des Skripts [S. 55]: K für $p = 0.05$ und $p = 0.01$ beim selben 99%-Ziel berechnen — siehe Übungsteil.)

5.7 Wissenswertes aus Randnotizen und Fußnoten

- **Loss-Landschaften neuronaler Netze [S. 47]:** Li et al. (2018, Fußnote 8: „Visualizing the loss landscape of neural nets“, NeurIPS) zeigen das komplexe Terrain der Optimierung neuronaler Netze — verschiedene Initialisierungen führen zu verschiedenen Endlösungen.
- **Fußnote 9 [S. 48]:** J. Shekel, „Test functions for multimodal search techniques“, 1971 — Ursprung der Shekel-Funktion.
- **Implizite Random Restarts [S. 51]:** Das Training mehrerer Netze mit verschiedenen Seeds entspricht implizitem Random Restarts; für große Modelle nicht praktikabel.
- **Code [S. 54]:** https://github.com/Quillstacks/lecturecode_numericalmethods.git
- **Teaser [S. 56]:** Gradientenabstieg neigt zum Steckenbleiben — „so far our answer was merely better than brute-forcing“. Nächstes Kapitel: die Loss-Landschaft *glätten*, um die Robustheit des Gradientenabstiegs zu erhöhen. Randnotiz-Denkfrage: Was, wenn wir einer *Hochfrequenz-Version* von Shekel’s Foxholes gegenüberstehen — welches Problem entsteht? (Antwort in Thema 6.)

5.8 Übungsaufgaben und Selbstreflexionsfragen des Skripts

Übungsaufgaben [S. 48–55] (nur Fragen):

- Newton auf der 1D-Shekel-Funktion von einem *anderen* Startpunkt als $x_0 = 7.2$ anwenden und beobachten, wohin die Methode konvergiert [S. 48].
- Die 1D-Shekel-Funktion als Funktion zweiter Ordnung approximieren, indem drei Punkte auf der Foxhole-Kurve gewählt werden [S. 52].
- Berechne K für $p = 0.05$ und $p = 0.01$ beim selben 99%-Ziel [S. 55].
- *On your machine* [S. 55]: Eigene 1D-Shekel-Funktion designen (anfangs einfach halten, später komplexer machen); die bisher gelernten lokalen Optimierungsmethoden darauf anwenden; zeigen, wo Newtons Methode (wieder) scheitert; verschiedene x_0 erkunden und das Konvergenzverhalten beobachten; über Loss-Landschaft und Basin-Grenzen nachdenken.
- *Directing the search towards minima* [S. 55]: Der $|f''|$ -Trick dreht das Vorzeichen der Krümmung im Nenner um und erzwingt Bergab-Schritte — was bedeutet das geometrisch für die Tangentenkonstruktion? Danach im Code ausprobieren.
- *Escaping Local Minima* [S. 55]: Random Restarts und Basin Hopping auf der eigenen 1D-Shekel-Funktion einsetzen; vergleichen, wie viele Funktionsauswertungen jede Methode zum Finden des globalen Minimums braucht. Wie anfällig sind sie für das Steckenbleiben in lokalen Minima? Wie beeinflusst die Wahl der Schrittweitenverteilung die Performance von Basin Hopping?
- *Noise and landscape engineering* [S. 55]: Die Loss-Landschaft ist nicht statisch, sondern zu konstruieren und zu formen; durch Approximation auf Mini-Batches (verschiedene Punktauswahlen) ... [Anm.: Der Satz endet im Original abrupt mit Komma — unvollständig im PDF, Inhalt nur teilweise rekonstruierbar.]
- *Code exercise* [S. 55]: Random Restarts und Basin Hopping auf der 1D-Shekel-Funktion implementieren; vergleichen, wie viele Funktionsauswertungen jede Methode braucht, um das globale Minimum bei $x \approx 4$ zu finden.
- *Basin of Attraction* (Randnotiz [S. 55]): Die Plots der Shekel-Funktion erneut ansehen, die Anziehungsgebiete jedes lokalen Minimums markieren und ihre relativen Größen schätzen. Welches gehört zum globalen Minimum? Wie hängt das mit der Erfolgswahrscheinlichkeit von Random Restarts zusammen?
- *Non-Convex Basins* (Randnotiz [S. 55]): Shekel ist insgesamt nicht-konvex, die Basins jedoch schon — eine Funktion notieren, bei der die Anziehungsgebiete nicht so leicht zu kartieren sind.
- *Schrittweiten-Heuristik* (Randnotiz [S. 55]): Gibt es einen klugen Weg, die Schrittweitenverteilung während der Optimierung automatisch anzupassen? Was wäre eine gute Heuristik? Wie verhindert man katastrophale Sprünge? Implementieren.
- *Basins of Attraction kartieren* [S. 55]: Für die 1D-Shekel-Funktion (Annahme: ein lokaler Optimierer konvergiert immer zum nächstgelegenen Foxhole [Anm.: im PDF folgt ein Fußnotenrest „foxhole.2“]):
 - (a) Die Anziehungsgebiete der drei Foxholes bei $x = 0, 4, 7$ grob skizzieren — wo liegen die Basin-Grenzen?
 - (b) Wenn Basin Hopping einen Sprung $\delta \sim \text{Uniform}(-3, 3)$ vom lokalen Minimum $x^\circ = 7$ aus nutzt: Mit welcher Wahrscheinlichkeit landet ein einzelner Sprung im Basin des globalen Minimums?
 - (c) Warum könnte Basin Hopping das globale Minimum hier schneller finden als Random Restarts?

Selbstreflexionsfragen [S. 56]:

- Warum scheitert lokale Optimierung auf nicht-konvexen Landschaften wie Shekel's Fox-holes?
- Wie skaliert der nötige Aufwand bei Random Restarts mit der Größe des globalen Basins?
- Wie unterscheidet sich Basin Hopping von Random Restarts? [Originalfrage grammatisch fehlerhaft: „How do basin hopping differently from random restarts?“ [sic]]
- Was sagt uns $f''(x_n)$ über die Loss-Landschaft, was charakterisiert es, und wie können wir es nutzen?
- Wie hilft die Approximation der Loss-Landschaft mit verrauschten Schätzungen (Mini-Batches) beim Entkommen aus lokalen Optima?
- Wähle x_0 so, dass es im Basin eines lokalen Minimums liegt; finde verschiedene Wege, mit denen die Optimierungsmethode entkommen kann — wie effektiv ist was?

5.9 Recap of Key Concepts [S. 56]

- Lokale Optimierungsmethoden (wie Newtons Methode) können auf nicht-konvexen Funktionen in lokalen Minima stecken bleiben.
- Globale Optimierungsstrategien (Random Restarts, Basin Hopping, Simulated Annealing, Stochastizität) erkunden den Suchraum gezielt breiter, um das globale Optimum zu finden.
- Die Modifikation von Optimierungsmethoden mit Krümmungsinformation $f''(x)$ erlaubt es, die Abstiegsrichtung genauer zu spezifizieren.
- Die Loss-Landschaft „gehört uns“ — sie ist durch die Wahl der Loss-Funktion und durch Rauschen (etwa stochastische Gradienten) zu konstruieren (loss landscape engineering).

Typische Fehlerquellen & Merksätze

Typische Fehlerquellen:

- **Newton auf f statt f' :** Für Optimierung wird das Nullstellenproblem auf der *Ableitung* gelöst — Update $x_{n+1} = x_n - f'(x_n)/f''(x_n)$ (Gl. (36)/(40)), nicht f/f' .
- **$f'(x^*) = 0$ heißt nicht automatisch Minimum:** Newton auf f' findet Extremstellen ($f' = 0$), die auch Maxima oder Sattelpunkte sein können — immer mit dem Second-Derivative-Test klassifizieren. (Achtung: Das Original druckt auf S. 53 fälschlich „ $f''(x) = 0$ “ — korrekt ist $f'(x) = 0$.)
- **Vorzeichen von f'' ignoriert:** In Regionen mit $f'' < 0$ läuft das reine Newton-Update bergauf zum Maximum; der $|f''|$ -Trick (Gl. (41)) erzwingt Bergab-Schritte.
- **Wahrscheinlichkeiten falsch kombiniert:** Erfolgswahrscheinlichkeit ist $1 - (1 - p)^K$ — nicht $K \cdot p$ (das kann > 1 werden und ignoriert die Unabhängigkeit der Versuche).
- **K abrunden:** Beim Auflösen nach K stets *auf*runden ($28.3 \rightarrow 29$), sonst wird das Wahrscheinlichkeitsziel verfehlt.
- **Random Restarts und Basin Hopping verwechseln:** Restarts setzen die Suche komplett zurück; Basin Hopping springt vom zuletzt gefundenen lokalen Minimum aus weiter ($x_{\text{current}} \leftarrow x^* + \delta$).

Merksätze:

- „Die Minima von f sind die Nullstellen von f' .“

- Konvexität ist die Trennlinie: konvex $\Rightarrow$ lokal = global; die ganze Mühe dieses Kapitels gilt dem nicht-konvexen Fall.
- Erst finden ($f' = 0$), dann klassifizieren (f''): positiv = Tal, negativ = Gipfel, null = unklar.
- Simulated Annealing: heiß = experimentierfreudig ($\exp(-\Delta f/T) \approx 1$), kalt = konservativ.
- Rauschen ist ein Werkzeug, kein Feind: Mini-Batch-Rauschen hilft beim Entkommen — aber der Newton-Schritt verträgt Rauschen schlecht.

6 Thema 6: Numerische Integration (Numerical Integration) [S. 57–70]

6.1 Motivation und Einordnung [S. 57]

Kapitelmotto: „Smooth Operator.“ (Randnotiz [S. 57]: „Melts all your memories and change into gold“ — Songzitat-Anspielung auf Sade.) — Im vorigen Kapitel ging es um globale Optimierungsstrategien zum Entkommen aus lokalen Minima. Man stelle sich nun vor, eine Funktion mit *Tausenden winziger, scharfer lokaler Minima* zu optimieren — wie eine Hochfrequenz-Version von Shekel’s Foxholes. Ein Gradient-Descent-Algorithmus bliebe sofort im ersten mikroskopischen „Schlagloch“ (pothole) stecken. Die Lösung fühlt sich vertraut an, blieb aber bisher wohl unbemerkt: **numerische Integration**, also die näherungsweise Berechnung des bestimmten Integrals (Original-Gl. (42)):

$$I = \int_a^b f(x) dx \quad (42)$$

Gute Gründe, numerische Integration zu verstehen: Sie erlaubt es, die Loss-Landschaft zu *glätten* und Optimierungsmethoden robuster zu machen; und sie findet durch den *Mittelungseffekt* (averaging effect, Fußnote 10 [S. 57]: Keskar et al., „On large-batch training for deep learning: Generalization gap and sharp minima“, 2016) über eine Region tendenziell robustere Optima. Bezug Deep Learning: Stochastic Gradient Descent (SGD) ist der Arbeitspferd-Optimierungsalgorithmus — eine *Monte-Carlo-Methode* (Fußnoten 11, 12 [S. 57]: Bergstra/Bengio, „Random search for hyper-parameter optimization“, JMLR 2012; Gal/Ghahramani, „Dropout as a Bayesian approximation“, 2016), die den Gradienten der Loss-Funktion durch Mittelung über den Gradienten eines zufälligen Mini-Batches schätzt. Ohne numerische Integration wäre das Training auf großskaligen Datensätzen prohibitiv teuer.

6.2 Lernziele (Learning Objectives) [S. 60]

1. Mittelpunktmethode (midpoint method), Trapezregel (trapezoid rule) und Simpson-Regel (Simpson’s rule) für numerische Integration herleiten und anwenden.
2. Zusammengesetzte Regeln (composite rules) für verbesserte Genauigkeit verstehen.
3. Lagrange-Interpolation kennen und wissen, wie sie mit Quadraturregeln (quadrature rules) zusammenhängt.
4. Verstehen, wie der Fluch der Dimensionalität (curse of dimensionality) Monte Carlo motiviert.
5. Monte-Carlo-Integration für hochdimensionale Probleme herleiten und anwenden.
6. Monte-Carlo-Integration anwenden und erkennen, dass Batch-Mittelung (batch averaging) im maschinellen Lernen numerische Integration ist.

6.3 Hands On: Hochfrequente Landschaften und Glättung durch Integration [S. 58–60]

Ausgangslage [S. 58]: Wir wissen bereits, wie man eine kontinuierliche Funktion diskretisiert und wie man mit endlicher Präzision und Systemen mit spärlicher Information an diskreten Punkten umgeht. Die Shekel-Funktion als hochfrequente Funktion war eine Herausforderung für die globale Optimierung, machte die generellen Ansätze aber nicht obsolet — wir diskretisieren und behandeln diese Funktionen wie gewohnt. Als Beispiel dient

$$f(x) = \sin(x) + \sin(2\pi x) \quad \text{auf } [-3, 1], \quad h = 0.2$$

(Figures 29–31 [S. 58]: glatter $\sin(x)$, hochfrequent oszillierendes $\sin(x) + \sin(2\pi x)$ mit vielen lokalen Minima, sowie dieselbe Funktion als Balkendiagramm). Ein genauer Blick auf beliebige lokale Minima zeigt, wie die hochfrequente Loss-Landschaft den Optimierungsalgorithmus ins *nächstgelegene* lokale Minimum stupst (nudges) — z. B. bei $x = 0.6$ oder $x = -0.4$.

Beispiel 6.1: Glättung durch Integration bei $x = -0.4$ (vollständige Rechnung), Gl. (43) [S. 59]

Die direkte Auswertung liefert (Original-Gl. (43)):

$$f(-0.4) = -0.97720 \tag{43}$$

Statt diesen Wert direkt zu verwenden, approximieren wir ihn durch *Integration* über ein Fenster der Breite $h = 0.2$ um -0.4 , also über $[-0.5, -0.3]$ (Trapez-Näherung mit den beiden Randpunkten). Die Randwerte sind

$$f(-0.5) = \sin(-0.5) + \sin(-\pi) \approx -0.47943, \quad f(-0.3) = \sin(-0.3) + \sin(-0.6\pi) \approx -1.24658,$$

und damit

$$h \cdot f(-0.4) \approx \int_{-0.5}^{-0.3} f(x) dx \approx \frac{h}{2} [f(-0.5) + f(-0.3)] \approx 0.2 \cdot \frac{-0.47943 - 1.24658}{2} = -0.17260$$

Division durch h ergibt den geglätteten Funktionswert:

$$f(-0.4) \approx \frac{-0.47943 - 1.24658}{2} = -0.86300$$

[Korrektur ggü. Original, S. 59: Das PDF setzt in diese Rechnung $f(-0.5) = -1.14973$ und $f(-0.3) = -0.97720$ ein und erhält $h \cdot f(-0.4) \approx -0.11285$ bzw. geglättet -1.06347 . Diese Stützwerte sind jedoch falsch beschriftet: -1.14973 ist tatsächlich $f(-0.2)$ und -0.97720 ist $f(-0.4)$ selbst — das Original hat offenbar das Fenster $[-0.4, -0.2]$ gemittelt. Für das beschriebene Fenster $[-0.5, -0.3]$ gelten die obigen Werte. Erst mit ihnen ist die folgende Bewertung in sich stimmig: Der geglättete Wert -0.86300 liegt *über* der Direktauswertung -0.97720 ; mit den PDF-Zahlen läge er fälschlich darunter ($-1.06347 < -0.97720$) und widerspräche der eigenen Aussage „correction towards higher losses.“] **Bewertung [S. 59]:** Mit Blick auf globale Optima ist das eine viel bessere Approximation als die direkte Auswertung bei $x = -0.4$ (die -0.97720 ergab). Die numerische Integration liefert eine *Korrektur der lokalen Loss-Landschaft hin zu höheren Losses an diesem lokalen Minimum* ($-0.86300 > -0.97720$) — das mikroskopische Schlagloch wird zugeschüttet. Figure 32 [S. 59] zeigt den Glättungseffekt auf der ganzen Kurve.

Randnotiz [S. 62]: Die Division des Integrals durch h ergibt einen *gewichteten Durchschnitt* der Funktionswerte — eine bessere Approximation als nur Mittelpunkt oder Endpunkte, die implizit lokale Krümmung und Information über das Funktionsverhalten über das Intervall hinweg reflektiert.

Glättung gezielt einsetzen [S. 59–60]: Die Glättung wird noch deutlicher, wenn man über größere Intervalle integriert — oder, im konstruierten Beispiel, h so wählt, dass die Frequenz des Rauschsignals herausgemittelt wird: Figure 33 [S. 60] nutzt Nachbarmittelung mit $h = 0.48$, sodass $h/2 = 0.24$ die $\sin(2\pi x)$ -Komponente nahezu auslöscht — übrig bleibt eine fast glatte, sinusähnliche Kurve. Randnotiz-Denkfrage [S. 59]: Was passiert, wenn wir über ein Intervall integrieren, das *destruktive Interferenz* der zugrunde liegenden höherfrequenten Sinuskurve ergibt? (Original: „destructive inference“ [sic, gemeint: interference].) An Wellenlängen denken!

Überleitung [S. 59]: Die bisherige Intervall-Approximation ist ziemlich grob — sie stützt sich auf nur zwei Punkte. Das motiviert die systematischen Quadraturregeln.

6.4 Quadraturregeln: Definitionen und Herleitungen [S. 61–64]

Definition 6.1: Riemann-Summe und Mittelpunktsregel (midpoint rule) [S. 61]

Riemann-Summen-Approximation: Teile $[a, b]$ in n Teilintervalle gleicher Breite h und summiere die Beiträge jedes Intervalls:

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x) dx \approx \sum_{i=0}^{n-1} h \cdot f(m_i)$$

mit $x_i = a + ih$ für $i = 0, 1, \dots, n$ und Mittelpunkten m_i .

Mittelpunktsregel: nutzt in diesen zusammengesetzten Intervallen den *Mittelpunkt* — den Durchschnitt der Endpunkte — jedes Intervalls, was oft bessere Ergebnisse liefert als Links- oder Rechts-Endpunkt-Methoden. Intuitiv approximiert der Mittelpunkt die *Krümmung* der Funktion auf dem Intervall besser. Randnotiz [S. 61]: Auf $[a, b]$ ist die Rechteckhöhe beim Mittelpunkt $f(m)$, beim linken Endpunkt $f(a)$, beim rechten $f(b)$.

Satz/Formel 6.1: Trivialste Integral-Approximation: ein Mittelpunkts-Rechteck [S. 61]

Für ein einzelnes Intervall ($n = 1$):

$$I = \int_a^b f(x) dx, \quad I \approx \frac{b-a}{n} \cdot f\left(\frac{a+b}{2}\right), \quad I \approx h \cdot f(m)$$

In eigenen Worten: Das Integral wird durch ein einziges Rechteck ersetzt — „It is easy to see that the height of the rectangle is the value at the midpoint, and the width is h .“ (Höhe = Funktionswert am Mittelpunkt m , Breite = h ; vgl. Figure 34 [S. 61] mit sich berührenden Rechtecken unter $\sin(x)$ auf $[-2, -1]$.) Randnotiz [S. 61]: Eine Verdopplung der Teilintervalle reduzierte den Fehler empirisch um $\sim 4\times$ — ein Hinweis auf $O(h^2)$ -Konvergenz.

Definition 6.2: Trapezregel (trapezoid rule), Gl. (44) [S. 62]

Trapeze sind geometrisch ausdrucksstärker als Rechtecke und können *lineare Änderungen* der Funktion zwischen den Endpunkten erfassen. Für ein Intervall mit $h = b - a$ (Original-

Gl. (44)):

$$\int_a^b f(x) dx \approx \frac{h}{2} [f(a) + f(b)] \quad (44)$$

[**Ergänzung**] Das ist exakt die Fläche des Trapezes unter der Verbindungsgeraden (Sekante) zwischen $(a, f(a))$ und $(b, f(b))$: Mittelwert der beiden Höhen mal Breite (vgl. Figure 35 [S. 62]).

Satz/Formel 6.2: Zusammengesetzte Trapezregel (composite trapezoid rule) — vollständige Herleitung [S. 62]

Trapezregel auf jedes Teilintervall anwenden und summieren ($x_i = a + ih$, $i = 0, 1, \dots, n$):

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x) dx \approx \sum_{i=0}^{n-1} \frac{h}{2} [f(x_i) + f(x_{i+1})]$$

Ausschreiben der Summe:

$$= \frac{h}{2} (f(x_0) + f(x_1)) + \frac{h}{2} (f(x_1) + f(x_2)) + \dots + \frac{h}{2} (f(x_{n-1}) + f(x_n))$$

Zusammenfassen gleicher Terme:

$$= \frac{h}{2} [f(x_0) + 2f(x_1) + 2f(x_2) + \dots + 2f(x_{n-1}) + f(x_n)] = \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right]$$

Gewichtungs-Intuition (1–2–...–2–1): Die Funktionswerte an den Endpunkten a und b erhalten jeweils Gewicht 1, alle inneren Punkte Gewicht 2. Der Grund: Jeder *innere* Punkt trägt zu *zwei* benachbarten Trapezen bei (einmal als rechter, einmal als linker Endpunkt), die Endpunkte nur zu je *einem* Trapez. Das Skript empfiehlt, die eingekreisten Endpunkte („circled endpoints“ [sic]) der Abbildung im Kopf durchzugehen und das doppelte Zählen der inneren Punkte nachzuvollziehen.

Definition 6.3: Simpson-Regel (Simpson's rule) [S. 63]

Die Simpson-Regel kann *Krümmung* erfassen und liefert daher oft eine viel bessere Approximation als die Trapezregel. Der Idee des Geraden-Fits folgend, wird statt einer Geraden durch 2 Punkte ein *quadratisches Polynom durch 3 Punkte* gelegt. Die Simpson-Regel ist *exakt für quadratische Polynome* [Ergänzung: durch Symmetrie sogar exakt für alle Polynome bis Grad 3 — vgl. die S_2 -Korrekturbox weiter unten; das ist kein Widerspruch, sondern eine stärkere Aussage].

Definition 6.4: Lagrange-Interpolation und Quadratur, Gl. (45) [S. 63]

Die **Lagrange-Interpolation** konstruiert das eindeutige Polynom vom Grad $\leq n$, das durch gegebene Stützstellen (nodes) $\{(x_j, y_j)\}_{j=0}^n$ verläuft. Die Basis-Polynome lauten (Original-Gl. (45)):

$$L_j(x) = \prod_{\substack{m=0 \\ m \neq j}}^n \frac{x - x_m}{x_j - x_m} \quad (45)$$

[**Ergänzung**] Jedes L_j ist so gebaut, dass $L_j(x_j) = 1$ und $L_j(x_m) = 0$ für $m \neq j$ — das Interpolationspolynom ist dann einfach $p(x) = \sum_j y_j L_j(x)$. Quadraturregeln entstehen,

indem man statt f dieses Polynom integriert; die Integrale der L_j werden zu den *Gewichten*. Randnotiz-Übung [S. 63]: Mit ein paar Punkten ausprobieren — bei $x = 0$ starten und sehen, wie die Linearfaktoren wirken.

Definition 6.5: Runge-Phänomen (Runge's phenomenon) [S. 64, Randnotiz]

Polynominterpolation *hohen Grades* oszilliert durch Overfitting — es entstehen große Fehler an den *Rändern* des Intervalls. **[Ergänzung]** Deshalb erhöht man in der Praxis nicht den Polynomgrad, sondern setzt zusammengesetzte Regeln niedrigen Grades ein (siehe „Stop at Simpson“ unten); dieselbe Beobachtung steckt hinter der Teaser-Frage [S. 69], warum Hochordnungs-Quadratur instabil wird.

Satz/Formel 6.3: Herleitung der Simpson-Gewichte über Lagrange-Interpolation [S. 63]

Allgemeine Form eines quadratischen Polynoms durch die drei Punkte bei a, m, b (Lagrange-Form):

$$f(x) \approx f(a) \cdot \frac{(x-m)(x-b)}{(a-m)(a-b)} + f(m) \cdot \frac{(x-a)(x-b)}{(m-a)(m-b)} + f(b) \cdot \frac{(x-a)(x-m)}{(b-a)(b-m)}$$

Wir definieren für diese Einzelintervall-Form durchgängig $h := b - a$ als *Gesamtbreite* des Intervalls. Für ein symmetrisches Intervall wird die Fläche unter $f(x)$ am besten approximiert, indem die *Endpunkte* das Gewicht $A = C = \frac{h}{6}$ und der *Mittelpunkt* das Gewicht $B = \frac{4h}{6} = \frac{2h}{3}$ erhalten. Das Skript: Man kann dies verifizieren, indem man die Lagrange-Basispolynome über $[a, b]$ integriert und bestätigt, dass sie genau diese Gewichte liefern (die detaillierte Rechnung wird im Original ausgelassen). Im symmetrischen Intervall sind die Stützstellen äquidistant: $x_0 = a, x_1 = m = \frac{a+b}{2}, x_2 = b$, sodass $m - a = b - m = \frac{h}{2}$ gilt. Somit lautet die **Simpson-Regel für ein einfaches Intervall**:

$$\int_a^b f(x) dx \approx \frac{h}{6} f(a) + \frac{4h}{6} f(m) + \frac{h}{6} f(b) = \frac{h}{6} [f(a) + 4f(m) + f(b)], \quad h = b - a$$

[Anm. zur Konvention: Das Skript schreibt an dieser Stelle „ $m - a = b - m = h$ “ — das ist die *Composite-Konvention*, in der h die Gitterweite (also die *halbe* Breite eines Teilintervall-Paars) bezeichnet; in jener Konvention lautet die Paar-Formel $\frac{2h}{6} [\dots] = \frac{h}{3} [f(a) + 4f(m) + f(b)]$ (siehe Composite-Herleitung unten). Beides zugleich — $m - a = h$ und Faktor $\frac{h}{6}$ — kann nicht stimmen; wer die Gitterweite in die $\frac{h}{6}$ -Form einsetzt, erhält den halben Wert. Hier gilt einheitlich $h = b - a$.] **In eigenen Worten / [Ergänzung]:** Die Gewichte 1 : 4 : 1 (nach Ausklammern von $h/6$) spiegeln wider, dass der Mittelpunkt die Krümmungsinformation trägt und daher viermal so stark zählt wie jeder Randpunkt. Randnotiz-Denkfrage [S. 64]: Was passiert, wenn man eine *lineare* Funktion mit einer Quadratik approximiert? (Vgl. Figure 36 [S. 63]: zwei Parabel-Segmente unter $\sin(x)$ auf $[-2, -1.5]$ und $[-1.5, -1]$.)

Satz/Formel 6.4: Zusammengesetzte Simpson-Regel (composite Simpson's rule) — vollständige Herleitung [S. 63–64]

Simpson-Regel auf jedes *Paar* von Teilintervallen anwenden und summieren (Annahme: n ist gerade, $x_i = a + ih$). Mit Gitterweite h und Paarbreite $2h$ beträgt der Faktor pro

Paar $\frac{2h}{6} = \frac{h}{3}$:

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x) dx \approx \sum_{k=0}^{n/2-1} \frac{h}{3} [f(x_{2k}) + 4f(x_{2k+1}) + f(x_{2k+2})]$$

Zusammenfassen (innere gerade Punkte sind Endpunkt zweier Paare und zählen doppelt, ungerade Punkte sind Paarmittelpunkte mit Gewicht 4):

$$\begin{aligned} &= \frac{h}{3} \left[f(x_0) + 4 \sum_{k=0}^{n/2-1} f(x_{2k+1}) + 2 \sum_{k=1}^{n/2-1} f(x_{2k}) + f(x_n) \right] \\ &= \frac{h}{3} \left[f(x_0) + 4 \sum_{\substack{i=1 \\ i \text{ ungerade}}}^{n-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ gerade}}}^{n-2} f(x_i) + f(x_n) \right] \end{aligned}$$

[Korrektur ggü. Original, S. 64: Das PDF druckt in den Zwischenzeilen der Herleitung pro Teilintervall-Paar den Faktor $\frac{h}{6}$; korrekt ist bei Gitterweite h und Paarbreite $2h$ der Faktor $\frac{2h}{6} = \frac{h}{3}$ pro Paar. Die im PDF gedruckte Endformel $\frac{h}{3} [f(x_0) + 4 \sum_{i \text{ ungerade}} f(x_i) + 2 \sum_{i \text{ gerade}} f(x_i) + f(x_n)]$ ist korrekt; nur die Zwischenzeilen sind inkonsistent (Faktorwechsel $h/6 \rightarrow h/3$ ohne Begründung).]

Gewichtsmuster: 1–4–2–4– $\dots$ –4–1.

Practical advice: Stop at Simpson [S. 64]. Für höhere Genauigkeit zusammengesetzte Regeln mit *mehr Teilintervallen* verwenden — nicht den Polynomgrad erhöhen: Hohe Grade ($n \geq 8$) entwickeln *negative Gewichte* und werden instabil (vgl. Runge-Phänomen).

Satz/Formel 6.5: Fehlerordnungen der Quadraturmethoden (Tabelle 3) [S. 64]

Vergleich der Quadraturmethoden-Genauigkeiten: Höherwertige Methoden erreichen eine gegebene Genauigkeit mit weniger Funktionsauswertungen. h ist die Schrittweite, n die Anzahl der Auswertungspunkte bei Gauß-Quadratur.

Methode (Method)	Fehler Einzelintervall (Single Interval Error)	Akkumulierter Fehler (Accumulated Error)
Left/Right	$O(h^2)$	$O(h)$
Midpoint	$O(h^3)$	$O(h^2)$
Trapezoid	$O(h^3)$	$O(h^2)$
Simpson's 1/3	$O(h^5)$	$O(h^4)$
Allgemein (Gaussian Quadrature)	$O(h^{2n})$	$O(h^{2n-1})$

In eigenen Worten / [Ergänzung]: Der Einzelintervall-Fehler ist der Fehler *eines* Teilintervalls; beim Zusammensetzen von $n \sim 1/h$ Intervallen akkumulieren sich die Fehler, wodurch eine Ordnung verloren geht (z. B. $O(h^3) \cdot \frac{1}{h} = O(h^2)$). Midpoint und Trapezoid sind beide *second-order accurate* ($O(h^2)$ akkumuliert) — das erklärt, warum sie in den Beispielen ähnliche Fehler liefern können. Simpson gewinnt durch die Krümmungserfassung gleich zwei Ordnungen.

6.5 Monte-Carlo-Integration und der Fluch der Dimensionalität [S. 64–65]

Definition 6.6: Fluch der Dimensionalität (curse of dimensionality) [S. 64]

Für d -dimensionale Integrale benötigt deterministische Quadratur mit N Punkten pro Dimension insgesamt N^d Punkte. „There is no way to do this, let alone iterate on it during iterative optimization.“ — Das ist nicht machbar, erst recht nicht wiederholt innerhalb einer iterativen Optimierung. Randnotiz [S. 64] (drastisches Beispiel): Ein neuronales Netz mit $d = 10^6$ (1 Million) Parametern bräuchte N^{10^6} Gitterpunkte — schon $N = 2$ ergibt 2^{10^6} , eine Zahl mit 300 000 Stellen.

Definition 6.7: Monte Carlo, i.i.d.-Samples und die Domäne Ω [S. 57, 64–65, Randnotizen]

Monte Carlo ist eine Strategie, die Probleme im Wesentlichen durch *zufälliges Sampling* löst („Embrace the beauty“ [S. 57]). **i.i.d.** (independent and identically distributed) bedeutet: Alle $\mathbf{X}_i$ sind unkorreliert und stammen aus derselben zugrunde liegenden Verteilung. Ω ist die *Integrationsdomäne*, $|\Omega|$ ihre Länge bzw. ihr Volumen: Für ein 1D-Integral über $[a, b]$ ist $|\Omega| = b - a$; in höheren Dimensionen das Produkt der Längen jeder Dimension.

Satz/Formel 6.6: Integral als Erwartungswert und Monte-Carlo-Schätzer, Gl. (46)–(47) [S. 64–65]

Random Sampling erlaubt es, das Integral *ohne volles Gitter* zu schätzen. Grundlage ist die Identität (Original-Gl. (46)):

$$I = \int_{\Omega} f(\mathbf{x}) d\mathbf{x} = |\Omega| \cdot \mathbb{E}_{\mathbf{X} \sim \text{Uniform}(\Omega)}[f(\mathbf{X})] \quad (46)$$

In eigenen Worten: Das bestimmte Integral ist gleich dem Domänenvolumen mal dem *Durchschnittswert* von f unter gleichverteiltem Ziehen aus Ω . Praktisch motiviert das einen Sampling-Schätzer: Punkte gleichverteilt in Ω ziehen, f dort auswerten, die Resultate mitteln und mit $|\Omega|$ multiplizieren. Der **Monte-Carlo-Schätzer** (im Beispiel: uniformes Sampling auf $[-2, -1]$, $X \sim \text{Uniform}(-2, -1)$) lautet (Original-Gl. (47)):

$$\hat{I}_N = \frac{|\Omega|}{N} \sum_{i=1}^N f(\mathbf{X}_i), \quad \mathbf{X}_i \stackrel{\text{i.i.d.}}{\sim} \text{Uniform}(\Omega) \quad (47)$$

[Anm.: Das PDF druckt links „ I “; gemeint ist der Schätzer $\hat{I}_N$ — das exakte Integral I ist Gl. (46), und Gl. (48) des Skripts verwendet selbst $\hat{I}_N$.] *Randnotiz [S. 65]:* Die Mittelpunktsregel ist ein *Spezialfall* von Monte Carlo mit $N = 1$ Sample am Mittelpunkt. — Figure 37 [S. 65] illustriert das Monte-Carlo-Sampling auf dem Integrationsintervall. [Anm.: Die Bildunterschrift von Figure 37 ist im Original offenbar ein Copy-Paste-Fehler — sie wiederholt wortgleich die Caption von Figure 34 zur Mittelpunktsregel („midpoint rule integrals ... bars now touch“), obwohl die Abbildung im Monte-Carlo-Kontext steht.]

Satz/Formel 6.7: Eigenschaften des Monte-Carlo-Schätzers: Bias, Varianz, Standardfehler, Gl. (48)–(51) [S. 65, 68]

Erwartungstreue (unbiasedness), Gl. (48): Weil der Erwartungswert *linear* ist, ist der Schätzer erwartungstreu (unbiased) — sein Erwartungswert ist gleich dem wahren Integral:

$$\mathbb{E}[\hat{I}_N] = I \quad (48)$$

[Anm. zur Formulierung des Skripts (S. 65): Das Original schreibt „unbiased ... for the number of $N \rightarrow \infty$ “ — das ist begrifflich unsauber. Erwartungstreue $\mathbb{E}[\hat{I}_N] = I$ gilt für *jedes endliche* N (schon für $N = 1$); eine $N \rightarrow \infty$ -Aussage ist erst die *Konsistenz* $\hat{I}_N \rightarrow I$ (der Schätzer konvergiert gegen den wahren Wert, vgl. die $O(1/\sqrt{N})$ -Konvergenz unten). Das Skript vermengt hier beide Begriffe.] **[Ergänzung]** „Unbiased“ heißt: Im Mittel über viele Wiederholungen trifft der Schätzer den wahren Wert — er streut nur darum, und zwar bei jedem festen Stichprobenumfang N .

Varianz, Gl. (49): Die Varianz — ein Maß für die Streuung des Schätzers um seinen Mittelwert bzw. dafür, wie groß der Fehler des Schätzers werden kann — folgt aus der *Unabhängigkeit* der Samples:

$$\text{Var}(\hat{I}_N) = \frac{|\Omega|^2}{N} \text{Var}(f(\mathbf{X})) = \frac{|\Omega|^2 \sigma_f^2}{N} \quad (49)$$

mit $\sigma_f^2 = \text{Var}(f(\mathbf{X}))$.

Standardfehler (standard error), Gl. (50):

$$\text{SE}(\hat{I}_N) = \frac{|\Omega| \sigma_f}{\sqrt{N}} \quad (50)$$

Dimensionsunabhängig: Für großes N konvergiert der Sampling-Fehler mit $O(1/\sqrt{N})$ — das macht Monte Carlo in hohen Dimensionen attraktiv. Quadraturregeln sind in ihren *Konvergenzraten* überlegen, aber der Konvergenzaufwand explodiert mit der Dimension, da N^d Punkte nötig sind, um dieselbe Genauigkeit zu halten. Der zentrale Trade-off dieses Kapitels.

Convergence in SGD [S. 65]: Mini-Batch-Gradienten in SGD *sind* Monte-Carlo-Schätzungen des wahren Gradienten, basierend auf den in einen Batch gezogenen Samples. Eine Erhöhung der Batch-Größe b reduziert die Varianz grob um $1/b$ — direkt analog zur $1/N$ -Varianz-Skalierung oben. Diese Verbindung erklärt, warum die Batch-Größe den Noise-Varianz-Trade-off im Training kontrolliert (Figure 38 [S. 65]: kleinere Batches $\Rightarrow$ größere Varianz im Gradienten-Schätzer, sichtbar als breiteres Band der Trainings-Loss-Kurve; Quelle: Stanford CS231n). Zusatznotiz: Als positiver Nebeneffekt sind die Rechenkosten pro Update-Schritt n/b -mal billiger — für $n = 10^6$ und $b = 100$ macht SGD 10 000 Iterationen mit dem Rechenaufwand, den GD für *eine* braucht.

Erwartungstreuer Stichprobenvarianz-Schätzer, Gl. (51) [S. 68]: In der Praxis ist σ_f^2 unbekannt und wird aus den Samples geschätzt:

$$\text{Var}(f(\mathbf{X})) \approx \frac{1}{N-1} \sum_{i=1}^N (f(X_i) - \bar{f})^2 \quad (51)$$

[Ergänzung] Der Nenner $N - 1$ (statt N) korrigiert die Verzerrung, die entsteht, weil das Stichprobenmittel $\bar{f}$ selbst aus denselben Daten geschätzt wird (Bessel-Korrektur).

6.6 Durchgerechnete Beispiele: $\int_{-2}^{-1} \sin(x) dx$ mit allen Methoden [S. 66–68]

Beispiel 6.2: Referenzwert: das exakte Integral [S. 66]

Für die Fehlerquantifizierung zuerst der exakte Wert (Stammfunktion von $\sin$ ist $-\cos$):

$$\tilde{I} = \int_{-2}^{-1} \sin(x) dx = [-\cos(x)]_{-2}^{-1} = -\cos(-1) + \cos(-2) \approx -0.9564491424$$

[Ergänzung] Zahlenwerte: $\cos(-1) = \cos(1) \approx 0.5403023$, $\cos(-2) = \cos(2) \approx -0.4161468$; also $\tilde{I} \approx -0.5403023 - 0.4161468 = -0.9564491$.

Beispiel 6.3: Mittelpunktsregel von Hand: $n = 1$ und $n = 2$ [S. 66]

$n = 1$ Intervall: Mittelpunkt $m = \frac{-2+(-1)}{2} = -1.5$:

$$\hat{I} \approx (b - a) \cdot f(m) = 1 \cdot \sin(-1.5) \approx -0.99749$$

Fehler: $|\tilde{I} - \hat{I}| \approx 0.04104$ — die kleine Differenz zwischen Mittelpunkt-Schätzung und wahren Wert.

$n = 2$ Intervalle: Zwei Mittelpunkte bei -1.75 und -1.25 , $h = 0.5$:

$$\hat{I} \approx h \cdot [f(-1.75) + f(-1.25)] = 0.5 \cdot [\sin(-1.75) + \sin(-1.25)] = 0.5 \cdot (-1.932971) \approx -0.966485$$

Fehler: $|\tilde{I} - \hat{I}| \approx 0.010036$ — deutlich kleiner als bei $n = 1$ (0.04104); das illustriert, wie Composites bzw. eine größere Intervallzahl die Approximation verbessern.

[Korrektur ggü. Original, S. 66: Das PDF druckt „ $\hat{I} \approx -0.927228$ “ und Fehler „ ≈ 0.02922 “; korrekt ist $\hat{I} = 0.5 \cdot [\sin(-1.75) + \sin(-1.25)] = 0.5 \cdot (-1.932971) \approx -0.966485$ mit Fehler $|\tilde{I} - \hat{I}| \approx 0.010036$ ($\sin(-1.75) \approx -0.983986$, $\sin(-1.25) \approx -0.948985$). Die qualitative Aussage „Fehler sinkt mit n “ bleibt richtig.]

Randnotiz [S. 66]: Weiter ausprobieren lohnt sich — die Approximation an den Endpunkten des Intervalls berechnen und den Fehler vergleichen.

Beispiel 6.4: Trapezregel von Hand: T_2 [S. 66–67]

Trapezregel mit $n = 2$ Intervallen; Stützstellen -2 , -1.5 , -1 , $h = 0.5$ (Gewichte 1–2–1):

$$T_2 = \frac{h}{2} [f(-2) + 2f(-1.5) + f(-1)] = 0.25 \cdot [-0.90930 + 2(-0.99749) + (-0.84147)] = 0.25 \cdot (-3.74575) \approx$$

Fehler: $|\tilde{I} - T_2| \approx 0.02001$.

[Korrektur ggü. Original, S. 67: Das PDF druckt $T_2 \approx -0.927228$ — das ist zufällig genau der gedruckte Midpoint- $n=2$ -/Monte-Carlo-Wert (offenbar ein Kopierfehler). Die im PDF selbst gedruckte Formel ergibt $0.25 \cdot (-3.74575) \approx -0.93644$ mit Fehler ≈ 0.02001 .]

[Anmerkung: Der Original-Kommentar „The error is the same as the midpoint rule with $n = 2$ intervals“ ist damit hinfällig — er beruht auf zwei fehlerhaften Werten (dem falschen Trapez-Wert -0.927228 und dem laut Erratum ebenfalls falschen Midpoint-Wert, korrekt -0.966485 mit Fehler 0.010036, siehe oben). Tatsächlich gilt hier $0.02001 \neq 0.010036$. Die generelle Aussage, dass beide Methoden zweiter Ordnung genau sind (second-order accurate, vgl. Tabelle 3) und daher Fehler ähnlicher Größenordnung liefern, bleibt korrekt.]

Beispiel 6.5: Simpson-Regel von Hand: S_2 [S. 67]

Simpson-Regel mit $n = 2$ Intervallen; Stützstellen $-2, -1.5, -1, h = 0.5$ (Gewichte 1–4–1, Composite-Faktor $h/3$):

$$S_2 = \frac{h}{3} [f(-2) + 4f(-1.5) + f(-1)] = \frac{0.5}{3} \cdot [-0.90930 + 4(-0.99749) + (-0.84147)] = \frac{1}{6} \cdot (-5.74073) \approx -0.956788$$

Fehler: $|\tilde{I} - S_2| \approx 3.4 \cdot 10^{-4}$ — sehr klein, aber nicht null; um etwa zwei Größenordnungen besser als Midpoint/Trapez und konsistent mit der $O(h^5)$ -Ordnung aus Tabelle 3.

[Korrektur ggü. Original, S. 67: Das PDF druckt $S_2 \approx -0.9564491424$ (also exakt den wahren Integralwert $\tilde{I}$) mit Fehler ≈ 0 und kommentiert „which is essentially zero, illustrating that Simpson’s rule is exact for this particular integral“. Die im PDF selbst gedruckte Formel ergibt jedoch $\frac{1}{6} \cdot (-5.74073) \approx -0.956788$ mit Fehler $\approx 3.4 \cdot 10^{-4}$. Simpson ist exakt für Polynome bis Grad 3; $\sin(x)$ ist kein solches Polynom, daher bleibt ein kleiner, aber nicht verschwindender Fehler. Die qualitative Kernaussage — Simpson ist deutlich genauer als Midpoint/Trapez — bleibt richtig.]

Randnotiz [S. 67]: Mit mehr Intervallen probieren — sehen, wie der Fehler sinkt, und prüfen, ob man die Composite-Varianten von Simpson- und Trapezregel beherrscht.

Beispiel 6.6: Monte Carlo von Hand: $N = 4$ Samples inkl. Varianz und Standardfehler [S. 67–68]

Monte-Carlo-Schätzung für $\int_{-2}^{-1} \sin(x) dx$ mit $N = 4$ Zufallssamples — ein ähnlicher Rechenaufwand wie bei der Trapezmethode. Angenommene Zufallssamples (vollständig):

$X_1 = -1.95,$	$f(X_1) = \sin(-1.95) \approx -0.92895,$
$X_2 = -1.55,$	$f(X_2) = \sin(-1.55) \approx -0.99978,$
$X_3 = -1.15,$	$f(X_3) = \sin(-1.15) \approx -0.91276,$
$X_4 = -1.05,$	$f(X_4) = \sin(-1.05) \approx -0.86742.$

[Anm.: $\sin(-1.95) = -0.9289597$, korrekt gerundet -0.92896 ; das PDF druckt -0.92895 (letzte Stelle). Alle Folgewerte sind konsistent mit dem gedruckten Wert gerechnet und auf der angegebenen Genauigkeit unverändert.] Monte-Carlo-Schätzung (Gl. (47) mit $|\Omega| = b - a = 1$):

$$\hat{I} = (b - a) \cdot \frac{1}{N} \sum_{i=1}^N f(X_i) = 1 \cdot \frac{1}{4} (-0.92895 - 0.99978 - 0.91276 - 0.86742) \approx -0.927228$$

[S. 68] Fehler: $|\tilde{I} - \hat{I}| \approx 0.02922$, laut Skript „the same as the midpoint method“.

[Anmerkung — Folgeänderung zu Erratum S. 67/68: Der Vergleich „the same as the midpoint method“ beruht auf dem im PDF gedruckten, laut Errata fehlerhaften Midpoint- $n=2$ -Wert -0.927228 /Fehler 0.02922 ; mit dem korrigierten Midpoint-Wert (-0.966485 , Fehler 0.010036) gilt er nicht mehr. Der Monte-Carlo-Wert -0.927228 und sein Fehler 0.02922 selbst sind für diese Samples arithmetisch korrekt.]

Alternative Fehlerschätzung über den Standardfehler (vollständige Rechnung)

[S. 68]: Mit Stichprobenmittel $\bar{f} \approx -0.927228$ lauten die vier Abweichungsquadrate

$$(f(X_i) - \bar{f})^2:$$

$$(-0.92895 + 0.927228)^2 = (-0.001722)^2 \approx 0.0000030,$$

$$(-0.99978 + 0.927228)^2 = (-0.072552)^2 \approx 0.0052638,$$

$$(-0.91276 + 0.927228)^2 = (0.014468)^2 \approx 0.0002093,$$

$$(-0.86742 + 0.927228)^2 = (0.059808)^2 \approx 0.0035770.$$

Summe ≈ 0.009053 ; der erwartungstreue Stichprobenvarianz-Schätzer (Gl. (51), Nenner $N - 1 = 3$) ergibt damit:

$$\sigma_f^2 = \text{Var}(f(\mathbf{X})) \approx \frac{0.009053}{3} \approx 0.003018, \quad \sigma_f \approx 0.05493$$

$$\text{SE}(\hat{I}_N) \approx \frac{|\Omega| \sigma_f}{\sqrt{N}} = \frac{1 \cdot 0.05493}{2} \approx 0.02747$$

[Korrektur ggü. Original, S. 68: Das PDF druckt $\sigma_f^2 \approx 0.003036$, $\sigma_f \approx 0.05512$ und $\text{SE} \approx 0.02756$ — diese Werte sind aus den angegebenen vier Samples nicht reproduzierbar (vermutlich aus gerundeten Zwischenwerten entstanden, Abweichung $\sim 0.6\%$). Die obige Rechnung zeigt die korrekten Werte $0.003018/0.05493/0.02747$.] **[Ergänzung]** Der Standardfehler 0.02747 passt gut zum tatsächlichen Fehler 0.02922 — die Fehlerschätzung aus den Samples selbst (ohne Kenntnis des wahren Werts!) ist hier also realistisch.

Folgefrage [S. 68]: Was passiert, wenn wir N auf 16 erhöhen? Der Fehler sollte um den Faktor 2 sinken, da der Standardfehler mit $1/\sqrt{N}$ skaliert ($\sqrt{16}/\sqrt{4} = 2$).

6.7 Wissenswertes aus Randnotizen und Fußnoten

- **Verdopplung der Teilintervalle [S. 61]:** reduzierte den Fehler empirisch um $\sim 4\times$ — Hinweis auf $O(h^2)$ -Konvergenz.
- **Division durch h [S. 62]:** macht aus dem Integral einen gewichteten Durchschnitt der Funktionswerte — implizit wird lokale Krümmung reflektiert.
- **Lagrange-Übung [S. 63]:** mit Punkten ausprobieren, bei $x = 0$ starten und die Wirkung der Linearfaktoren beobachten.
- **Denkfrage [S. 64]:** Was passiert, wenn man eine lineare Funktion mit einer Quadratik approximiert?
- **Runge-Phänomen [S. 64]:** Interpolation hohen Grades oszilliert (Overfitting), große Fehler an den Intervallrändern — darum „Stop at Simpson“ und Composite-Regeln.
- **2^{10^6} -Beispiel [S. 64]:** Netz mit 10^6 Parametern: Gitter mit $N = 2$ Punkten pro Dimension ergibt bereits 2^{10^6} Punkte (300 000 Stellen) — Curse of Dimensionality in Reinform.
- **Mittelpunktsregel als Monte Carlo [S. 65]:** Spezialfall mit $N = 1$ Sample am Mittelpunkt.
- **SGD als Monte Carlo / Batch-Size-Varianz [S. 65]:** Mini-Batch- Gradienten sind MC-Schätzer; Varianz $\sim 1/b$; Kosten pro Update n/b -mal billiger ($n = 10^6$, $b = 100$: 10 000 SGD-Iterationen für den Preis einer GD-Iteration); kleinere Batches $\Rightarrow$ breiteres Loss-Band (Figure 38, Stanford CS231n).
- **Was-wäre-wenn-Frage [S. 59]:** Integration über ein Intervall mit destruktiver Interferenz [sic: „inference“] der höherfrequenten Sinuskomponente — an Wellenlängen denken

(vgl. $h = 0.48$ -Beispiel).

- **Code [S. 68]:** https://github.com/Quillstacks/lecturecode_numericalmethods.git
- **Teaser [S. 69–70]:** Bisher bezog sich Stabilität auf die Sensitivität der numerischen Lösung gegenüber kleinen Eingabeänderungen; man kann aber auch über die Stabilität der *Methode selbst* sprechen — beim Approximieren verschiedener Funktionstypen. Randnotiz-Denkfrage [S. 69]: Warum werden Hochordnungs-Quadraturmethoden instabil, wenn sie Polynome *niedrigen* Grades approximieren? (Bezug: Runge-Phänomen / negative Gewichte.) [Anm.: Das angekündigte Folgekapitel ist im PDF nicht enthalten — „unfinished lecture notes“; die Materialien decken Kap. 1–6 ab.]

6.8 Übungsaufgaben und Selbstreflexionsfragen des Skripts

Übungsaufgaben [S. 59–68] (nur Fragen):

- Approximation an den *Endpunkten* des Intervalls berechnen und den Fehler mit der Mittelpunktsregel vergleichen [S. 66].
- Mit mehr Intervallen rechnen: Wie sinkt der Fehler? Composite-Varianten von Simpson- und Trapezregel sicher beherrschen [S. 67].
- Lagrange-Basis: Mit einigen Punkten durchspielen, Start bei $x = 0$ — wie wirken die Linearfaktoren? [S. 63]
- Was-wäre-wenn: Integration über ein Intervall, das destruktive Interferenz der zugrunde liegenden höherfrequenten Sinuskurve ergibt — was passiert? (An Wellenlängen denken.) [S. 59]
- Was passiert, wenn man eine lineare Funktion mit einer Quadratik approximiert? [S. 64]
- Folgefrage zur MC-Fehlerschätzung [S. 68]: N auf 16 erhöhen — der Fehler sollte um Faktor 2 sinken ($1/\sqrt{N}$ -Skalierung). Kurz erläutern, wie der Fehler des Monte-Carlo-Schätzers dimensionsunabhängig (dimension independant [sic]) ist.
- *Enter higher dimensions on your machine* [S. 68]: Monte-Carlo- Integration für ein d -dimensionales Integral implementieren (z. B. multivariate Gauß-Funktion über einen Hyperwürfel); d langsam erhöhen und das Fehlerverhalten verschiedener Methoden beobachten; beobachten, wie gitterbasierte Quadratur mit wachsendem d rechnerisch infeasible wird; optional ähnliche Effekte in der Optimierung untersuchen, indem Gradientenabstieg und Mini-Batch-SGD auf einer hochdimensionalen Funktion implementiert werden — nur die Rechenzeiten beobachten.

Selbstreflexionsfragen [S. 69] (alle neun):

- Wie vergleichen sich die Fehler von Mittelpunkts-, Trapez-, Simpson-Regel und Monte Carlo?
- Warum hat die Mittelpunktsregel eine bessere Genauigkeit als die Links- oder Rechts-Endpunkt-Regeln?
- Wie verbessern Composite-Methoden die Genauigkeit, und was ist der Trade-off? [Originalfrage grammatisch fehlerhaft: „How does composite methods ...“ [sic]]
- Warum scheitern deterministische Quadraturmethoden in hohen Dimensionen?
- Inwiefern ist die Mittelpunktsregel ein Spezialfall der Monte-Carlo-Schätzung?
- Was ist die Intuition dahinter, dass Monte Carlo in hohen Dimensionen nicht explodiert?
- Wann würde man die Simpson-Regel verwenden, wann Monte Carlo?

- Wie hängt das Mini-Batch-Mittel in SGD mit der Monte-Carlo-Schätzung zusammen?
- Wie hilft die Gradientenschätzung in SGD beim Entkommen aus lokalen Minima?

6.9 Recap of Key Concepts [S. 69]

- Deterministische Quadraturmethoden (Mittelpunkt, Trapez, Simpson) approximieren Integrale über *gewichtete Summen* von Funktionswerten an spezifischen Punkten — genau und schnell konvergierend für glatte, niedrigdimensionale Probleme.
- Monte-Carlo-Integration schätzt Integrale durch Mittelung von Funktionswerten an *zufälligen* Samples — erwartungstreu (unbiased), Konvergenzrate $O(1/\sqrt{N})$, unabhängig von der Dimensionalität des Problems.
- In SGD sind Mini-Batch-Gradienten Monte-Carlo-Schätzungen des wahren Gradienten.

Typische Fehlerquellen & Merksätze

Typische Fehlerquellen:

- **Composite-Faktoren verwechseln:** Trapez $\frac{h}{2}$ mit Gewichten 1–2–...–2–1; Simpson $\frac{h}{3}$ mit Gewichten 1–4–2–4–...–4–1. Beim Composite Simpson gilt pro Teilintervall-Paar (Breite $2h$) der Faktor $\frac{2h}{6} = \frac{h}{3}$ — nicht $\frac{h}{6}$ (so der Druckfehler des Originals, S. 64); $\frac{h}{6}$ gehört zur Einzelintervall-Form $\frac{h}{6}[f(a) + 4f(m) + f(b)]$ mit $h = b - a$.
- **Simpson braucht gerades n :** Die Composite-Simpson-Regel setzt eine *gerade* Anzahl von Teilintervallen voraus (Paarbildung!).
- **Gewichte 4 und 2 vertauschen:** Gewicht 4 für die *ungeraden* Indizes (Paarmittelpunkte), Gewicht 2 für die inneren *geraden* Indizes (geteilte Paarendpunkte).
- **Mittelpunkte vs. Endpunkte:** Die Mittelpunktsregel wertet bei den Intervallmitten aus ($-1.75, -1.25$), die Trapezregel an den *Endpunkten* ($-2, -1.5, -1$) — nicht mischen. (Genau hier liegt der Originalfehler S. 66: korrekt ist Midpoint $n=2 \approx -0.966485$, Fehler ≈ 0.010036 .)
- **Polynomgrad hochschrauben statt Composite:** Hohe Grade ($n \geq 8$) entwickeln negative Gewichte und werden instabil (Runge-Phänomen) — „Stop at Simpson“, lieber mehr Teilintervalle.
- **$|\Omega|$ vergessen:** Beim MC-Schätzer das Stichprobenmittel mit dem Domänenvolumen $|\Omega|$ multiplizieren; bei der Stichprobenvarianz durch $N - 1$ (nicht N) teilen (Gl. (51)).
- **MC-Konvergenz überschätzen:** Der Fehler fällt nur mit $1/\sqrt{N}$ — für eine zusätzliche Dezimalstelle braucht man $100\times$ mehr Samples; der Gewinn liegt in der *Dimensionsunabhängigkeit*, nicht im Tempo.

Merksätze:

- Integration glättet: Mittelung über ein Fenster schüttet mikroskopische Schlaglöcher der Loss-Landschaft zu („Smooth Operator“).
- Quadratur-Hierarchie: Rechteck (konstant) $\rightarrow$ Trapez (linear) $\rightarrow$ Simpson (quadratisch) — jede Stufe erfasst mehr Funktionsverhalten.
- Fehlerordnungen (akkumuliert): Left/Right $O(h)$, Midpoint/Trapez $O(h^2)$, Simpson $O(h^4)$ (Tabelle 3).

- Niedrige Dimension + glatt $\Rightarrow$ Quadratur; hohe Dimension $\Rightarrow$ Monte Carlo (N^d vs. $O(1/\sqrt{N})$).
- Integral = Volumen $\times$ Mittelwert (Gl. (46)) — deshalb funktioniert Sampling überhaupt.
- SGD *ist* Monte Carlo: Mini-Batch-Mittelung = numerische Integration; Batch-Größe steuert die Varianz ($\sim 1/b$).